



TRABAJO FIN DE GRADO

INGENIERÍA INFORMÁTICA

Sistemas Inteligentes de Detección de Amenazas en Redes mediante Machine Learning

Autor

Eduardo Domínguez López

Directores

Ignacio Javier Pérez Gálvez

José Ramón Trillo Vílchez



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Septiembre de 2026



Sistemas Inteligentes de Detección de Amenazas en Redes mediante Machine Learning

Autor

Eduardo Domínguez López

Directores

Ignacio Javier Pérez Gálvez

José Ramón Trillo Vílchez

Sistemas Inteligentes de Detección de Amenazas en Redes mediante Machine Learning

Eduardo Domínguez López

Palabras clave: Sistemas de detección de intrusiones en red, DDoS, Aprendizaje automático, Aprendizaje supervisado, Árbol de decisión, F1

Resumen

Los sistemas de detección de intrusiones en red (NIDS) son un pilar fundamental de la seguridad de las redes informáticas puesto que permiten identificar de manera automatizada cualquier tipo de amenaza. En este Trabajo Fin de Grado, se presentan varios modelos de inteligencia artificial capaces de detectar ataques de denegación de servicio distribuido (DDoS) en redes informáticas y generar alertas automáticas ante posibles amenazas. Para ello, se ha recopilado una versión reducida del conjunto de datos público CIC-DDoS2019, sobre la que se ha aplicado una metodología que combina limpieza y análisis exploratorio de los datos, codificación de características, transformaciones de datos y técnicas de selección de características. Seguidamente se desarrollan dos fases de experimentación: una primera sobre un conjunto de referencia de 8000 muestras de tráfico de red para establecer un punto de partida, y una segunda sobre un conjunto de 225745 muestras para validar y ampliar las conclusiones obtenidas. Usando un enfoque de aprendizaje supervisado, se entrenan y comparan distintos algoritmos de aprendizaje automático, entre ellos, la Regresión Logística, Máquinas de Vector Soporte, Perceptrón Multicapa y Árboles de Decisión. Se comparan ambos experimentos mediante métricas como F1, tasa de falsos positivos, errores de clasificación o coste computacional. Los resultados revelan que se requiere de una representación adecuada de datos de tráfico de red para obtener un elevado rendimiento sobre los modelos sensibles a escala. El modelo seleccionado es un árbol de decisión entrenado sobre un conjunto reducido de 43 características, que alcanza un F1 del 99,9% y una tasa de falsos positivos del 0,1%. Se demuestra así, que modelos de complejidad moderada proporcionan resultados competitivos cuando se realiza un proceso exhaustivo de análisis y preparación de los datos.

Intelligent Network Threat Detection Systems Using Machine Learning

Eduardo Domínguez López

Keywords: Network intrusion detection systems, DDoS, Machine learning, Supervised learning, Decision tree, F1

Abstract

Network intrusion detection systems (NIDS) are a fundamental pillar of computer network security since they allow any type of threat to be identified automatically. This Bachelor's Thesis presents several artificial intelligence models capable of detecting Distributed Denial-of-Service (DDoS) attacks in computer networks and generating automatic alerts for potential threats. To this end, a reduced version of the public CIC-DDoS2019 dataset has been collected, on which a methodology combining data cleaning and exploratory analysis, feature encoding, data transformations, and feature selection techniques has been applied. Subsequently, two phases of experimentation are developed: the first one on a reference set of 8000 network traffic samples to establish a baseline. The second one on a set of 225,745 samples to validate and expand the findings. Using a supervised learning approach, different machine learning algorithms are trained and compared, including Logistic Regression, Support Vector Machines, Multilayer Perceptron, and Decision Trees. Both experiments are compared using metrics such as F1, false positive rate, classification errors, and computational cost. The results reveal that an appropriate representation of network traffic data is required to achieve high performance on scale-sensitive models. The selected model is a decision tree trained on a reduced set of 43 features, achieving an F1 of 99.9% and a false positive rate of 0.1%. This demonstrates that models of moderate complexity provide competitive results when an exhaustive process of data analysis and preparation is carried out.

Yo, **Eduardo Domínguez López**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 78267949F, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Eduardo Domínguez López

Granada a 1 de septiembre de 2026.

D. **Ignacio Javier Pérez Gálvez**, Profesor del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

D. **José Ramón Trillo Vílchez**, Profesor del Departamento de Informática e Ingeniería de Sistemas de la Universidad de Zaragoza.

Informan:

Que el presente trabajo, titulado ***Sistemas Inteligentes de Detección de Amenazas en Redes mediante Machine Learning***, ha sido realizado bajo su supervisión por **Eduardo Domínguez López**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 1 de septiembre de 2026.

Los directores:

Ignacio Javier Pérez Gálvez José Ramón Trillo Vílchez

Agradecimientos

Después de analizar cientos sino miles, de flujos de red ha llegado el momento de analizar algo todavía más difícil: a quién agradecer todo esto. Aunque por suerte, en este caso no necesitamos ningún modelo de IA para ello.

En primer lugar, me gustaría dirigirme a mis padres. Sin ellos, nada de esto habría sido posible. Gracias por acompañarme durante toda esta carrera de fondo llamada Grado de Ingeniería Informática, por vuestro apoyo constante y por estar siempre presentes durante estos años.

También quiero agradecer a mis tutores, Ignacio Javier Pérez Gálvez y José Ramón Trillo Vílchez, por su esfuerzo y dedicación durante todo el desarrollo de mi TFG. Sin vuestra guía y orientación no habría sido posible llevar a cabo este trabajo.

Por supuesto, que no me puedo olvidar de todos mis amigos y compañeros de la ETSIIT. Son miles los recuerdos que me llevo conmigo. En especial, me gustaría mencionar a Andrés y a Linaje. Juntos hemos formado la tupla de tres elementos que ha ido superando, curso a curso y examen a examen, todo tipo de dificultades y obstáculos. También quiero agradecer a Ana y Aarón, sin quienes hubiera sido imposible superar tercero de carrera. No solo habéis sido compañeros de clase, sino amigos que conservaré para toda la vida.

También quiero agradecer a Nuria, coordinadora del programa Erasmus, por hacer posible mi experiencia académica y personal en Oslo.

No puedo despedirme sin mencionar al equipo del departamento de Network Intelligence de CUJO AI. Gracias a Ike Kunze y Adrian Pekar por todas las enseñanzas que me transmitisteis durante mi estancia de prácticas en Oslo. Me enseñasteis qué es realmente el mundo de la detección de intrusiones, y que, detrás de un resultado aparentemente simple como un número, existen miles de horas de trabajo.

Por último, quería dedicarme unas palabras a mí mismo. Por todo el esfuerzo y sacrificio que has invertido durante esta carrera, por haber seguido adelante ante las dificultades y por haber llegado hasta aquí.

Dicho esto, ¡vamos a lío!

Índice general

1. Introducción	1
1.1. Objetivos y Metodología de la Investigación	9
1.1.1. Objetivo General	9
1.1.2. Objetivos Específicos	9
1.1.3. Metodología	10
1.2. Planificación y Presupuesto	11
1.2.1. Planificación de tiempo	11
1.2.2. Presupuesto	12
2. Marco teórico	13
2.1. Ataques DDoS	13
2.2. Detección de intrusiones	18
2.2.1. Detección basada en anomalías	18
2.2.2. Detección basada en firmas	19
2.3. Sistemas de detección de intrusiones	19
2.3.1. Network Intrusion Detection System	19
2.3.2. Desafíos de los IDS/NIDS	21
2.4. Incorporación del aprendizaje automático a los IDS	22
2.5. Regresión Logística	24
2.6. Árboles de decisión	25
2.7. Support Vector Machines	27
2.8. Perceptrón multicapa	29
3. Desarrollo del proyecto	33
3.1. Descripción general del pipeline	33
3.2. Obtención y construcción de los datos	34
3.3. Limpieza universal	35
3.4. Análisis exploratorio	36
3.4.1. Taxonomía de las características	37
3.4.2. Distribución de clases	41
3.4.3. Análisis temporal del tráfico	42
3.4.4. Análisis preliminar de variables categóricas	44
3.4.5. Análisis estadístico descriptivo	45

3.4.6.	Análisis granular de características	46
3.4.7.	Análisis de valores atípicos	48
3.4.8.	Conclusiones del análisis exploratorio	49
3.5.	Preparación y transformación de los datos	50
3.5.1.	Codificación y escalado de variables categóricas	50
3.5.2.	Filtro estructural	51
3.5.3.	Dependencias entre características y análisis de correlación	51
3.5.4.	Transformación logarítmica	55
3.5.5.	Escalado de características	56
3.6.	Particionado del conjunto de datos	57
3.7.	Construcción de conjuntos de entrenamiento	57
3.8.	Entrenamiento	59
3.8.1.	Modelos seleccionados	59
3.8.2.	Metodología de entrenamiento	60
3.9.	Implementación	62
3.9.1.	Tecnologías y entorno de desarrollo	62
3.9.2.	Organización del proyecto	63
4.	Experimentación y Análisis de resultados	65
4.1.	Métricas	65
4.1.1.	F1 Score	66
4.1.2.	Tasa de falsos positivos	66
4.2.	Diseño experimental	67
4.3.	Análisis e interpretación de resultados	67
4.3.1.	Fase I: establecimiento de la línea base	68
4.3.2.	Fase II: optimización sobre el conjunto completo	70
4.3.3.	Experimentos adicionales: efecto de la estandarización	74
4.4.	Comparación final de configuraciones	75
5.	Conclusiones y trabajo a futuro	77
5.1.	Conclusiones	77
5.2.	Trabajo a futuro	79
5.3.	Viabilidad comercial y consideraciones de sostenibilidad	80
	Bibliografía	80

Índice de figuras

1.1. Código de <i>Creeper</i> . Fuente: Xataka	2
1.2. ARPANET. Fuente: UPC Blog	3
1.3. Estudiantes usando ordenadores domésticos. Fuente: Genbeta	4
1.4. Mapa representativo aumento internet. Fuente: Medium	4
1.5. TCP/IP protocol suite [14]	5
1.6. Ejemplo de la botnet Mirai [23]	6
1.7. 380 millones de solicitudes por segundo. Fuente: Google Cloud Blog	7
1.8. Metodología CRISP-DM [25]	10
1.9. Diagrama de Gantt	12
2.1. Arquitectura de un ataque DDoS [37]	14
2.2. Tipos de ataques DDoS [27]	16
2.3. Arquitectura de un IDS [21]	21
2.4. Ejemplo de regresión lineal [32]	24
2.5. Ejemplo de regresión logística [15]	25
2.6. Estructura de un árbol de decisión [8]	26
2.7. A y B muestran el concepto de margen máximo. C y D mues- tran el funcionamiento de la variable holgura [40]	28
2.8. Estructura de un perceptrón [10]	30
2.9. Arquitectura de un MLP [10]	31
2.10. Ejemplo de una red de retropropagación totalmente conec- tada, con las direcciones del flujo de activación y del error indicadas [42]	32
3.1. Pipeline del proyecto.	34
3.2. Distribución de clases en el conjunto de referencia	41
3.3. Distribución de clases en el conjunto completo	42
3.4. Análisis temporal del tráfico de red	43
3.5. Análisis temporal del tráfico benigno	43
3.6. Análisis temporal tráfico malicioso	44
3.7. Análisis de banderas de control TCP	47
3.8. Análisis del tamaño de la ventana inicial recibida	47

3.9. Análisis variables temporales <i>Active</i> e <i>Idle</i>	48
3.10. Análisis de valores atípicos sobre características de tasas de transmisión	49
3.11. Matrices de correlación de volumen de tráfico pre/post elimi- nación	53
3.12. Matrices de correlación de tamaño de los paquetes pre/post eliminación	54
3.13. Matrices de correlación de temporales pre/post eliminación .	54
3.14. Matrices de correlación de tasas de transmisión pre/post eli- minación	55
3.15. Resultado transformación logarítmica sobre características de tasas de transmisión	56
4.1. Comparación métrica F1	68
4.2. Efecto reducción características en regresión logística	69
4.3. Resultado de las configuraciones numeric y encoded	70
4.4. Comparación métrica F1	71
4.5. Resultados de la transformación logarítmica	72
4.6. Árbol de decisión sobre conjunto correlation_098	73
4.7. Top 3 características árbol decisión	74

Índice de cuadros

1.1. Estimación del presupuesto del proyecto	12
3.1. Conjuntos de datos utilizados en las diferentes fases de experimentación.	35
3.2. Comparación de la evaluación de calidad y los resultados de la limpieza universal de los conjuntos de datos.	36
3.3. Identificación de flujo	37
3.4. Volumen de tráfico	37
3.5. Distribución del tamaño de paquetes	38
3.6. Características temporales	38
3.7. Tasa de tráfico	39
3.8. Características de conexión	39
3.9. Cabeceras (<i>header</i>)	39
3.10. Transferencias bulk	39
3.11. Banderas TCP	40
3.12. Ventana TCP (<i>initial window</i>)	40
3.13. Reducción de características mediante análisis de correlación	52
3.14. Conjuntos de características utilizados en la Fase I.	58
3.15. Conjuntos de características utilizados en la Fase II.	58
4.1. Resultados de los modelos Fase I	68
4.2. Resultados de los modelos Fase II	71
4.3. Coste computacional SVC	72
4.4. Resultados de los modelos con el conjunto de características <code>correlation_098_scaled</code>	74
4.5. Ranking final	75

Capítulo 1

Introducción

Han transcurrido casi 30 años desde que se registró uno de los primeros casos de ataques de denegación de servicio hacia el proveedor de servicios de Internet Panix ISP el 6 de septiembre de 1996, cuyos servidores fueron afectados por un ataque de inundación de paquetes TCP SYN [31]. Desde entonces, estos ataques han ido evolucionando junto con la propia infraestructura de Internet. Como consecuencia de esta evolución, la protección de estos sistemas e infraestructuras se ha convertido en un aspecto fundamental para garantizar su disponibilidad y funcionamiento.

En la década de 1950 y durante la aparición de las primeras redes informáticas y módems, los sistemas y redes se encontraban generalmente aislados o conectados a otras redes de dimensiones reducidas. No fue hasta 1960 cuando comenzó a tomar forma el desarrollo de las redes, y con ello el nacimiento de las primeras amenazas informáticas. En este contexto, la única manera de piratear un ordenador era acceder físicamente a él. Más tarde, dichas amenazas informáticas fueron evolucionando paralelamente a las propias tecnologías, condicionadas por factores como el aumento de la capacidad de procesamiento, la invención de las primeras formas de Internet, la aparición de nuevas arquitecturas y servicios conectados.

A finales de los años 60, la situación comenzó a cambiar. La Agencia de Proyectos de Investigación Avanzada (ARPA) del Pentágono desarrolló un sistema que permitía a los ordenadores comunicarse entre sí a grandes distancias, dando nacimiento a lo que se conoce hoy en día como ciberespacio. La interconexión introdujo una nueva dimensión en la seguridad informática ya que, un problema que anteriormente quedaba aislado en un único ordenador, ahora podía propagarse a varios sistemas conectados.

Dos años más tarde de la invención de ARPANET, un investigador llamado Bob Thomas que trabajaba en el proyecto creó *Creeper*. Se trataba de un programa sencillo que se propagaba de forma independiente de un ordenador a otro que estuviera conectado a la red, mostrando el mensaje *I'm the creeper, catch me if you can!*. Posteriormente, otro miembro del equipo

llamado Ray Tomlinson, el inventor del primer sistema de correo electrónico, creó un programa para localizar y eliminar a Creeper en los sistemas infectados llamado *Reaper*¹. Desde una perspectiva histórica, la aparición de ambos programas muestra el primer ejemplo que tenemos de virus y antivirus informático.

```
BBN-TENEX 1.25, BBN EXEC 1.30
@FULL
@LOGIN RT
JOB 3 ON TTY12 08-APR-72
YOU HAVE A MESSAGE
@SYSTAT
UP 85:33:19    3 JOBS
LOAD AV    3.87    2.95    2.14
JOB TTY  USER      SUBSYS
1  DET  SYSTEM      NETSER
2  DET  SYSTEM      TIPSER
3  12   RT          EXEC
@
I'M THE CREEPER : CATCH ME IF YOU CAN
```

Figura 1.1: Código de *Creeper*. Fuente: Xataka

A medida que avanzaba la década de 1970, la expansión y adopción de las tecnologías y redes fue en aumento. Sin embargo, todo ello entrañaba riesgos, ya que el propio canal de comunicación podía convertirse en un mecanismo de propagación. En este punto, ya no solo bastaba con proteger físicamente los equipos, sino que se requerían de nuevas medidas de seguridad software como el control de acceso, la autenticación de los usuarios, la protección de la información y la identificación de vulnerabilidades. El gobierno estadounidense fue uno de los primeros en adoptar ARPNET, mientras que empezó a desarrollar software para limitar el acceso no autorizado en el Information Sciences Institute de la Universidad del Sur de California. El objetivo de este nuevo proyecto, llamado Análisis de Protección, en el que participaron grandes compañías tecnológicas del momento como Digital Equipment Corporation (DEC), era encontrar soluciones de seguridad automatizadas. En 1979, un joven estadounidense llamado Kevin Mitnick pirateó un sistema informático desarrollado por DEC y robó grandes cantidades de datos confidenciales de la empresa².

¹<https://www.xataka.com/historia-tecnologica>

²<https://nordvpn.com/es/blog/historia-ciberseguridad/>

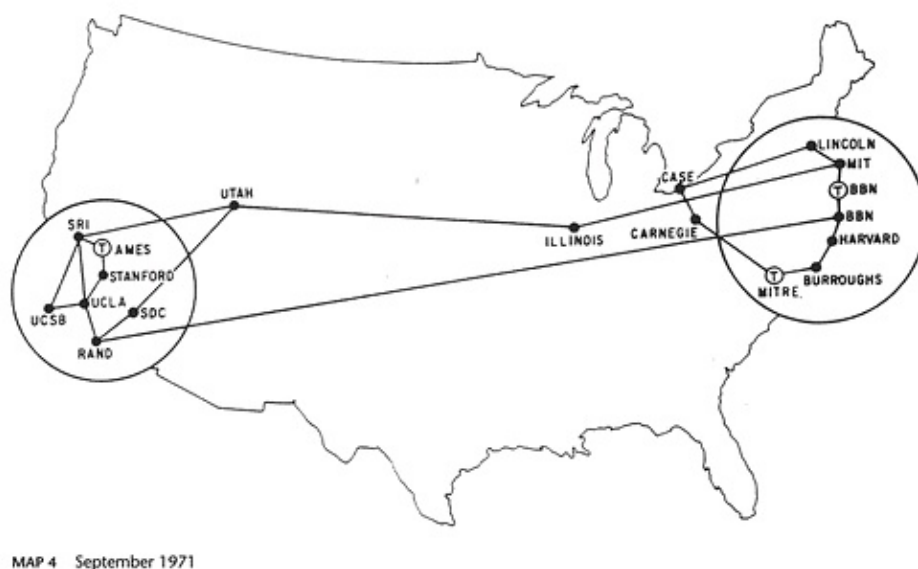


Figura 1.2: ARPANET. Fuente: UPC Blog

Durante la década de los 80, la superficie de exposición aumentó considerablemente. Los ordenadores con acceso a Internet comenzaron a utilizarse en la administración pública, instituciones financieras y muchos otros ámbitos. En el año 1988, un estudiante llamado Robert Tappan Morris, creó el primer gusano de Internet que se propagó por diferentes máquinas. En un primer lugar, Robert desarrolló su gusano con fines de investigación. Sin embargo, un error oculto en el código provocó que el programa creara un gran número de copias de sí mismo constantemente, provocando así un consumo que agotó los recursos del sistema afectado y que produjo interrupciones en el funcionamiento de la red³. Ese mismo año, apareció el primer software antivirus comercial *VirusScan* de la mano de la empresa de seguridad *McAfee*. Estos elementos sientan las bases sobre el comienzo de la era de Internet en la década de los 90, en la que se desarrollarán los primeros ataques dirigidos contra los servicios red y, entre ellos, los ataques de denegación de servicio.

En esa década comienza la proliferación generalizada de Internet. La interconexión de dispositivos es cada vez mayor, los ordenadores pueden ser utilizados de forma remota, los recursos informáticos son cada vez más asequibles y están al alcance de todos. Cualquier ciudadano podía acceder a Internet desde la comodidad de su propia casa. Esta situación favoreció la aparición de ataques remotos que explotaban diferentes vulnerabilidades y con ello el comienzo del desarrollo de los primeros ataques de denegación de

³<https://eugene.kaspersky.es/2016/12/06/una-breve-historia-de-los-ataques-ddos/>

servicio.



Figura 1.3: Estudiantes usando ordenadores domésticos. Fuente: Gen-beta

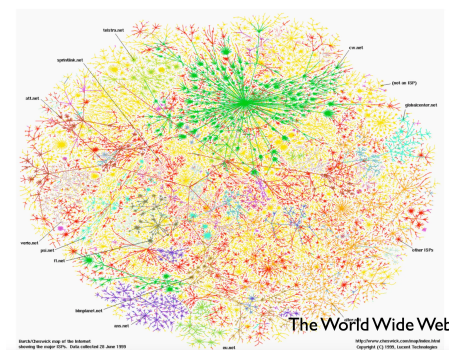


Figura 1.4: Mapa representativo aumento internet. Fuente: Medium

En este contexto, los usuarios se podían ver afectados por el envío de archivos o peticiones especialmente construidas a servicios de Internet como la mensajería. En 1999, David Smith creó un virus que enviaba correos en masa que afectó a Microsoft y a Intel. Cada vez que un usuario ejecutaba el virus, éste se enviaba automáticamente a otros destinatarios provocando un ataque distribuido que se extendió por el mundo. Este incidente puso de manifiesto como un único sistema comprometido puede convertirse en un punto de propagación que extiende la amenaza a una escala mucho mayor. Uno de los acontecimientos más relevantes se produjo en el año 2000, cuando un ciberdelincuente de 16 años apodado *Mafia Boy*, logró tumbar todos los portales principales de Internet de Yahoo!, Dell, eBay o la CNN tras infectar 50 redes informáticas ⁴. Es aquí donde se produce un punto de inflexión y los ataques DDoS comienzan a ser el titular principal en los periódicos.

«*The Net is under attack*»

—Wired, 10 de febrero de 2000

En todos los ejemplos descritos, se puede observar que existe un mismo *modus operandi*. El atacante tiene como objetivo saturar el servidor y sus recursos de red con millones de peticiones de acceso simultáneas. Es así como surge la definición formal de un ataque de denegación de servicio (*Denial of Service*, DoS). Se trata de un tipo de ciberataque que tiene como objetivo colapsar una red o un servidor mediante una inundación de solicitudes para que un servicio o recurso sea inaccesible a los usuarios legítimos. Una extensión del ataque DoS es el llamado ataque de denegación de servicio distribuido (DDoS) cuyo tráfico maligno se genera desde varios puntos hacia un

⁴<https://www.wired.com/2000/02/dos-defense-is-the-best-offense/>

mismo destino [29]. Estos ataques utilizan varios equipos conectados una red llamados *botnets* para distribuir el tráfico generado sobre varios sistemas.

El uso de *botnets* marcó un antes y un después en el ámbito de la seguridad informática, puesto que se desarrollaban para llevar a cabo diferentes ataques y actividad maliciosas como el envío de spam, el robo de datos, los ransomware o los ataques de denegación de servicio distribuido. En el año 2007, se lleva a cabo uno de los acontecimientos más notorios del impacto de una *botnet* con la primera gran ofensiva cibernética coordinada por un país contra Estonia. El ataque DDoS tuvo consecuencias económicas, institucionales y sociales. A este ataque le siguieron nuevas campañas de reflexión y amplificación. En estas campañas el atacante utilizaba protocolos y servicios como DNS, NTP o SSDP para enviar una solicitud pequeña hacia un servidor vulnerable utilizando la dirección de la víctima como origen. Cuando la respuesta es mayor que la solicitud original se produce un efecto tsunami de amplificación que genera un volumen de tráfico mucho más grande que el inicialmente enviado por el atacante. Un ejemplo claro fue el ataque masivo de denegación de servicio contra la organización sin fines de lucro Spamhaus en 2013. Mediante técnicas de amplificación DNS los culpable del ciberataque llegaron a provocar un tráfico de 300 gigabits de tráfico malicioso que fluía a través de la infraestructura de Spamhausand#39 ⁵. Este ataque cambió el panorama digital y de las comunicaciones ya que puso de manifiesto que los servicios de Internet podía ser utilizados indirectamente como vectores de ataque de amplificación.

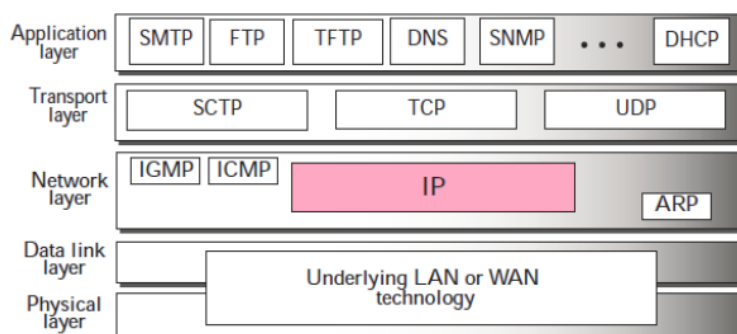


Figura 1.5: TCP/IP protocol suite [14]

Con la expansión del Internet de las cosas (*Internet of Things*, IoT), esta evolución siguió tomando forma al proporcionar un nuevo número de vectores de ataque: cámaras de vigilancia, sensores, routers... En este contexto surgió en 2016 Mirai, un malware que permitió comprometer miles de dispositivos IoT ⁶. Esta botnet ha sido utilizada en numerosas ocasiones para

⁵<https://codestax.medium.com/ddos-symphony-the-layered-assault-on-spamhaus-in-2013-7d0127e847d9>

⁶<https://www.theguardian.com/technology/2016/oct/26/>

ejecutar ataques DDoS. El ataque más conocido se llevó a cabo contra el proveedor de servicios DNS Dyn en octubre de 2016.

Mirai at a Glance

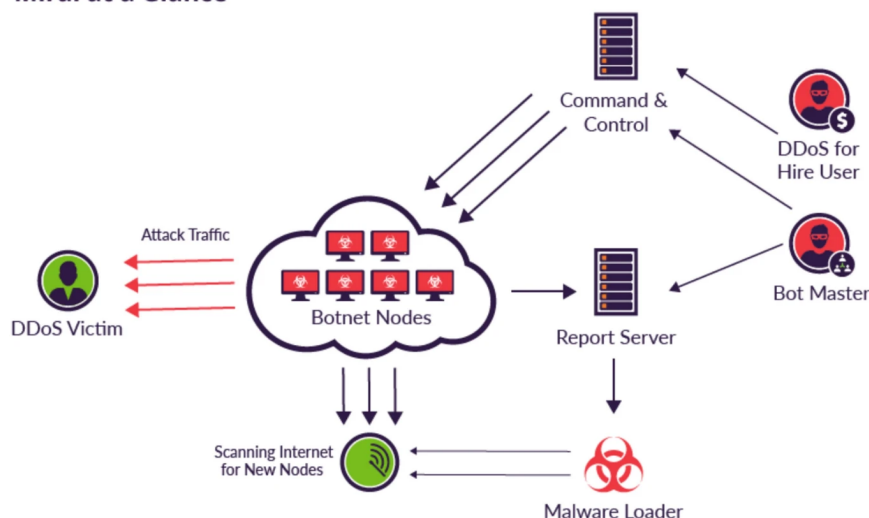


Figura 1.6: Ejemplo de la botnet Mirai [23]

A día de hoy, seguimos registrando nuevos ataques que aprovechan diferentes capas y protocolos, así como comportamientos específicos de protocolos web modernos como HTTP/2 [38]. Según Google, en octubre de 2023, mitigaron el mayor ataque DDoS de la historia sobre el protocolo HTTP/2, que alcanzó un máximo de 380 millones de solicitudes por segundo ⁷. Según el informe publicado por la Agencia de la Unión Europea para la Ciberseguridad (ENISA), los ataques Distribuidos de Denegación de Servicio (DDoS) representan el 76,7 % de los incidentes registrados. Esta categoría está dominada principalmente por grupos hacktivistas, que representan la mayoría de los incidentes DDoS recopilados. Estos grupos perpetraron campañas contra los sitios y portales web de las administraciones públicas de los Estados miembros de la Unión Europea [13].

ddos-attack-dyn-mirai-botnet

⁷<https://www.cloudflare.com/es-es/learning/ddos/famous-ddos-attacks/>

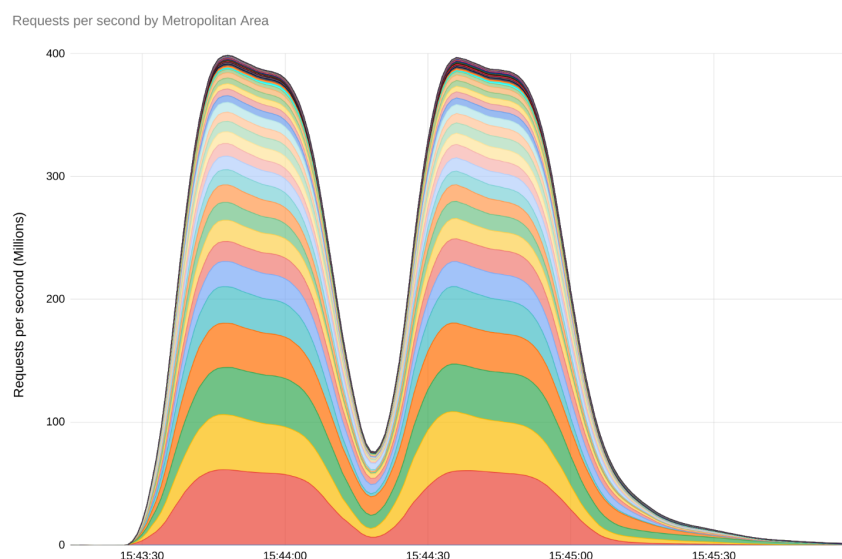


Figura 1.7: 380 millones de solicitudes por segundo. Fuente: Google Cloud Blog

La evolución descrita anteriormente, plantea un problema fundamental: ¿cómo detectar y mitigar un ataque DDoS de manera rápida y eficaz? ¿Qué técnicas son las más adecuadas y eficientes? ¿Cómo distinguimos antes un incremento de tráfico legítimo del tráfico y una actividad maliciosa? La detección de un ataque DDoS en curso es uno de los principales retos a día de hoy debido a diferentes factores. En primer lugar, la rapidez con la que evolucionan estos ataques supone el desarrollo de nuevas técnicas y fuentes empleadas para generarlos. En segundo lugar, la ineficacia de medidas de defensa básicas como el *rate limiting* o filtración de paquetes de red o la incapacidad de detección de los métodos estáticos que establecen reglas para detectar tráfico malicioso que no contemplan nuevas variantes o comportamientos de tráfico malicioso [20].

En este contexto, las técnicas de aprendizaje automático son especialmente útiles porque no requieren asumir una distribución previa conocida de los datos. En particular, diversos estudios han demostrado un rendimiento elevado en algoritmos de aprendizaje supervisado sobre diferentes conjuntos de datos previamente etiquetados [1, 6, 39]. Estos algoritmos han resultado ser interesantes en el ámbito de la detección de intrusiones en red debido a la alta variabilidad del tráfico que depende de diversos factores como los protocolos empleados o el número de usuarios conectados. Sin embargo, la calidad y fiabilidad de los resultados obtenidos radica en la existencia de conjuntos de datos bien diseñados y la definición del conjunto de características más adecuado [36]. Un conjunto de datos no representativo puede provocar que un modelo aprenda patrones específicos y no sea aplicable a

otras situaciones. En este sentido, el conjunto desarrollado por Sharafaldin et al. (2019) llamado *CIC-DDoS2019* soluciona esta problemática y permite el estudio de los ataques de denegación de servicio distribuidos. En él, el tráfico de red se representa mediante flujos de red que describen numerosas características como la duración de un flujo, el volumen de información en bytes transmitidos, las tasas de transferencia, la distribución temporal de las comunicaciones, etcétera. Además, los autores definen una taxonomía de ataques DDoS en función de los protocolos y mecanismos empleados. Entre ellos tenemos ataques basados en TCP, UDP, ICMP, DNS o LDAP. Esta diversidad permite representar un gran abanico de escenarios y evita que el problema de detección quede limitado a un único patrón de ataque.

La principal motivación de este Trabajo Fin de Grado surge del interés personal por abordar un ámbito de carácter multidisciplinar entre la inteligencia artificial y la ciberseguridad. La detección de intrusiones de amenazas en red mediante aprendizaje automático requiere integrar conocimientos procedentes de diferentes áreas o campos de la ingeniería informática, entre ellas la ciencia de datos, la ciberseguridad, la inteligencia artificial o las redes de computadores. Primero se necesita comprender las características de cada amenaza que se quiera identificar. Para ello, se requieren de conocimientos previos sobre redes informáticas, protocolos de comunicación, y programación que dependen del escenario de ataque. A partir de estos escenarios, los datos se recopilan y se tratan mediante técnicas propias de la ciencia de datos. Por último, se necesita seleccionar y evaluar modelos de aprendizaje automático que sean adecuados para este tipo de problemas para que así, puedan distinguir entre comportamientos maliciosos o legítimos. Esta combinación de disciplinas convierte la detección de intrusiones en un reto tanto técnico como académico.

También es motivador que la Universidad de Granada, en colaboración con la Universidad de Sevilla, cuenta con un grupo de amplia experiencia en el campo de los sistemas de detección de intrusiones en el que abordan un gran abanico de líneas de trabajo como el desarrollo de sistemas de detección de intrusiones basados en anomalías o metodologías de evaluación de sistemas de detección de intrusiones ⁸.

Como se ha mencionado anteriormente, los ataques DDoS son una amenaza que a día de hoy sigue activa, por lo que es necesario disponer de mecanismos eficaces para su detección. Por tanto, la justificación de este trabajo recae principalmente en tres elementos: la constante evolución de los ataques DDoS y la necesidad de mecanismos de defensa para ello, la oportunidad única que ofrecen las técnicas de aprendizaje automático para analizar grandes volúmenes de datos y por último la existencia de líneas de investigación en el desarrollo y evaluación de los sistemas de detección de intrusiones.

⁸<https://dtstc.ugr.es/neus-cslab/lineas-de-investigacion/ids/>

En base a todo lo anterior, el presente trabajo tiene como objetivo abordar el problema de la detección de ataques DDoS combinando el análisis de datos, la ingeniería de características y el uso de técnicas avanzadas de aprendizaje automático. Consecuentemente, este trabajo se estructura en varios capítulos que cubren desde el estudio de la importancia e impacto de los ataques DDoS, hasta la evaluación de diferentes modelos de aprendizaje automático. En el Capítulo 2, se sientan las bases teóricas del trabajo. En el Capítulo 3, se desarrolla la fase experimental realizada, donde se explica el pipeline de datos seguido, se obtiene el conjunto de datos, se aplican técnicas de optimización y se entrenan los modelos basados en técnicas de aprendizaje automático. En el Capítulo 4, se realiza un análisis e interpretación exhaustivo de los resultados mediante la evaluación de diferentes métricas que miden el rendimiento de los modelos. Finalmente, el estudio culmina en el Capítulo 5, donde se sintetizan los resultados y se identifican líneas de trabajo futuro.

1.1. Objetivos y Metodología de la Investigación

En esta sección se definen los objetivos que orientan el desarrollo de la investigación. Se establece un objetivo general, complementado por varios objetivos específicos, con el fin de delimitar con precisión el alcance y las metas del trabajo.

1.1.1. Objetivo General

Desarrollar un modelo de inteligencia artificial capaz de detectar intrusiones maliciosas en redes informáticas y generar alertas automáticas ante posibles amenazas.

1.1.2. Objetivos Específicos

1. **Diseñar y entrenar** varios modelos de clasificación para la detección de tráfico DDoS, utilizando algoritmos de aprendizaje automático y aprendizaje profundo sobre un conjunto de datos público de tráfico de red.
2. **Aplicar** distintas técnicas de preprocesamiento de datos e ingeniería de características con el objetivo de optimizar la representación de la información y analizar su impacto sobre el rendimiento de los modelos.
3. **Ajustar y comparar** diferentes modelos con el fin de maximizar la capacidad de detección de tráfico DDoS y minimizar la tasa de falsos positivos.

4. **Evaluar** el rendimiento del modelo mediante métricas de clasificación y analizar su viabilidad para su aplicabilidad en entornos de red reales.

1.1.3. Metodología

La metodología propuesta para este trabajo se basa en el modelo CRISP-DM (*C*Ross *I*ndustry *S*tandard *P*rocess for *D*ata *M*ining), que a día de hoy sigue siendo una de las más adoptadas en proyectos de minería de datos [25]. Se compone de un total de seis fases: *business understanding*, análisis de datos, preparación de los datos, modelado, evaluación y despliegue [41].

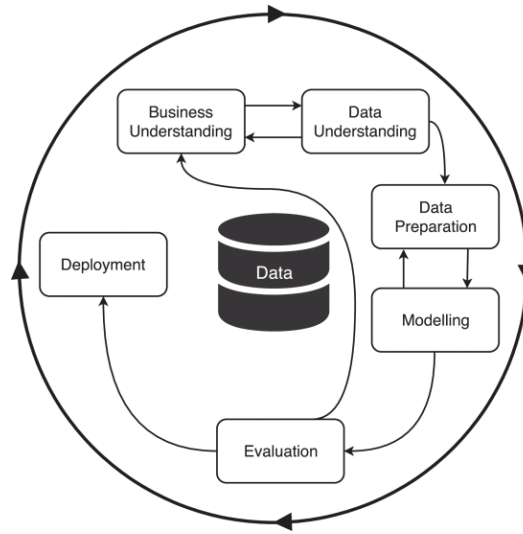


Figura 1.8: Metodología CRISP-DM [25]

En este trabajo, cada una de las fases ha sido adaptada al contexto de la detección de tráfico de red DDoS usando técnicas de aprendizaje automático:

1. **Compresión del problema:** se establece el problema a resolver y los objetivos que guiarán las siguientes etapas. Se realiza una investigación sobre la clasificación de tráfico de red mediante técnicas de aprendizaje automático junto a su capacidad de detección, métodos de clasificación y coste computacional asociado.
2. **Análisis de los datos:** se extraen los conjuntos de datos iniciales y se estudia su estructura o distribución. Esta fase incluye un análisis exploratorio de los datos o identificaciones de posibles problemas que puedan afectar al entrenamiento de los modelos.
3. **Preparación de los datos:** se realizan todas las operaciones necesarias para construir los conjuntos de datos listos para el entrenamiento, como por ejemplo, una limpieza de datos o una selección de características.

4. **Modelado:** se entrenan diferentes modelos de clasificación sobre los conjuntos de datos preparados y se ajustan sus hiperparámetros. El entrenamiento y evaluación se mantienen separados para evitar que la información de prueba influya en el proceso de aprendizaje.
5. **Evaluación:** se evalúan los resultados obtenidos usando diferentes métricas de clasificación adecuadas para el problema.
6. **Despliegue:** se contempla la utilización del modelo en un entorno real. Sin embargo, en el trabajo no se ha realizado un despliegue de los modelos en un entorno de producción.

1.2. Planificación y Presupuesto

A continuación se presenta la planificación temporal y económica del proyecto. En primer lugar, se muestra un diagrama de Gantt para visualizar las fases del proyecto, su duración y las relaciones de dependencias entre ellas. Seguidamente, se presenta una estimación del presupuesto del proyecto, desglosando los principales costes asociados a su desarrollo.

1.2.1. Planificación de tiempo

La planificación final del proyecto se muestra en la figura 1.9 organizada principalmente en cuatro fases:

- **Fase de investigación (Febrero 2026):** se lleva a cabo la tarea de *Revisión bibliográfica* que se centra en el problema de detección de ataques DDoS, con el propósito de conocer el contexto y enfoques del problema. Seguidamente se realiza la *Búsqueda y selección de datasets*, con el objetivo de realizar un estudio y comparación sobre los diferentes conjuntos de datos disponibles públicamente.
- **Fase de desarrollo (Marzo - Mayo 2026):** se comienzan a realizar unas pruebas iniciales que son la base de las tareas de *Construcción y limpieza de datos*, *Análisis exploratorio* e *Ingeniería de características*. Tras completar estas tareas, se procede con la *Experimentación de la fase 1* y la *Experimentación de la fase 2* donde se establece una línea base para una futura optimización del conjunto de datos completo.
- **Fase de redacción y documentación (Junio - Agosto 2026):** en esta fase se lleva a cabo la *Redacción de la memoria* que incluye la Introducción, Metodología, Desarrollo, Experimentación y Análisis de resultados.
- **Fase de conclusiones y cierre (Agosto - Septiembre 2026):** se realiza la tarea de *Conclusiones y cierre* en la que se sintetizan los

hallazgos, se redacta la Conclusión, se efectúa la revisión, corrección y entrega del Trabajo Fin de Grado.

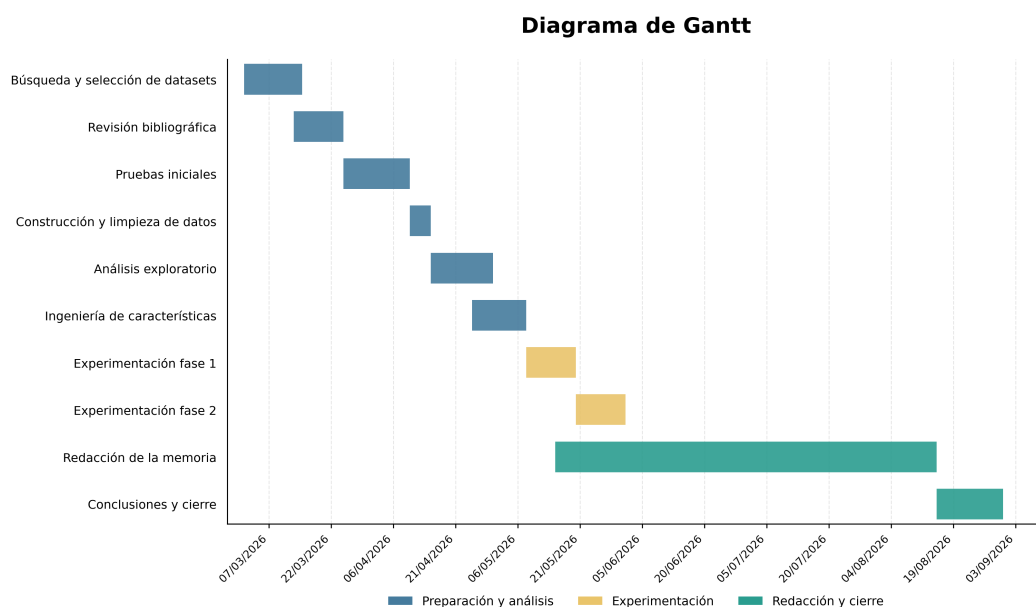


Figura 1.9: Diagrama de Gantt

1.2.2. Presupuesto

En la tabla 1.1 se recoge una estimación de los recursos económicos necesarios para desarrollar el proyecto. Para ello se consideran como referencia el salario de un Investigador en España que es de aproximadamente 1.337,59 €/mes ^[1]. Se ha incluido también costes asociados a equipo informático y acceso a artículos científicos de pago. Finalmente, se considera desarrollo efectivo del proyecto el periodo comprendido entre febrero y agosto de 2026, con un periodo interrumpido durante el mes de julio.

Concepto	Coste unitario	Cantidad	Coste total (€)
Personal investigador	1.337,59 €/mes	5 meses	6.687,95€
Equipo informático	1201,27€ a 4 años	5 meses	125,13€
Artículos científicos de pago	102,61€/año	5 meses	42,75€
Total estimado		6.855,83€	

Cuadro 1.1: Estimación del presupuesto del proyecto

Capítulo 2

Marco teórico

2.1. Ataques DDoS

Un ataque de denegación de servicio distribuido (en inglés, *Distributed Denial of Service (DDoS)*) es un tipo de ciberamenaza que provoca la saturación de un sistema informático conectado a Internet, quedando inaccesible para los usuarios legítimos. [11] Este tipo de ataque interrumpe los servicios del sistema, por lo que no altera, destruye o modifica los recursos del mismo. Eventualmente, la disminución del rendimiento puede derribar el sistema sin tener que destruir sus recursos.

Los ataques DDoS emplean una red o *botnet* formada por un servidor maestro, responsable de dirigir el ataque, que ordena a otros dispositivos intermediarios llamados esclavos o *daemons* atacar al objetivo. El nodo maestro instruye a cada uno de los nodos esclavos para que envíen múltiples solicitudes a los sistemas de la red víctima, saturando sus búferes de peticiones. Todas estas solicitudes pueden contener direcciones IP de retorno falsificadas (en inglés, *spoofed IPs*), por lo que los nodos afectados no pueden localizar al emisor cuando intentan responder. Mientras los sistemas esperan las confirmaciones, continúan recibiendo nuevas oleadas hasta que se agote la capacidad del sistema víctima para procesarlas y termine colapsando. La imagen [19] muestra una típica arquitectura de un ataque DDoS.

[37] Las razones o motivos detrás de un ataque DDoS son específicos y variados. Los atacantes no esperan obtener ningún beneficio material ya que este tipo de ciberamenaza no implica la intrusión en los sistemas y, por lo tanto, no daña la integridad de los recursos dentro de él. Sin embargo, las principales causas de los ataques DDoS son las siguientes [37]:

- Competencia: los ataques DDoS pueden dañar la reputación o arruinar las ventas de una empresa paralizando las operaciones de la misma y beneficiando así a sus competidores.
- Fines lucrativos: los atacantes pueden ser contratados y recibir pagos

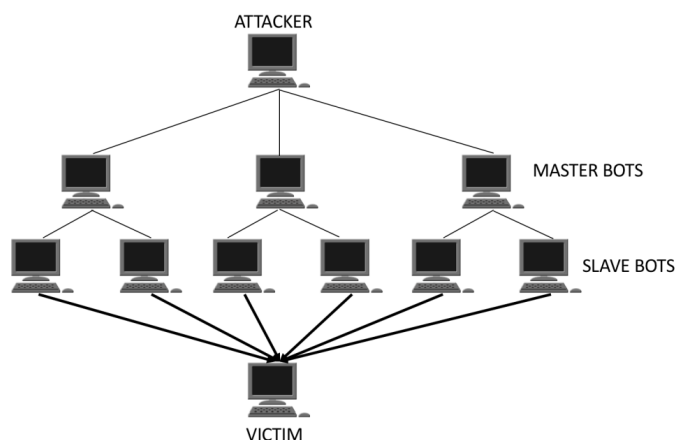


Figura 2.1: Arquitectura de un ataque DDoS [37]

por sus servicios o extorsionar exigiendo un rescate.

- Motivaciones políticas: los ataques DDoS pueden ser empleados por formaciones políticas o grupos terroristas para silenciar digitalmente partidos de oposición u otras voces disidentes.
- Venganza: antiguos empleados, clientes descontentos o cualquier persona con un conflicto con una organización puede tener motivos para lanzar un ataque DDoS.

Según el informe *ENISA Threat Landscape 2025*, las actividades cibernéticas que impactaron a la Unión Europea estuvieron mayoritariamente motivadas por motivos ideológicos y se llevaron a cabo principalmente por grupos *hacktivistas* mediante ataques DDoS. En concreto, las oleadas de ataques DDoS fueron lideradas por grupos hacktivistas pro-rusos en torno a eventos electorales que incluyen el grupo *Cyber Army of Russia Reborn*. Diversas fuentes documentan el uso de inteligencia artificial para aumentar la intensidad y el impacto de los ataques DDoS. Además, se ha experimentado un aumento del 50 % del número de dispositivos infectados en botnets, impulsado por la aparición de la botnet *Zergex* y la evolución de la botnet *DDoSia*, utilizada por el grupo *NoName057(16)* para actividades de comando y control. [13]

Los ataques DDoS presentan una gran diversidad, por lo que resulta complejo agruparlos todos bajo una única clasificación. [37] Primeramente podemos clasificarlos según su alcance en locales o globales. Los ataques locales inutilizan equipos dentro de una red LAN, mientras que los globales pueden originarse a kilómetros de distancia a través de Internet [21]. Otra clasificación ampliamente aceptada consiste en agrupar los ataques DDoS según la capa del modelo OSI en la que operan:

- Capa 3 (Red): en esta capa se sitúan los ataques que explotan el consumo de ancho de banda y la manipulación de paquetes IP. Un ejemplo representativo es la inundación ICMP (en inglés, *ICMP flood*). Este ataque se lleva a cabo cuando el atacante realiza un envío masivo de paquetes ICMP *Echo request* (ping), hacia un servidor con el fin de consumir todo el ancho de banda de red disponible mediante el uso del comando *ping*. La víctima debe responder con paquetes ICMP *Echo reply*, que derivan en una sobrecarga tanto en la red como en el sistema de la víctima.[17]

Dentro de esta capa también se encuentran variantes como el *Smurf attack*, en el que se envían peticiones ICMP Echo con direcciones IP de origen falsificadas a direcciones de difusión (broadcast), así como el *Ping of Death*, el cual utiliza paquetes IP de tamaño superior al máximo permitido (65.536 bytes), provocando bloqueos o caídas.

- Capa 4 (Transporte): en esta capa se agrupan los ataques que explotan el comportamiento de los protocolos de transporte como TCP y UDP. La inundación TCP o *SYN flood* [12] consiste en explotar el proceso de establecimiento de conexión conocido como *three-way handshake*, mediante el envío masivo de paquetes con el flag SYN activado. Cada paquete TCP/SYN es tratado como una petición de conexión, provocando un consumo de memoria y procesamiento que deriva en una saturación del servidor. Por otra parte, se encuentra la inundación UDP o *UDP flood*. En este caso se realiza un envío masivo de paquetes UDP a los puertos de la máquina víctima provocando un consumo de todo el ancho de banda de la red. Los servicios más comunes que se ven afectados por este tipo de ataques son DNS, NTP, SSDP y SNMP. Además, en esta capa se incluyen ataques más específicos como el *Land Attack* (*land.c*), o el *Teardrop* (*teardrop.c*).
- Capa 7 (Aplicación): saltamos a esta capa donde se encuentran ataques generalmente más sofisticados. A diferencia de los ataques de la capa de red y de transporte, los ataques de la capa de aplicación consumen menos ancho de banda. [19]. La inundación HTTP o *HTTP flood* tiene como objetivo saturar un servidor ya sea con solicitudes HTTP GET o HTTP POST. También encontramos otros tipos de ataques como los ataques DNS o los ataques basados en XML.

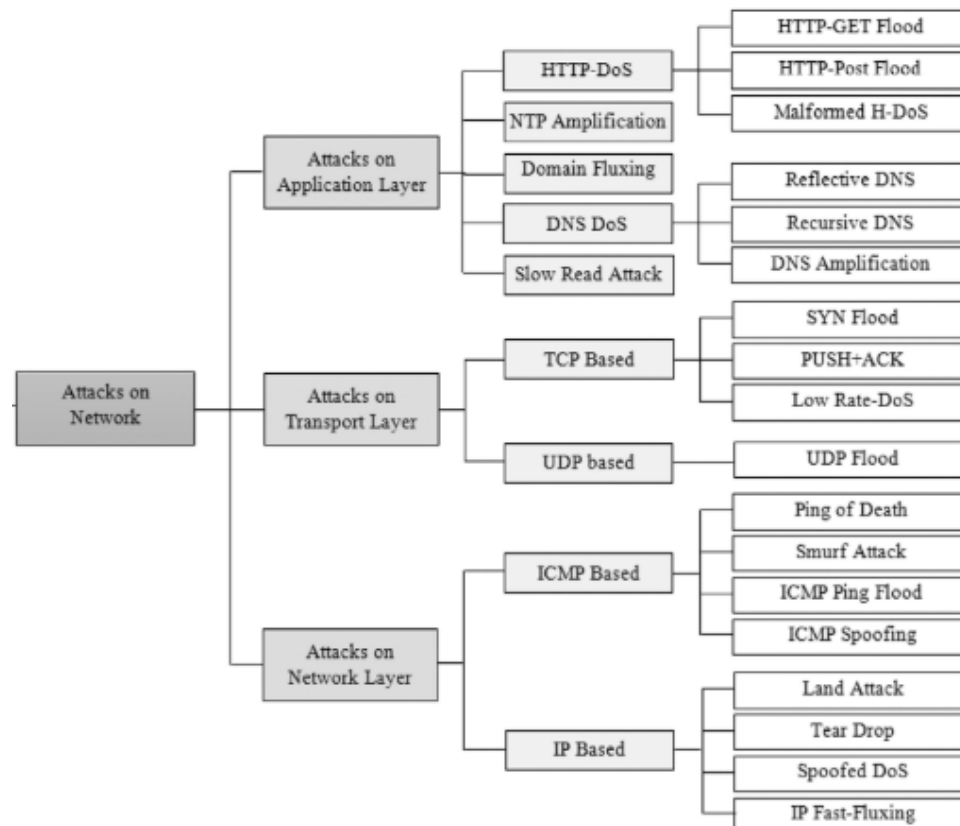


Figura 2.2: Tipos de ataques DDoS [27]

Por último, [37] presenta otra taxonomía válida que divide los ataques DDoS de la siguiente manera:

- Ataques DDoS volumétricos: generan una cantidad masiva de tráfico que suele medirse en Gigabits por segundo (Gbps). Dentro de esta categoría se encuentra *UDP flood attack*, *DNS amplification attack* o *SNMP Amplification Attack* entre otros.
- Ataques a protocolos: las reglas que rigen el intercambio de información en un protocolo pueden ser explotadas por un atacante. Algunos ataques conocidos son: *PSH+ACK flood*, *DNS flood*, *ICMP flood*, *IP Null Attack*, etc.
- Ataques DDoS de baja frecuencia: el tráfico generado por estos ataques se asemeja mucho al tráfico legítimo, y por tanto tienen una mayor capacidad de evasión. Ejemplos de esta categoría son los ataques tipo *flash crowd*, ataques por colisiones de hash o ataques *TCP Shrew*.
- Ataques DDoS basados en vulnerabilidades: en esta categoría los ata-

cantes explotan vulnerabilidades desconocidas o sin parchear en los servidores u protocolos para lanzar ataques. Algunos ejemplos son ataques *DDoS Zero-Day* o ataques *LAND (Local Area Network Denial)*.

Existen diferentes técnicas de prevención y mitigación de ataques DDoS. Estos mecanismos de defensa buscan por un lado, reducir la probabilidad de que el ataque tenga éxito y, por otro, minimizar su impacto una vez que se ha iniciado:

- **Arquitectura:** fortalecer la arquitectura de red ayuda a mejorar la estabilidad de una organización. Existen diversas medidas como la ubicación de los servidores en varios centros de datos distribuidos o la aseguración de que la infraestructura de la empresa no dependa de un único punto que provoque una caída total del sistema.
- **Mecanismos de seguridad *software*:** el uso de un *hardware* adecuado no evita por sí solo un ataque DDoS. Por ello, se suelen combinar módulos de *hardware* con mecanismos de seguridad de *software*. Algunos de estos mecanismos permiten identificar direcciones IP falsificadas como el filtrado por conteo de saltos (en inglés, *hop-count filtering*) o el protocolo *Source Address Validity Enforcement (SAVE)*. Otras medidas limitan la tasa de paquetes o de nuevas conexiones en puntos críticos de la red.
- **Ancho de banda:** el aumento del ancho de banda de la red permite un mayor volumen de tráfico. Esta medida tiene como objetivo contener el ataque DDoS durante más tiempo e intentar mitigarlo con la ayuda del proveedor de servicios de Internet.
- **Protección de servidores DNS:** Los servidores DNS son un vector de ataque que debe ser monitorizado periódicamente. Estos servidores contienen una base de datos con registros del nombre de dominio y su dirección IP correspondiente. Cuando un usuario solicita un sitio web por su dominio, el servidor DNS lo resuelve a su dirección IP correcta para mostrar el recurso solicitado. La monitorización permite identificar comportamientos sospechosos como consultas anómalas o accesos desde direcciones IP sospechosas.
- **Análisis de comportamiento y aprendizaje automático:** cada vez más empresas y agencias de seguridad buscan modelos de inteligencia artificial que puedan predecir ataques futuros y así, desarrollar estrategias de mitigación. En el contexto de la detección de ataques DDoS, las técnicas basadas en inteligencia artificial desempeñan un papel fundamental ya que permiten el análisis eficiente de grandes volúmenes de tráfico de red.

La detección de los ataques DDoS es fundamental para el correcto funcionamiento de cualquier mecanismo de mitigación. Uno de los principales retos consiste en distinguir entre flujos de tráfico malicioso y tráfico benigno. Para abordar este problema, se emplean mecanismos de detección de intrusiones.

2.2. Detección de intrusiones

La seguridad en redes informáticas se compone de tres elementos: prevención, detección y respuesta. La detección de intrusiones es una técnica especializada en detectar el acceso no autorizado, exitoso o no, a una red o sistema informático por parte de un *outsider* para acceder, vulnerar, manipular o hacer un uso indebido de un recurso valioso. Este uso indebido puede comprometer su fiabilidad o incluso hacerlo inutilizable [21]. Existen cinco tipos de intrusiones:

- Ataques enmascarados: detectados por IDS basados en anomalías.
- Penetraciones en el sistema de control de seguridad: detectados mediante monitorización.
- Fuga: detectada por un uso atípico de los recursos del sistema.
- Denegación de servicio: detectado por un uso atípico de los recursos del sistema.
- Uso malicioso o uso de privilegios especiales: detectado por perfiles de comportamiento atípico.

Estas intrusiones se pueden agrupar en tres mecanismos de detección de intrusiones: **detección basada en anomalías**, **detección basada en firmas** y **detección híbrida**.

2.2.1. Detección basada en anomalías

La detección basada en anomalías se encarga de identificar aquellas actividades que se desvían de lo considerado normal o que no son consistentes con el comportamiento habitual. Idealmente, se requiere de una definición clara de lo que se considera un comportamiento habitual. Sin embargo, en entornos reales estos patrones de comportamiento van cambiando a lo largo del tiempo por lo que el modelo debe actualizarse continuamente a medida que aparecen nuevos patrones que deben ser clasificados como normales o anómalos. Uno de los riesgos más comunes de este tipo de detección es entrenar a un modelo con datos que ya no son fiables aumentando en consecuencia el número de falsos positivos.

2.2.2. Detección basada en firmas

La detección basada en firmas se centra en identificar patrones específicos asociados a actividades maliciosas previamente conocidas. Este enfoque requiere una base de datos de firmas que recoja las acciones consideradas como maliciosas. Para facilitar la trazabilidad y la organización de la base de datos, estas actividades se pueden clasificar en: acceso no autorizado, modificación no autorizada y denegación de servicio. Sin embargo, este enfoque es que únicamente puede identificar ataques previamente conocidos, siendo incapaz de modelar nuevas amenazas o variantes desconocidas.

Debido a las limitaciones existentes en ambos métodos de detección, se están desarrollando nuevos métodos híbridos que combinen la detección basada en anomalías y la detección basada en firmas.

2.3. Sistemas de detección de intrusiones

Los sistemas de detección de intrusiones (en inglés *Intrusion Detection Systems, IDS*) también pueden ser clasificados según su ámbito o alcance de monitorización. Los Sistemas de Detección de Intrusiones Basados en Red (en inglés, *NIDS*) se encargan de monitorizar el tráfico de toda una red en comparación con los Sistemas de Detección de Intrusiones Basados en Host (en inglés, *HIDS*) cuyo ámbito de monitorización se reduce a un único ordenador.

2.3.1. Network Intrusion Detection System

Los **NIDS** tienen como objetivo detectar comportamientos que puedan considerarse no autorizados y potencialmente dañino dentro de una red. Conviene aclarar que mientras que los *firewalls* permiten el tráfico siempre que cumpla con un patrón de reglas predefinidas sin analizar en profundidad su contenido, un *NIDS* captura e inspecciona todos los paquetes dirigidos a la red, independientemente de si han sido permitidos o no por el *firewall*. Si los paquetes presentan características sospechosas, el sistema genera una alerta.

Existen dos formas para desplegar un NIDS. La primera consiste en un sistema independiente que monitoriza de forma pasiva el tráfico de la red (modo promiscuo). La segunda es un sistema que supervisa únicamente el tráfico de la máquina en la que está instalada permitiendo detectar ataques dirigidos específicamente al *host* como la inundación SYN o el escaneo de puertos.

La arquitectura de un *IDS* o un *NIDS* está compuesta por varios módulos que trabajan de forma coordinada para generar alertas ante posibles amenazas. En primer lugar, el **distribuidor de carga de red** (en inglés, *Network Tap/Load Balancer*) se encarga de interceptar todo el tráfico que

circula por la red y distribuirlo hacia todos los sensores. Este componente puede ser hardware (como un router) o software. A continuación, los sensores de red reciben el tráfico desde el distribuidor de carga, lo analizan y separan el tráfico normal del sospechoso. Pueden basarse en técnicas de detección por firmas o por anomalías, dependiendo del enfoque del sistema. Seguidamente, el tráfico considerado relevante se envía al módulo de preprocesamiento y análisis, que constituye el núcleo del *NIDS*. En esta fase, se realiza una normalización del tráfico de red (reensamblado de fragmentos, ordenación de paquetes, etc.) y posteriormente se evalúa el nivel de amenaza en diferentes fases: la primera determina la severidad del ataque y la segunda determina su alcance, intención y frecuencia. De esta manera queda el tráfico clasificado como legítimo o malicioso.

Una vez detectado un posible incidente, se activa el **módulo de notificación de alertas**, que informa al personal de seguridad ya sea mediante correo electrónico, alertas visuales o alertas sonoras. Cabe destacar que para aumentar la efectividad del sistema, es importante ajustar el sistema según número y tipo de alertas generadas: amenazas aparentemente triviales generan una alta tasa de falsos positivos, pero una falta de informes frecuentes que ignoran amenazas reales produce falsos negativos. El control central del NIDS recae en la consola de gestión global que permite configurar políticas, gestionar alertas y supervisar el funcionamiento global del sistema. También, el sistema NIDS puede incorporar un subsistema de respuesta para ejecutar de forma automática o manual determinar acciones frente a amenazas detectadas (cierre de conexiones, bloqueo de direcciones IP, reconfiguración de routers o firewalls, etc.). Por último, toda la información generada (estadísticas, incidentes detectados, falsos positivos, etc.) se almacena en una base de datos. Este módulo permite la mejora de los métodos de detección usados y la realización de tareas de auditoría o investigación de incidentes pasados [21].

Sin embargo los NIDS no son completamente precisos. Existen ambigüedades en el flujo de tráfico que son explotadas por atacantes con el fin de evadir su detección. Muchos *NIDS* no disponen de capacidades de análisis para evaluar todo el rango de comportamientos que un usuario o atacante puede generar. Dado que los *NIDS* están alejados de los dispositivos individuales (término conocido también como *No Endpoint Installation*), no disponen de un conocimiento completo sobre la implementación de los protocolos de cada uno. Esto dificulta determinar cómo reaccionará realmente el *host* ante el tráfico generado. Por otro lado, al no tener una visión completa de la topología de red existente entre el NIDS y los dispositivos finales, el sistema será incapaz de saber si un paquete concreto llegará realmente a su destino o será descartado en algún punto.

Una de las mayores limitaciones de un *NIDS* está en la incapacidad para detectar ataques dentro del tráfico cifrado. Si bien pueden inspeccionar partes no cifradas como la cabecera de un paquete, resultan ineficaces para

examinar el contenido completo.

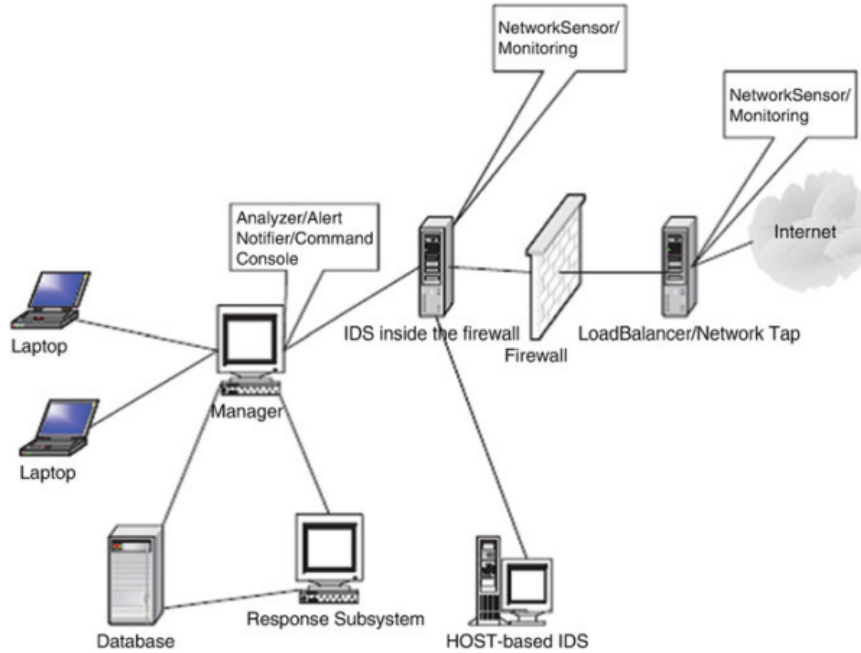


Figura 2.3: Arquitectura de un IDS [21]

2.3.2. Desafíos de los IDS/NIDS

Axelsson (2000), profesor de la Universidad Noruega de Ciencia y Tecnología especialista en informática forense, en su investigación introduce el concepto de la falacia de la frecuencia base (en inglés, *The base-rate fallacy*) y concluye que, en condiciones realistas, la limitación principal de un IDS no viene dada por su capacidad de identificar comportamientos maliciosos, sino por su capacidad para mantener una tasa de falsas alarmas extremadamente baja, que en muchos casos llega a ser inalcanzable [4]. Este complicado concepto estadístico, se extrae directamente del *Teorema de Bayes*, herramienta fundamental en el análisis estadístico y toma de decisiones muy utilizado en la inteligencia artificial, que establece la relación entre la probabilidad condicional de un evento A dado su opuesto B con la probabilidad de B dado A :

$$P(A|B) = \frac{P(A) \cdot P(B|A)}{P(B)}$$

donde

$$P(B) = \sum_{i=1}^n P(A_i) P(B | A_i)$$

Apliquemos este concepto en el siguiente supuesto: tenemos un detector de intrusiones que tiene una exactitud o *accuracy* del 99 %, es decir, cuando el detector se probó en un tráfico donde todos los eventos eran intrusiones se detectaron el 99 % de ellas. Análogamente, cuando el 100 % del tráfico es normal, nuestro modelo ha detectado el 99 % como tráfico normal. Ahora supongamos que tenemos un total de 10000 eventos de tráfico de red y sólo 1 de ellos es una intrusión real. ¿Cuál sería la probabilidad de que realmente el modelo detecte la intrusión?. Pues bien, denotemos $S = \text{intrusion}$, $\neg S = \text{no intrusion}$, $R = \text{alarma}$, $\neg R = \text{no alarma}$. Recopilando la información anterior tenemos: $P(R|S) = 0.99$, $P(\neg R|\neg S) = 0.99$, y $P(S) = \frac{1}{10000}$, ¿cuál es la probabilidad de $P(S|R)$? Aplicando el Teorema de Bayes y sustituyendo $P(B)$ nos queda:

$$P(S | R) = \frac{P(S) \cdot P(R | S)}{P(S) \cdot P(R | S) + P(\neg S) \cdot P(R | \neg S)} = \quad (2.1)$$

$$\frac{\frac{1}{10000} \cdot 0.99}{\frac{1}{10000} \cdot 0.99 + (1 - \frac{1}{10000}) \cdot 0.01} = 0.0098 \approx 1 \% \quad (2.2)$$

Es decir, aunque el modelo tenga un *accuracy* del 99 %, la probabilidad de que realmente una alarma sea una intrusión cuando se detecte una intrusión es sólo $\frac{1}{100}$, porque la cantidad de eventos de tráfico de red normales es mucho mayor que la de intrusiones. En otras palabras, una tasa de falsos positivos del 1 % genera un gran cantidad de alertas erróneas debido a la gran cantidad de datos normales. Es por ello que los enfoques de intrusión basados en anomalías siguen siendo un gran desafío en la actualidad.

2.4. Incorporación del aprendizaje automático a los IDS

La incorporación de técnicas de inteligencia artificial en los sistemas de detección de intrusiones responde principalmente al hecho de que son sistemas basados en software, ya sea integrado en dispositivos o desplegado de forma independiente, y al enorme volumen de datos que se genera y circula en entornos modernos. En este contexto, los algoritmos de aprendizaje automático permiten procesar grandes cantidades de datos de manera automatizada, identificar patrones complejos y detectar anomalías que serían difíciles de reconocer mediante técnicas convencionales. Existe una amplia variedad de algoritmos utilizados en IDS. Entre ellos, los más destacados son los **árboles de decisión (ID3)**, el **random forest**, la **regresión logística**, **SVM** y las **redes neuronales** [21].

Para que dichas técnicas y algoritmos de aprendizaje automático sean fiables y robustos es necesario disponer de conjuntos de datos que representen los diferentes tipos de escenarios y ataques que se pretendan detectar. Entre

los primeros conjuntos de datos proporcionados por la comunidad científica para la investigación y detección de intrusiones se encontraban conjuntos de datos como DARPA y su derivado, *KDD Cup 1999 (KDD99)*. Ambos sirvieron como puntos de referencia para la investigación de NIDS. Más tarde, surgió el famoso *NSL-KDD* dataset, donde se solucionaban algunos problemas que presentaba KDD99. Sin embargo, presentan importantes limitaciones que los hacen poco adecuados en entornos actuales. En particular KDD99 y NSL-KDD contienen registros redundantes y problemas de alineación con arquitecturas de red reales, además de haber quedado obsoletos con el paso del tiempo [22]. En el caso de *DARPA*, los ataques fueron generados por un atacante *naive* lo que implica que no representan técnicas avanzadas utilizadas en escenarios reales. Asimismo, muchos datasets, como *CAIDA 2007*, contienen ventanas temporales limitadas lo que dificulta el diseño de sistemas de detección robustos en tiempo real. En general, unas limitaciones comunes son que muchos de estos conjuntos de datos se basan en tráfico sintético, falta de diversidad de ataques y una representación pobre de las interacciones de red [36].

En el caso de los conjuntos de datos *CICIDS2017* y *CICIDS2018*, si bien suponen una mejora respecto a los anteriores al incorporar tráfico más reciente y variado, diversos estudios han evidenciado problemas significativos en su calidad. Se han identificado errores en el proceso de etiquetado que afectan a un porcentaje considerable de los datos. [22] Por otro lado, también se han detectado problemas adicionales como incoherencias en las marcas temporales y errores en la extracción de características debido al mal funcionamiento de herramientas como *CICFlowMeter*. Además, ciertos ataques, como los escaneos de puertos, no están correctamente etiquetados. [24] En conjunto, estos problemas comprometen la validez de los resultados obtenidos y dificultan la generalización de los modelos a entornos reales.

En este contexto, *CIC-DDoS2019* aborda muchas de las limitaciones mencionadas anteriormente al incluir los ataques DDoS más comunes y recientes, especialmente ataques de reflexión y amplificación como NTP, SSDP o TFTP, que no están presentes en datasets anteriores. [36] Además, proporciona un entorno más realista, con mayor diversidad de ataques convirtiéndolo en una opción más robusta y fiable para el entrenamiento y evaluación de modelos de detección de DDoS en entornos actuales. Este dataset incluye registros de tráfico de red capturados en el formato de archivo PCAP (Packet Capture o registros de flujo). Cada registro contiene 85 características, de las cuales 84 son características de tráfico de red y la última es la etiqueta.

Cuando utilizamos enfoques basados en aprendizaje automático, normalmente utilizamos algún tipo de proceso de extracción de características para crear características estadísticas de nivel superior con el fin de alimentar un algoritmo. Por eso, tradicionalmente, se condensan los datos en flujos de red. Los **flujos de red** son la suma de las estadísticas de todos los paquete-

tes transmitidos desde una única conexión de protocolo, compuesta por una quintupla: dirección IP de origen, dirección IP de destino, puerto de origen, puerto de destino y protocolo. Además, estos flujos de red son captados según un tiempo de espera. Si se produce un ataque malicioso, no podemos esperar a que la conexión finalice o se cierre.

2.5. Regresión Logística

La regresión logística es un modelo de clasificación lineal dentro del aprendizaje supervisado que se remonta a la década de los sesenta [9]. Es un modelo sencillo, interpretable y eficiente. Estima la probabilidad de que una observación pertenezca a una determinada clase a partir de una combinación lineal de las características de entrada:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

done x_i representa cada característica, β_i su correspondiente peso y β_0 el término independiente.

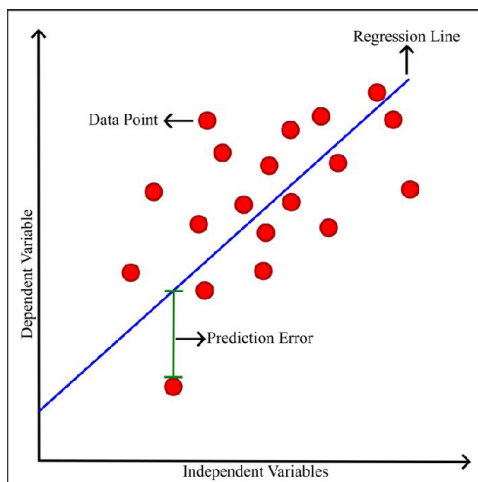


Figura 2.4: Ejemplo de regresión lineal [32]

Este algoritmo de regresión puede utilizarse para problemas de clasificación binaria, donde se pretende deducir a qué clase pertenece una instancia determinada. Esta combinación lineal se transforma mediante una función logística, permitiendo obtener una probabilidad que posteriormente se utiliza para asignar la clase correspondiente. Dicha función logística es una función sigmoidea con forma de S que da como resultado un número entre 0 y 1. Si la probabilidad calculada es superior a un umbral determinado, se clasifica esa instancia dentro una clase positiva (etiquetada como 1). En otro

caso, la instancia pertenece a la clase negativa (etiquetada como 0) [18].

$$P(y = 1 | X) = \frac{1}{1 + e^{-z}}$$

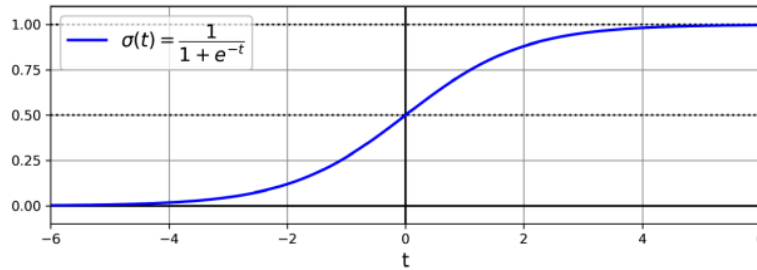


Figura 2.5: Ejemplo de regresión logística [15]

2.6. Árboles de decisión

Los árboles de decisión son algoritmos de aprendizaje automático supervisado que realizan tareas tanto de clasificación como de regresión mediante una secuencia de divisiones recursivas del espacio de datos. Además, este tipo de modelo es un componente fundamental de los *random forests*.

Se encuentran entre los más usados y potentes en la investigación sobre detección de intrusiones en red ya que ofrecen un enfoque transparente e interpretable, permitiendo a los analistas de seguridad entender la toma de decisiones del modelo y las características importantes que contribuyen a la clasificación. [5]

Están compuestos por nodos y ramas. Los nodos se dividen en nodos internos y nodos terminales denominados hojas. Cada nodo de decisión m aplica una función o condición sobre una característica de entrada $f_m(x)$, generando distintos posibles resultados que determinan las ramas a seguir. La construcción del modelo es recursiva, comenzando desde la raíz, el modelo evalúa de manera recursiva estas condiciones en función de los valores de entrada, recorriendo el árbol hasta alcanzar una hoja. El valor asociado a cada hoja constituye la salida final del modelo, ya sea una clase (en problemas de clasificación) o un valor numérico (en problemas de regresión).

Un árbol de decisión se considera un modelo no paramétrico porque no asume ninguna forma matemática predefinida sobre cómo deben organizarse los datos. En cambio, en modelos paramétricos, como la regresión lineal donde existe una ecuación con ciertos parámetros, se fija de antemano una estructura de los mismos. El árbol se construye de forma dinámica durante el proceso de aprendizaje, añadiendo nuevas ramas y hojas en función de la complejidad del problema. [3]

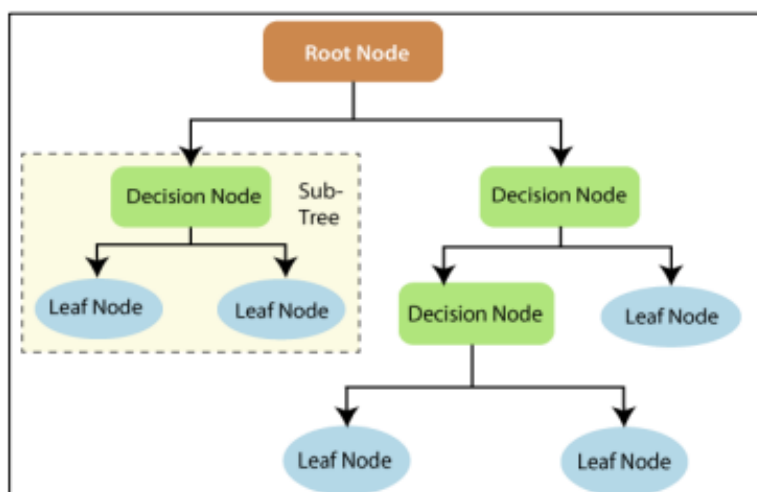


Figura 2.6: Estructura de un árbol de decisión [8]

El objetivo de un árbol de decisión es dividir el espacio de características en regiones cada vez más homogéneas respecto a la variable objetivo. Para ello, en tareas de clasificación, esta división de atributos se realiza minimizando las métricas de impureza como el índice de Gini o la Entropía. El índice de Gini mide la probabilidad de clasificar incorrectamente una muestra si se asigna aleatoriamente una clase según la distribución del nodo:

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

donde:

- n es el número de clases.
- $p_{i,k}$ es la probabilidad de pertenecer a la clase i

Un nodo puro presenta un valor de Gini igual a 0.

La entropía se extendió a gran variedad de ámbitos, incluida la teoría de la información de Shannon, donde mide el contenido medio de información de un mensaje.

$$H = - \sum_{k=1}^n p_{i,k} \log_2 p_{i,k}$$

La entropía de un conjunto es cero cuando contiene instancias de una sola clase.

Scikit-Learn utiliza el algoritmo *Classification and Regression Tree (CART)* para entrenar árboles de decisión. El algoritmo funciona dividiendo primero el conjunto de entrenamiento en dos subconjuntos utilizando una única característica k y un umbral t_k .

Se busca el par (k, t_k) que produce subconjuntos más puros, ponderados por su tamaño. La siguiente ecuación da la función de coste que el algoritmo intenta minimizar :

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

donde :

- $G_{\text{left/right}}$ mide la impureza del subconjunto de la izquierda/derecha.
- $m_{\text{left/right}}$ es el numero de instancias del subconjunto de la izquierda/-derecha.

Una vez dividido con éxito, divide los subconjuntos utilizando la misma lógica, así, de forma recursiva. Deja de buscar cuando alcanza la profundidad máxima o si no puede encontrar una división que reduzca la impureza. Los árboles de decisión son algoritmos voraces, van añadiendo a la solución el subconjunto que mejor impureza tenga. Sin embargo, encontrar la solución óptima es un problem *NP-completo* ya que se requiere un tiempo $O(\exp(m))$. [15]

En cuanto a complejidad computacional, una predicción supone recorrer el árbol hasta la hoja, pasando aproximadamente por $O(\log_2(m))$ nodos. En cada nodo, sólo se comprueba el valor de una característica, por lo que la complejidad global de la predicción es $O(\log_2(m))$. Por otra parte la complejidad de entrenamiento viene dada por el total de las características (n) a comparar en cada nodo: $O(n \cdot m \log_2(m))$.

2.7. Support Vector Machines

La máquina de vector soporte (SVM) (del inglés, *Support Vector Machine*) es un modelo de aprendizaje automático supervisado. Es un modelo versátil capaz de realizar tareas de clasificación linear y no linear, regresión e incluso detección de *outliers*. Es un modelo diseñado para encontrar el hiperplano óptimo que separe las diferentes clases de datos. En problemas de clasificación, el objetivo es maximizar el margen (o distancia) entre el hiperplano y los ejemplos más cercanos de cada clase llamados vectores de soporte. Este proceso se conoce como clasificación de gran margen o de margen máximo. Sin embargo, esta clasificación no siempre es trivial. Los datos pueden presentar valores atípicos que violen los límites de los márgenes o pueden no ser linealmente separables. La clasificación del margen blando introduce el parámetro de regularización C que reajusta el hiperplano de la mejor manera para separar los datos. Se busca un equilibrio entre mantener margen lo más ancho posible y reducir el número de violaciones del mismo. [15] Sea un conjunto de entrenamiento formado por

$$(x_i, y_i) \}_{i=1}^l$$

donde x_i representa el vector de características de la muestra i , $y_i \in \{-1, 1\}$ representa la su etiqueta de clase para clasificación binaria y l es el número de total de muestras. El problema de optimización puede formularse como:

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} w^T w + C \sum_{i=1}^l \xi_i \\ \text{sujeto a} \quad & y_i (w^T \phi(x_i) + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \quad i = 1, \dots, l. \end{aligned}$$

donde w es el vector normal al hiperplano, b es el término de sesgo, ξ_i son variables de holgura que miden cuánto viola cada punto la condición del margen y C es el parámetro de regularización. La función objetivo está compuesta por dos términos:

$$\frac{1}{2} w^T w$$

que busca maximizar el margen entre clases minimizando la normal de w , y

$$C \sum_{i=1}^l \xi_i$$

que penaliza los errores de clasificación. Valores elevados de C implica una penalización mayor de los errores provocando un margen de decisión más estrecho y, en consecuencia riesgo mayor de sobreajuste. Valores pequeños permiten un margen más amplio a costa de admitir más errores. [7]

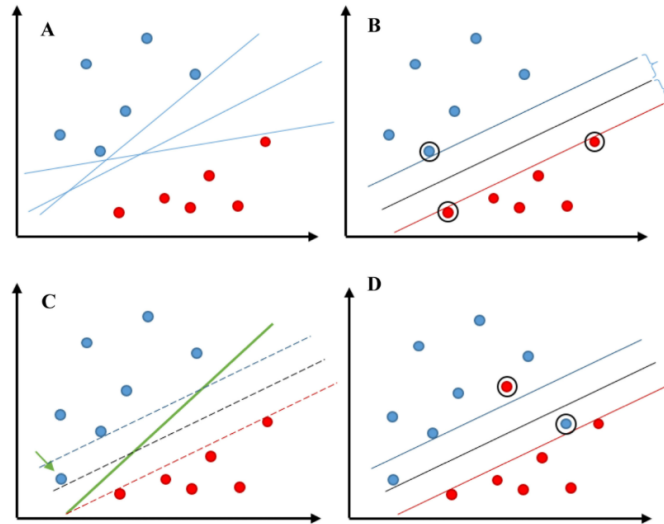


Figura 2.7: A y B muestran el concepto de margen máximo. C y D muestran el funcionamiento de la variable holgura [40]

Cuando las clases no son linealmente separables, los SVM utilizan un método que consiste en añadir características polinómicas que, en algunos casos, da lugar a un conjunto de datos linealmente separable. Esta proyección de los datos a espacios de mayor dimensión es una técnica matemática llamada el *truco del kernel*. Esta técnica permite obtener un resultado similar al que se obtendría si se hubieran añadido muchas características polinómicas pero sin hacerlo realmente. Existen diferentes tipos de kernels que se usan dependiendo de el tipo de estructura de datos del conjunto de entrenamiento. Entre los más usados se destacan el kernel RBF gaussiano y el kernel lineal.

La complejidad en tiempo de entrenamiento depende del tipo de kernel: usando el kernel lineal es aproximadamente $O(m \times n)$, mientras que usando el kernel RBF gaussiano suele estar entre $O(m^2 \times n)$ y $O(m^2 \times n)$, donde m es el número de instancias y n es el número de características.

2.8. Perceptrón multicapa

Las redes neuronales artificiales son un modelo de aprendizaje automático inspirado en el funcionamiento de las neuronas del cerebro humano. Gracias a su escalabilidad, potencia y versatilidad, estos modelos son ideales para resolver problemas de aprendizaje automático de gran envergadura. [15] Uno de los modelos más utilizados dentro de este paradigma es el perceptrón multicapa (*Multi-Layer Perceptron*, MLP). [16]

El origen de este modelo recae en el llamado perceptrón simple. Se basa en una neurona artificial, llamada unidad lógica de umbral (TLU), que calcula una función lineal de sus entradas:

$$z = \sum_{i=1}^n w_i x_i + b$$

donde x_i representan las características de entrada, w_i son las ponderaciones o pesos de cada entrada y b es el término de sesgo *obias*.

El resultado de esta suma se aplica posteriormente en una función de activación:

$$a = f(z)$$

Los pesos w_i determinan la importancia que tiene cada característica de entrada en la clasificación final del modelo. Por tanto, el objetivo principal del entrenamiento será encontrar los valores óptimos de dichos pesos para que la salida generada coincida con la clase real esperada.

Sin embargo, Minsky y Papert (1969) [28], demostraron que el perceptrón simple no es capaz de modelar relaciones no lineales complejas, lo que limitaba su aplicabilidad en escenarios reales. Para superar esta limitación surge el modelo *MLP*, compuesto por múltiples capas de neuronas conectadas entre

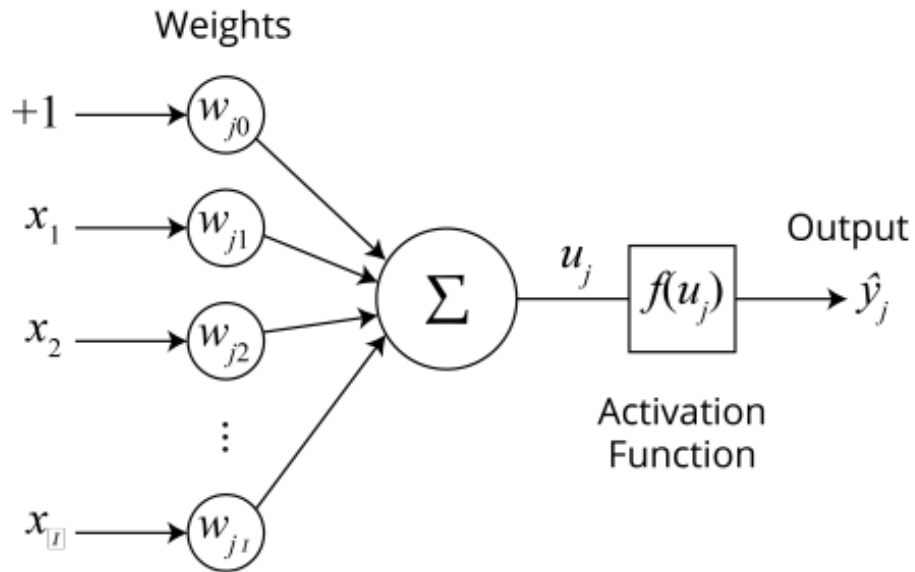


Figura 2.8: Estructura de un perceptrón [10]

sí mediante una capa de entrada, una o varias capas ocultas y una capa de salida. Cada capa, excepto la capa de salida, incluye un sesgo y está completamente conectada con la siguiente capa. Cada neurona artificial aplicará la función de activación no lineal ReLU (*Rectified Linear Unit*):

$$ReLU(x) = \max\{0, x\}$$

Durante el entrenamiento, la red neuronal ajustará de forma iterativa cada peso utilizando el algoritmo de retropropagación del error (en inglés, *backpropagation*). En sólo dos pasadas por la red (una hacia delante y otra hacia atrás), el algoritmo es capaz de calcular los gradientes del error de la red neuronal con respecto a cada uno de los parámetros del modelo. A continuación, estos gradientes se utilizan para realizar un paso de descenso de gradiente. Si se repite este proceso de calcular los gradientes automáticamente y dar un paso de descenso de gradiente, el error de la red neuronal disminuirá gradualmente hasta que finalmente alcance un mínimo. Más concretamente, el algoritmo de retropropagación funciona de la siguiente manera:

1. Preparación: se inicializan los pesos y sesgos de todas las capas con valores aleatorios.
2. Selección del minilote: se toma un conjunto de m instancias del conjunto de entrenamiento llamado minilote. Cada pasada se denomina época.

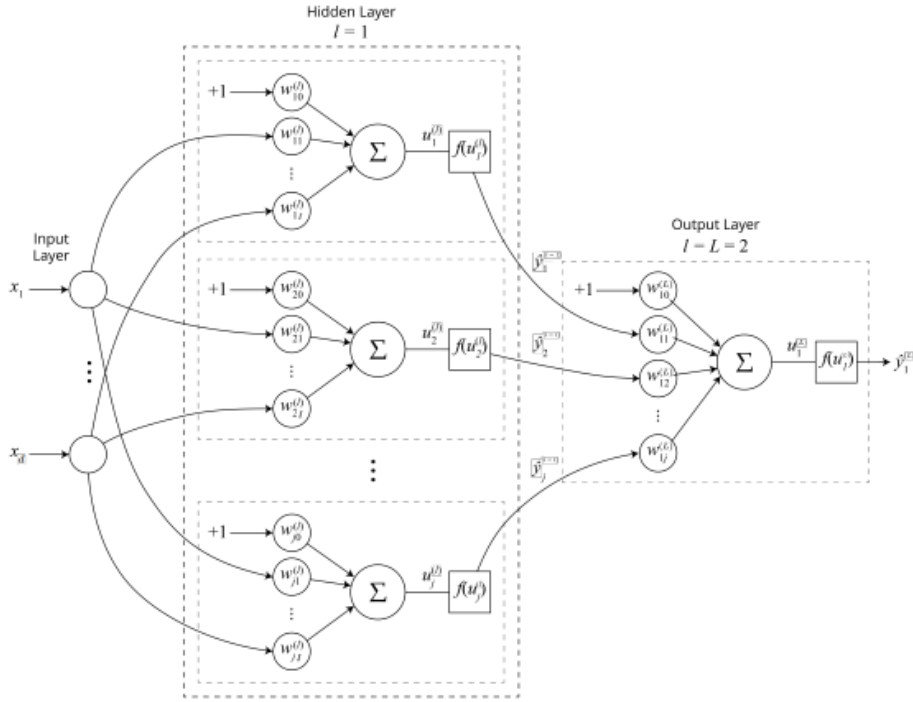


Figura 2.9: Arquitectura de un MLP [10]

3. Paso hacia delante: cada minilote entra en la red a través de la capa de entrada. A continuación, se calculan las salidas de todas las neuronas capa por capa, guardando los resultados intermedios que serán necesarios para el paso hacia atrás.
4. Cálculo del error: el algoritmo mide el error de salida de la red mediante una función de pérdida.
5. Cálculo de gradientes: usando la regla de la cadena, se calcula cuánto ha contribuido al error cada sesgo de salida y cada conexión a la capa de salida.
6. Retropropagación: se propaga el gradiente hacia atrás hasta llegar a la capa de entrada.
7. Actualización de pesos: se aplica descenso por gradiente para ajustar los pesos utilizando los gradientes de error previamente calculados. Este proceso sigue la regla:

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

donde η es la tasa de aprendizaje y L es la función de pérdida.

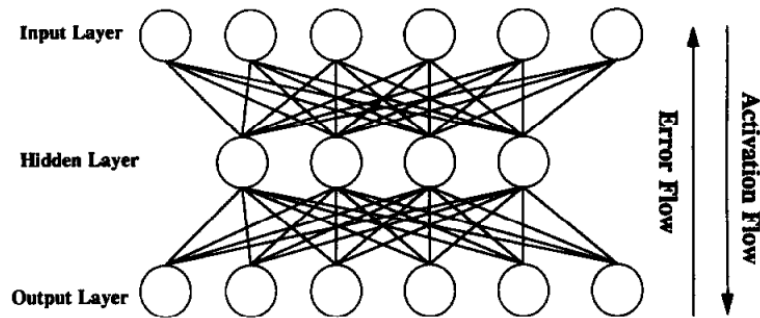


Figura 2.10: Ejemplo de una red de retropropagación totalmente conectada, con las direcciones del flujo de activación y del error indicadas [42]

Para problemas de clasificación binaria sólo se necesita una única neurona de salida que utilice la función de activación sigmoidea: la salida será un número entre 0 y 1, que se puede interpretar como la probabilidad estimada de la clase positiva.

Por otro lado, la complejidad en tiempo de entrenamiento del perceptrón multicapa se caracteriza por ser costosa en grandes volúmenes de datos y por depender de factores críticos como el tipo de algoritmo de optimización, el número de iteraciones y neuronas, etcétera.

Capítulo 3

Desarrollo del proyecto

En este capítulo se describe el proceso seguido para el diseño, implementación y entrenamiento del sistema de detección de amenazas en red. Se adopta un enfoque basado en aprendizaje automático supervisado, dado que se dispone de datos etiquetados.

En primer lugar, describiremos y justificaremos el *pipeline* de trabajo definido, estructurado en distintas fases. A continuación, realizaremos un análisis exploratorio del conjunto de datos seleccionado para detectar posibles problemas que puedan afectar al rendimiento de los modelos. Seguidamente, abordaremos las técnicas de preprocesamiento e ingeniería de características aplicadas, así como los criterios empleados para la construcción de los diferentes subconjuntos de características. Finalmente, describiremos los modelos de clasificación seleccionados y la metodología de entrenamiento seguida, detallando las decisiones adoptadas para garantizar un procedimiento experimental homogéneo y evitar la fuga de información entre los diferentes subconjuntos de datos.

3.1. Descripción general del pipeline

En la figura 3.1 se muestra el flujo de trabajo seguido durante el proyecto.

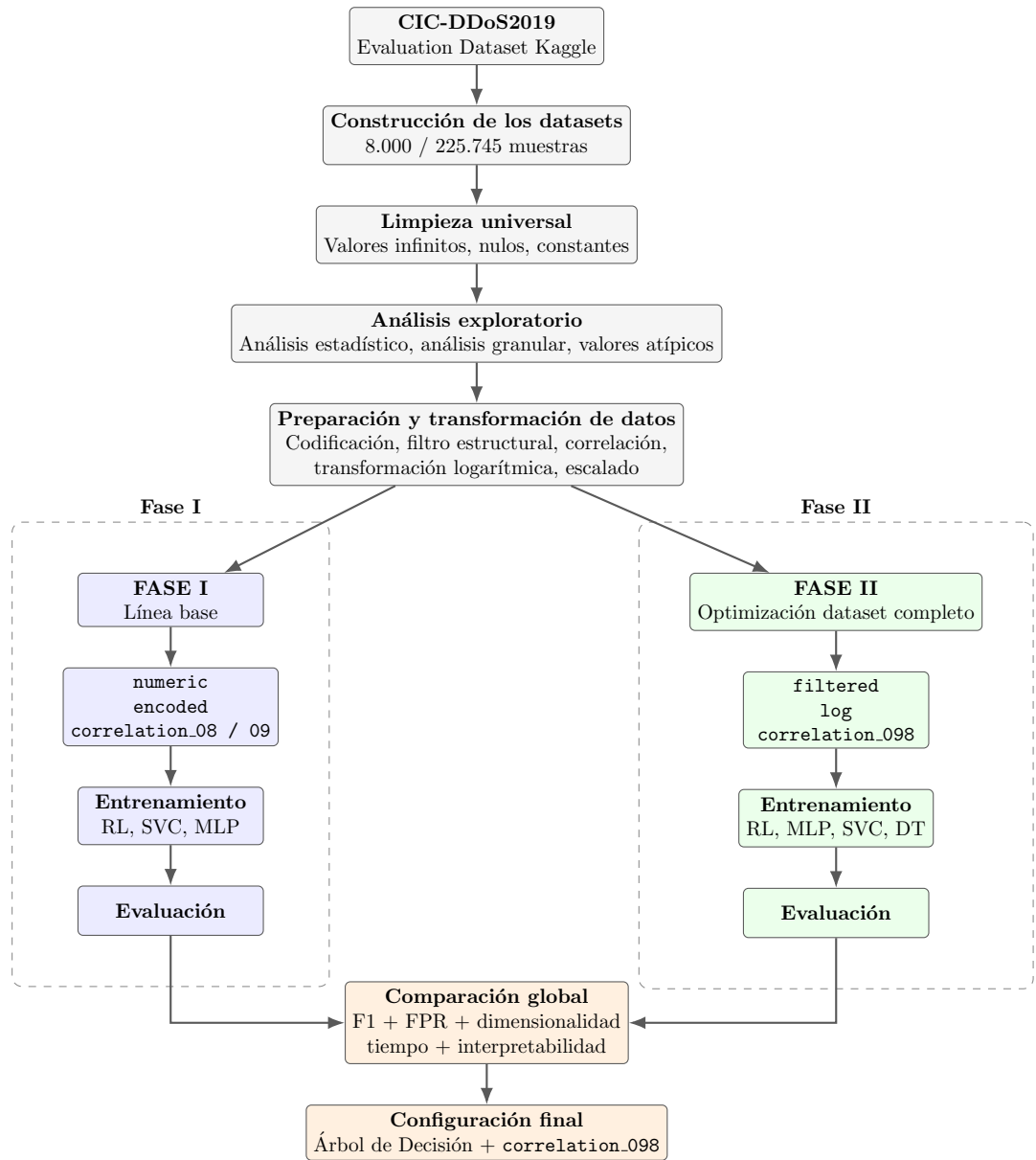


Figura 3.1: Pipeline del proyecto.

3.2. Obtención y construcción de los datos

Previo a cualquier experimentación, describiremos cómo se han extraído y recopilado los datos. Además explicaremos el procedimiento seguido para obtener los conjuntos de trabajo empleados durante el proyecto.

Se ha utilizado una versión reducida del conjunto de datos original *CIC-DDoS2019* disponible a través de la plataforma web Kaggle a partir de la

cual se han construido dos conjuntos de datos utilizados en las diferentes fases de experimentación. Durante el proceso de extracción de características, el Instituto de Ciberseguridad de Canadá, ha utilizado la herramienta de generación y análisis de flujo de tráfico de red llamada *CICFlowMeter* (formalmente *ISCXFlowMeter*) y ha obtenido más de 80 características de tráfico de red.

En primer lugar, construiremos una versión reducida y balanceada del conjunto de datos original. El objetivo es disponer de un conjunto de menor tamaño (8000 flujos de red) para realizar una primera aproximación al problema de clasificación y establecer una línea base. En segundo lugar, utilizaremos el conjunto de datos completo, compuesto por 225,745 flujos de red. El objetivo es estudiar el comportamiento de los modelos cuando se dispone de un volumen de datos considerablemente mayor. En la tabla 3.1 se muestra de manera resumida ambos conjuntos de datos.

Dataset	Muestras	Características	Distribución
<i>Reference dataset</i>	8,000	85	DDoS: 50.00 %, BENIGN: 50.00 %
<i>Full dataset</i>	225,745	85	DDoS: 56.72 %, BENIGN: 43.28 %

Cuadro 3.1: Conjuntos de datos utilizados en las diferentes fases de experimentación.

Cada conjunto de datos se ha almacenado en formato Apache Parquet (*.parquet*) ya que permite guardar los datos de forma más eficiente cuando se trabaja con un gran elevado número de registros y atributos. Otra ventaja del formato Parquet es que conserva información sobre el esquema y los tipos de datos de cada columna. Sin embargo, CSV almacena los valores como texto y requiere inferir continuamente los tipos de datos.

3.3. Limpieza universal

La calidad de limpieza de datos tiene un gran impacto en el rendimiento de los modelos. Existen diversas problemáticas en ambos conjuntos de datos como la presencia de valores infinitos, atributos constantes, duplicados o innecesarios. La evaluación de calidad de los datos establece una base sólida para continuar con las siguientes fases del proyecto garantizando que los datos sean consistentes y relevantes para los modelos.

En primer lugar, se normalizan los nombres de las columnas eliminando espacio en blanco al principio de cada nombre. Seguidamente, se realiza una evaluación de calidad que comprueba: valores ausentes, valores infinitos, atributos constantes y valores negativos. El objetivo es conservar únicamente aquellas características que describen el comportamiento generalizable del tráfico de red por lo que eliminamos atributos constantes o susceptibles de introducir fuga de información. Los valores infinitos los sustituimos por *NaN*

para unificar su tratamiento. A continuación se eliminan las filas que contienen algún valor ausente. Cabe destacar que el proceso de limpieza se aborda de **forma iterativa**, pues tras cada transformación del conjunto de datos pueden aparecer nuevas inconsistencias que dejan de aportar información relevante.

Criterio	Referencia (8k)	Completo (225k)
Muestras iniciales	8,000	225,745
Atributos iniciales	85	85
Atributos con valores ausentes	0	1
Valores ausentes	0	4
Atributos con valores infinitos	2	2
Valores infinitos	2	64
Atributos constantes	10	10
Atributos con valores negativos	3	9
<i>Init_Win_bytes_forward</i> negativos	1,345 (16.812 %)	32,925 (14.585 %)
<i>Init_Win_bytes_backward</i> negativos	3,208 (40.100 %)	88,299 (39.114 %)
Resultado de la limpieza		
Filas eliminadas	3 (0.04 %)	142 (0.06 %)
Atributos eliminados	10 (11.76 %)	10 (11.76 %)
Muestras finales	7,997	225,603
Atributos finales	75	75

Cuadro 3.2: Comparación de la evaluación de calidad y los resultados de la limpieza universal de los conjuntos de datos.

3.4. Análisis exploratorio

En este punto del trabajo, no importa si el problema es de clasificación o de regresión, ni si se trata de un aprendizaje no supervisado o por refuerzo. En un análisis exploratorio de datos, la diferencia radica únicamente en la distribución del muestreo de los datos recopilados. [3] El análisis exploratorio de datos (en inglés, Exploratory Data Analysis (EDA)) permite que cualquier conclusión que obtengamos no sea subjetiva ni producto del azar. Además, en esta etapa lo que se pretende es responder tres cuestiones principales:

- Verificar la calidad de los datos,
- analizar la distribución de las variables,
- identificar diferencias significativas entre el tráfico benigno y el tráfico malicioso.

Para ello realizaremos un análisis visual de los datos mediante histogramas, gráficos de barras, diagramas de sectores, etcétera. [3]

3.4.1. Taxonomía de las características

Para tener una visión más clara del conjunto de datos, clasificaremos las características de dos maneras. La primera según el tipo de dato y la segunda según el comportamiento del tráfico de red que representan.

Desde el punto de vista del tipo de dato, el conjunto de datos contiene 8 características categóricas, 43 discretas y 24 continuas. Sin embargo, esta clasificación no proporciona información sobre el contexto del tráfico de red. Es por ello que la segunda **taxonomía basada en grupos semánticos** facilita un análisis ordenado y una toma de decisiones basada en patrones de comportamiento del tráfico de red. Las características quedan organizadas en nueve grupos: identificadores de flujo (8 características) en 3.3, características de volumen de tráfico (8) en 3.4, características de distribución del tamaño de paquetes (16) en 3.5, características temporales (23) en 3.6, características de tasa de tráfico (4) en 3.7, características de banderas de control TCP (12) en 3.11, características de cabeceras de paquetes (3) en 3.9, características de transferencias bulk (6) en 3.10, características de ventana TCP (2) en 3.12, características de conexión (3) en 3.8. Esta organización se utilizará en las siguientes etapas del proyecto para estructurar y fundamentar las decisiones de eliminación, transformación y selección de atributos.

Atributo	Descripción
Flow ID	Identificador único del flujo de red
Source IP	Dirección IPv4 de origen del flujo
Destination IP	Dirección IPv4 de destino del flujo
Source Port	Puerto de origen
Destination Port	Puerto de destino
Protocol	Protocolo de transporte
Timestamp	Marca temporal de inicio (o registro) del flujo.
Label	Etiqueta de clase (benigno/malicioso).

Cuadro 3.3: Identificación de flujo

Atributo	Descripción
Total Fwd Packets	Número total de paquetes enviados
Total Backward Packets	Número total de paquetes recibidos
Total Length of Fwd Packets	Suma total de longitudes de paquetes enviados
Total Length of Bwd Packets	Suma total de longitudes de paquetes recibidos
Subflow Fwd Packets	Paquetes por subflujo enviados
Subflow Bwd Packets	Paquetes por subflujo recibidos
Subflow Fwd Bytes	Bytes por subflujo enviados
Subflow Bwd Bytes	Bytes por subflujo recibidos

Cuadro 3.4: Volumen de tráfico

Atributo	Descripción
Fwd Packet Length Max	Longitud máxima de paquetes enviados.
Fwd Packet Length Min	Longitud mínima de paquetes enviados.
Fwd Packet Length Mean	Longitud media de paquetes enviados
Fwd Packet Length Std	Desviación típica de las longitudes de paquetes enviados
Bwd Packet Length Max	Longitud máxima de paquetes recibidos
Bwd Packet Length Min	Longitud mínima de paquetes recibidos
Bwd Packet Length Mean	Longitud media de paquetes recibidos
Bwd Packet Length Std	Desviación típica de las longitudes de paquetes recibidos
Min Packet Length	Longitud mínima global de paquetes en el flujo
Max Packet Length	Longitud máxima global de paquetes en el flujo
Packet Length Mean	Longitud media de paquetes
Packet Length Std	Desviación típica de la longitud de paquetes
Packet Length Variance	Varianza de la longitud de paquetes
Average Packet Size	Tamaño medio de los paquetes del flujo
Avg Fwd Packet Size	Tamaño medio de paquetes enviados
Avg Bwd Packet Size	Tamaño medio de paquetes recibidos

Cuadro 3.5: Distribución del tamaño de paquetes

Atributo	Descripción
Flow Duration	Duración total del flujo (tiempo entre inicio y fin)
Flow IAT Mean	Media del tiempo entre dos paquetes
Flow IAT Std	Desviación típica del tiempo entre dos paquetes en el flujo
Flow IAT Max	Mayor tiempo entre dos paquetes del flujo
Flow IAT Min	Menor tiempo entre dos paquetes del flujo
Fwd IAT Total	Suma total de los tiempos entre dos paquetes enviados
Fwd IAT Mean	Media del tiempo entre dos paquetes enviados
Fwd IAT Std	Desviación típica del tiempo entre dos paquetes enviados
Fwd IAT Max	Mayor tiempo entre dos paquetes enviados
Fwd IAT Min	Menor tiempo entre dos paquetes enviados
Bwd IAT Total	Suma total de los tiempos entre dos paquetes recibidos
Bwd IAT Mean	Media del tiempo entre dos paquetes recibidos
Bwd IAT Std	Desviación típica del tiempo entre dos paquetes recibidos
Bwd IAT Max	Mayor tiempo entre dos paquetes recibidos
Bwd IAT Min	Menor tiempo entre dos paquetes recibidos
Active Mean	Media de la duración de los periodos activos del flujo
Active Std	Desviación típica de la duración de los periodos activos del flujo
Active Max	Duración máxima de los periodos activos del flujo
Active Min	Duración mínima de los periodos activos del flujo
Idle Mean	Media de la duración de los periodos de inactividad del flujo
Idle Std	Desviación típica de la duración de los periodos de inactividad del flujo
Idle Max	Duración máxima de los periodos de inactividad del flujo
Idle Min	Duración mínima de los periodos de inactividad del flujo

Cuadro 3.6: Características temporales

Atributo	Descripción
Flow Bytes/s	Bytes por segundo en el flujo
Flow Packets/s	Paquetes por segundo en el flujo
Fwd Packets/s	Paquetes por segundo enviados
Bwd Packets/s	Paquetes por segundo recibidos

Cuadro 3.7: Tasa de tráfico

Atributo	Descripción
Down/Up Ratio	Relación entre el tráfico recibido y enviado
act_data_pkt_fwd	Paquetes con datos efectivos enviados
min_seg_size_forward	Tamaño mínimo de segmento TCP enviado

Cuadro 3.8: Características de conexión

Atributo	Descripción
Fwd Header Length	Longitud (bytes) de la cabecera de paquetes enviados
Bwd Header Length	Longitud (bytes) de la cabecera de paquetes recibidos
Fwd Header Length.1	Duplicado de Fwd Header Length

Cuadro 3.9: Cabeceras (*header*)

Atributo	Descripción
Fwd Avg Bytes/Bulk	Media de bytes enviados por operación bulk enviada
Bwd Avg Bytes/Bulk	Media de bytes recibidos por operación bulk recibida
Fwd Avg Packets/Bulk	Media de paquetes enviados por operación bulk enviada
Bwd Avg Packets/Bulk	Media de paquetes recibidos por operación bulk recibida
Fwd Avg Bulk Rate	Tasa media de transferencia de datos enviada por operación bulk enviada
Bwd Avg Bulk Rate	Tasa media de transferencia de datos recibida por operación bulk recibida

Cuadro 3.10: Transferencias bulk

Atributo	Descripción
FIN Flag Count	Conteo de banderas FIN observadas
SYN Flag Count	Conteo de banderas SYN observadas
RST Flag Count	Conteo de banderas RST observadas
PSH Flag Count	Conteo de banderas PSH observadas
ACK Flag Count	Conteo de banderas ACK observadas
URG Flag Count	Conteo de banderas URG observadas
CWE Flag Count	Conteo de banderas CWE observadas
ECE Flag Count	Conteo de banderas ECE observadas
Fwd PSH Flags	Indicador de PSH enviada
Bwd PSH Flags	Indicador de PSH recibidos
Fwd URG Flags	Indicador de URG enviada
Bwd URG Flags	Indicador de URG recibidos

Cuadro 3.11: Banderas TCP

Atributo	Descripción
Init_Win_bytes_forward	Tamaño inicial de ventana enviada
Init_Win_bytes_backward	Tamaño inicial de ventana recibida

Cuadro 3.12: Ventana TCP (*initial window*)

3.4.2. Distribución de clases

El análisis de la variable objetivo *Label* nos ayuda a comprender cómo se distribuyen las clases en el conjunto de datos. Como podemos observar en los gráficos de sectores 3.2 y 3.3, existen diferencias entre los dos conjuntos de datos empleados durante el proyecto. En el conjunto de referencia, la distribución es completamente equilibrada, con 4,000 muestras correspondientes tanto a tráfico *DDoS* como a tráfico *BENIGN*. En el conjunto de datos completo existe un ligero desequilibrio entre las clases, con una ligera predominancia de tráfico malicioso.

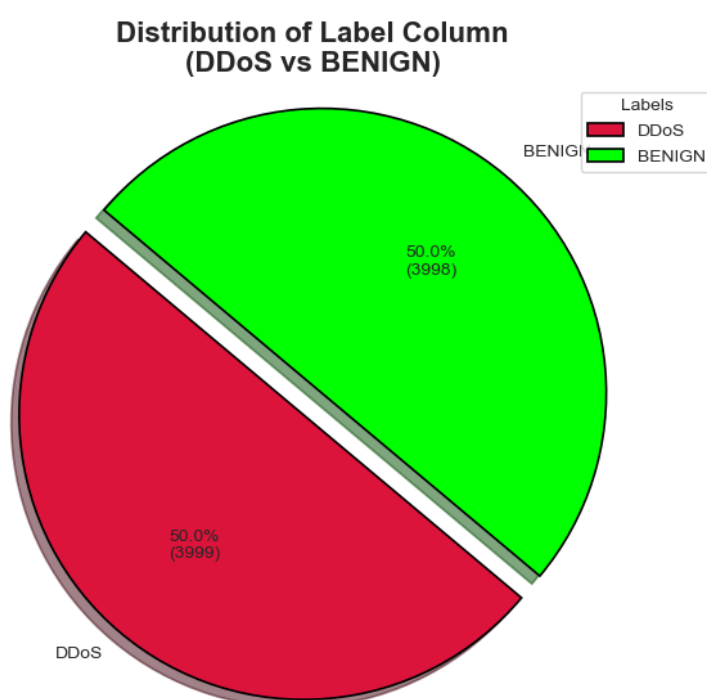


Figura 3.2: Distribución de clases en el conjunto de referencia

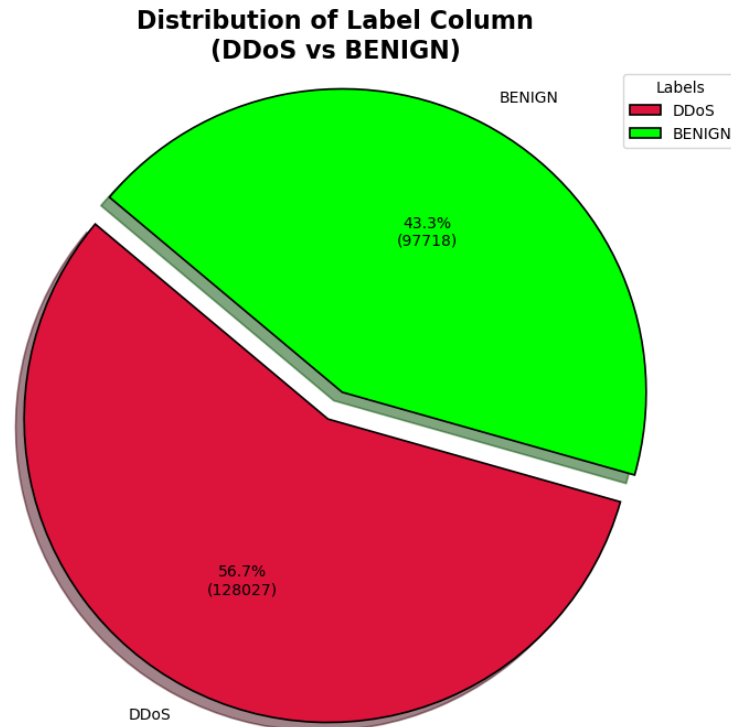


Figura 3.3: Distribución de clases en el conjunto completo

3.4.3. Análisis temporal del tráfico

Para entender cómo se llevan a cabo los ataques DDoS realizaremos un análisis temporal del tráfico de red. Si analizamos el conjunto de datos completo, observamos que a largo del tiempo hay un patrón claramente diferenciado entre el tráfico normal y el tráfico malicioso. En la figura 3.4 se observa claramente el comienzo del ataque DDoS el 2017-07-07 a las 03:56:00 y, tras 20 minutos, a las 04:16:00 finaliza. Durante ese intervalo, el tráfico benigno también aumenta, pero con menor intensidad y mayor variabilidad. Tras el pico, el tráfico DDoS cae bruscamente, mientras que el tráfico benigno mantiene un comportamiento más distribuido.

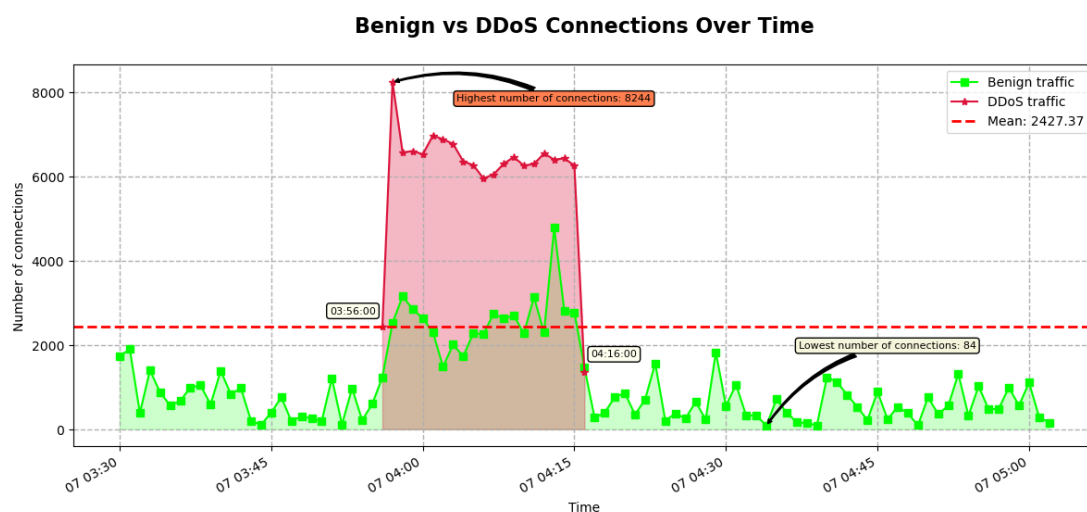


Figura 3.4: Análisis temporal del tráfico de red

A continuación, detallaremos la evolución del número de flujos a lo largo del tiempo, el número de flujos medio y la identificación de picos de actividad para cada tipo de tráfico y así, comprender con mayor precisión qué patrones característicos aparecen en cada escenario. Por un lado, en la figura 3.5 observamos que el tráfico benigno tiene un número de conexiones heterogéneo, cubriendo periodos de mayor intensidad (máximo de 4785 conexiones) y periodos de menor intensidad (mínimo de 84 conexiones). En la figura, la línea roja representa la media del número de conexiones, cuyo valor es 1050.73. La representación del tráfico normal es coherente, una distribución temporal menos predecible.

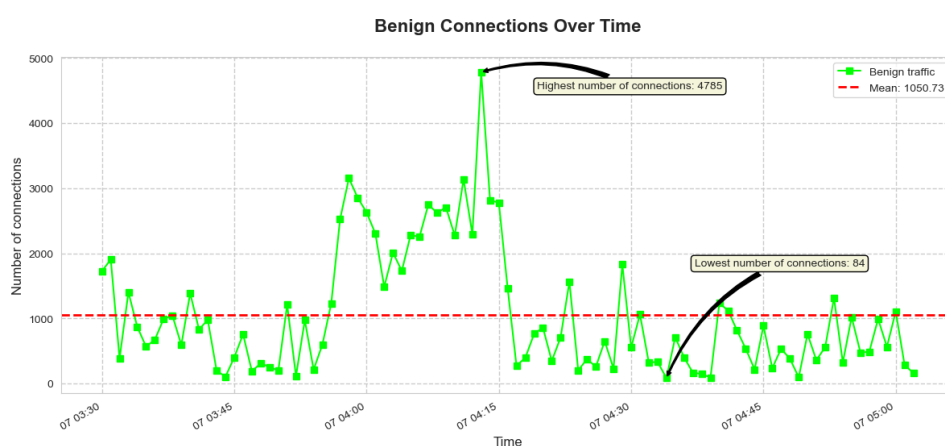


Figura 3.5: Análisis temporal del tráfico benigno

Por otro lado, en la figura 3.6 observamos que el tráfico malicioso presenta una evolución del número de conexiones mucho más concentrada, cubriendo una actividad cuyo valor de conexiones máximo es 8244 y de valor mínimo 1348. A pesar de haber una diferencia abrupta entre ambos picos, se observa un actividad mucho más uniforme, con un valor medio de conexiones de 6096.52.

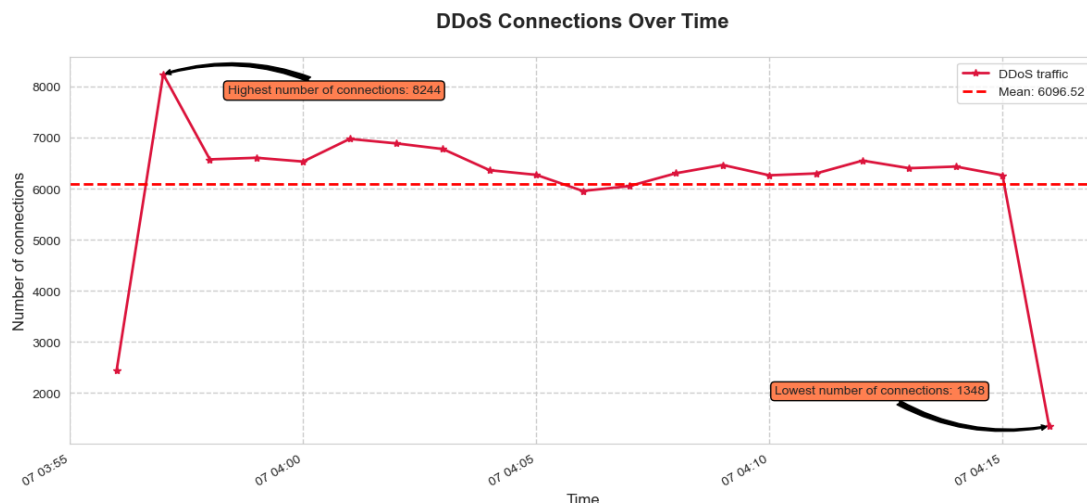


Figura 3.6: Análisis temporal tráfico malicioso

Este análisis temporal del tráfico permite concluir que el comportamiento es coherente con la naturaleza de los ataques DDoS, cuyo objetivo es la saturación de un sistema informático mediante el envío de múltiples solicitudes a la red víctima en un corto periodo de tiempo. Esta correcta reproducción de un ataque DDoS en el *dataset* es fundamental para la validez del modelo.

3.4.4. Análisis preliminar de variables categóricas

Con el fin de identificar patrones estructurales en el tráfico y tener una visión preliminar, se han analizado las variables categóricas del conjunto de datos completo, compuesto por 225,745 flujos de red. El objetivo es estudiar si ciertas características contextuales ayudan a diferenciar el tráfico benigno del tráfico malicioso. El tráfico malicioso presenta las siguientes características:

- Puerto atacado: 80/TCP (HTTP).
- Protocolo usado por el atacante: 6 (TCP).
- Puertos origen observados: 50975/TCP, 50086/TCP, 50722/TCP, 50064/TCP, 51004/TCP.

- IP del atacante: 172.16.0.1.
- IP de la víctima: 192.168.10.50.

El ataque ha sido originado desde una única dirección IP, por lo que nos encontramos ante un ataque DDoS de un solo nodo, DoS. El tráfico se dirige exclusivamente a servicios HTTP, lo que puede indicar un ataque a nivel de aplicación (Capa 7). Algunos tipos comunes de este ataque son la inundación HTTP, la inundación con peticiones GET/POST u otros tipos de ataques que agotan los recursos del servidor web. Por tanto, concluimos que esta información contextual denota un comportamiento típico de un ataque programado más que de una actividad legítima de un usuario. Por otro lado, el tráfico benigno presenta las siguientes características:

- Puertos específicos: 53/UDP (DNS), 443/TCP (HTTPS), 80/TCP (HTTP).
- Protocolos específicos: 6 (TCP) y 17 (UDP).
- Puertos origen observados: 80/TCP (HTTP), 443/TCP (HTTPS), 123/UDP (NTP).
- IPs de usuarios legítimos: 192.168.10.50, 192.168.10.15, 192.168.10.12...
- La mayoría de las solicitudes de usuarios legítimos se dirigen a la IP del atacante (172.16.0.1).

A partir del análisis precedente podemos confirmar que el tráfico malicioso tiene un patrón muy concentrado a diferencia del tráfico benigno, con una actividad mixta: muchos hosts distintos, diversos puertos de servicios web, mezcla de protocolos UDP y TCP, etcétera. Con este análisis confirmamos que las variables categóricas presentan cierto poder discriminatorio para la detección de ataques DDoS. Si bien estos identificadores de flujo pueden mejorar el rendimiento del modelo, también incrementan el riesgo de que el clasificador memorice información específica de la captura en vez de aprender patrones de comportamiento generalizables del tráfico de red. Este fenómeno puede producir una fuga de información (en inglés, *data leakage*) reduciendo la capacidad del modelo para generalizar en entornos reales.

3.4.5. Análisis estadístico descriptivo

A continuación, cambiamos el enfoque para realizar un análisis estadístico descriptivo con el objetivo de obtener una visión global del comportamiento de las variables del conjunto de datos completo e identificar distribuciones asimétricas, diferencias en la escala de las variables, atributos anómalos, etcétera. Hemos empleado el método *pandas.DataFrame.describe* que proporciona las principales medidas estadísticas de cada atributo numérico:

número de observaciones, media, desviación típica, valores mínimo y máximo y percentiles primero (25 %), segundo o mediana (50 %) y tercero (75 %). Las conclusiones obtenidas son las siguientes:

- Variables correspondientes al volumen de tráfico de subflujos respecto a sus variables totales, así como variables de cabeceras de los paquetes, presentan valores estadísticos idénticos. Esto sugiere la presencia de atributos redundantes que se estudiarán mediante un análisis de correlación. Véase en 3.5.3
- En general, las banderas de control TCP muestran una baja variabilidad. Únicamente las banderas de control ACK, PSH y URG presentan una mayor capacidad discriminativa. Se realizará un análisis más en profundidad sobre este grupo de variables. Véase en 3.4.6
- La mayoría de variables presentan distribuciones asimétricas pronunciadas. Por ejemplo, las variables relacionadas con el tamaño de los paquetes, las variables temporales o las variables de tasas de transmisión presentan desviaciones estándar elevadas con valores máximos muy superiores al tercer percentil. Este comportamiento también indica la presencia de posibles valores atípicos que deberá analizarse mediante diagramas de cajas. Véase en 3.4.7
- Variables temporales como *Active* o *Idle* presentan una elevada concentración de ceros. Será necesario un análisis concreto de estas variables para investigar si son atributos casi constantes. Véase en 3.4.6
- Finalmente, existen diferencias de escala entre atributos muy elevadas. Encontramos desde variables binarias (banderas de control TCP) hasta otras cuyos valores alcanzan varios miles de millones ($\max(\text{Flow Bytes/s}) = 2\,070\,000\,000$). Esta heterogeneidad implica aplicar un proceso de estandarización para aquellos modelos sensibles a la escala de los datos como MLP.

3.4.6. Análisis granular de características

Banderas de control TCP

La figura 3.7 muestra las principales banderas de control TCP presentes en los flujos de red. Se observa que *FIN*, *RST*, *ECE Flag Count* permanecen a cero en prácticamente todas las conexiones (99.7 %, 99.9 %, 99.9 % respectivamente). Concluimos que dichas características apenas contienen información útil, por lo que son candidatos adecuados para su eliminación. Seguidamente, *SYN Flag Count* y *Fwd PSH Flags* codifican la misma información al presentar exactamente la misma distribución (218,114 valores a cero y 7489 valores a 1). Sólo se conservará *SIN Flag Count*.

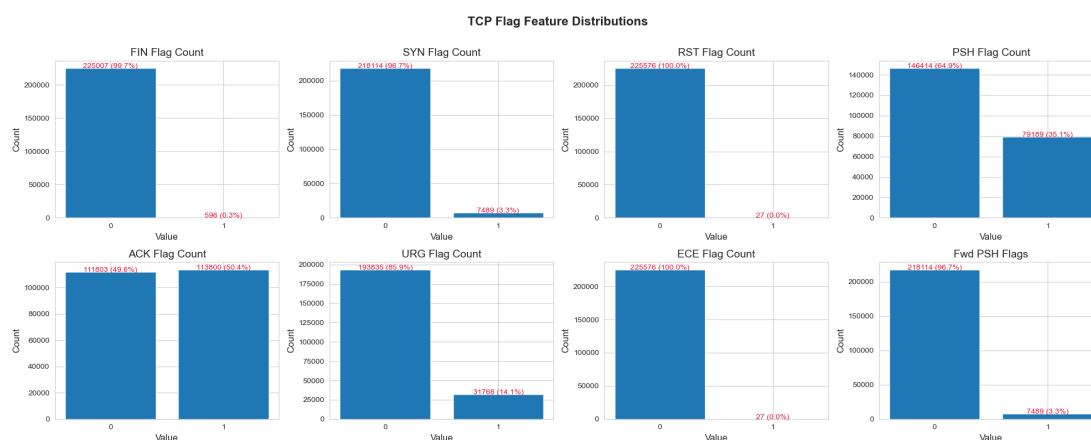


Figura 3.7: Análisis de banderas de control TCP

Tamaño de la ventana inicial TCP

Las dos características asociadas al tamaño inicial de la ventana TCP presentan problemas de calidad. `Init_Win_bytes_forward` contiene 32,925 valores negativos (14,6% del conjunto de datos completo), mientras que `Init_Win_bytes_backward` presenta 88,274 valores negativos (39%). En ambos casos el valor mínimo es -1 . Aunque estas características contienen información relevante al protocolo TCP, la elevada presencia de estos valores negativos implica eliminar ambas características. Mantener estas observaciones podría introducir patrones artificiales no válidos. En la figura 3.8 podemos observar un ejemplo de la distribución de la ventana TCP recibida.

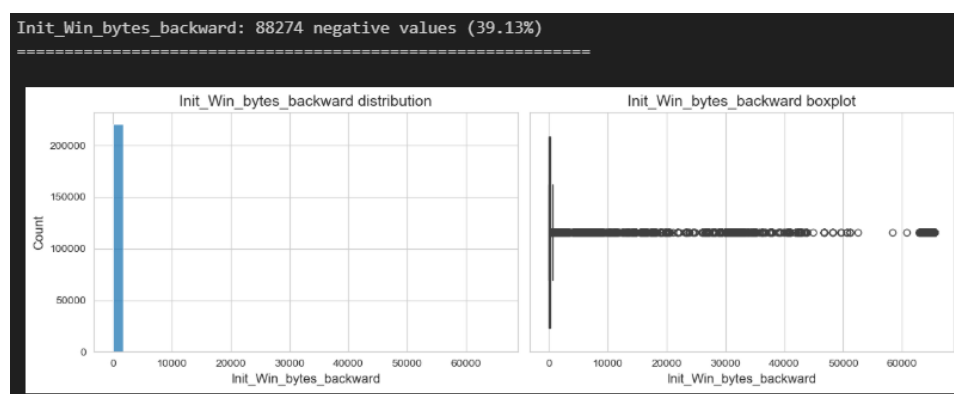


Figura 3.8: Análisis del tamaño de la ventana inicial recibida

Variables temporales

Las variables correspondientes a los tiempos de actividad e inactividad (Active, Idle Mean, Std, Min, Max) presentaban una alta concentración de valores iguales a cero en el análisis anterior. En la figura 3.9 se confirma este comportamiento utilizando un contador del número de ceros y un histograma de frecuencias. *Active Std* e *Idle Std* presentan un 97.48 % y 90 % de valores iguales a cero. El resto de variables analizadas también presentan una alta concentración de ceros pero el histograma permite revelar que tienen una distribución de valores más amplia. Por este motivo, *Active Std* e *Idle Std* son candidatos adecuados para su eliminación.

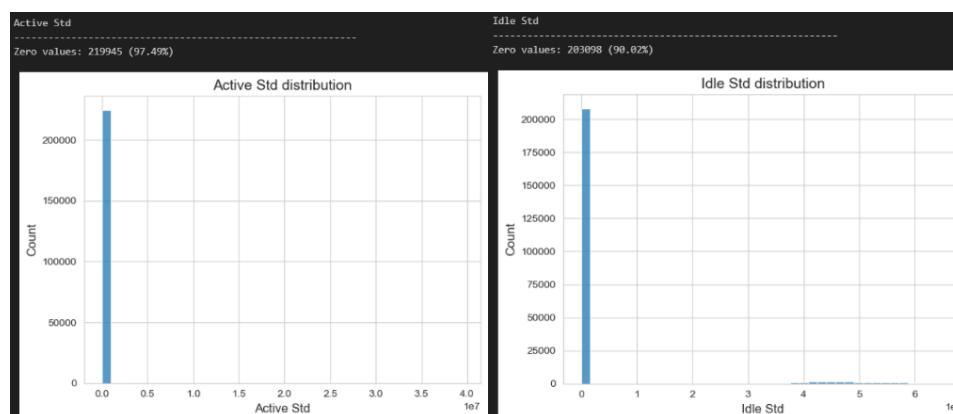


Figura 3.9: Análisis variables temporales *Active* e *Idle*

3.4.7. Análisis de valores atípicos

Como se ha concluido en el análisis estadístico descriptivo en 3.4.5, la mayoría de variables presentan distribuciones asimétricas pronunciadas indicando la posible presencia de valores atípicos. Mediante diagramas de cajas e histogramas acompañados por una curva de densidad identificaremos más detalladamente cómo son estas distribuciones asimétricas y determinaremos si pueden afectar negativamente al proceso de entrenamiento de los modelos. Agruparemos los gráficos según el significado semántico de las características. En la figura 3.10 se muestran los resultados correspondientes antes de aplicar cualquier transformación al grupo de características de tasas de transmisión. El resto de grupos de características, a excepción de menos *Down/Up Ratio* y *min_seg_size_forward*, también presentan una asimetría positiva, con la mediana situada cerca del primer percentil (Q1). Los diagramas de cajas muestran una gran cantidad de valores atípicos concentrados en el extremo superior. Este tipo de valores extremos no tienen porqué ser errores, sino observaciones que representan un comportamiento real del

tráfico ya que los ataques *DDoS* generan volúmenes de tráfico anómalos con respecto al tráfico normal.

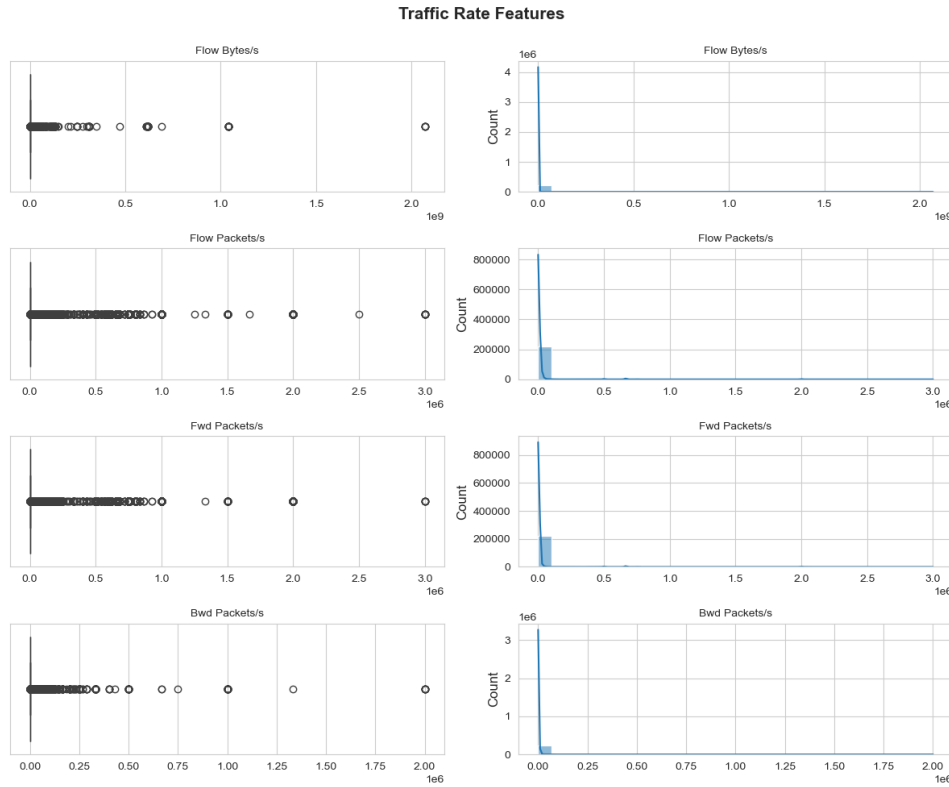


Figura 3.10: Análisis de valores atípicos sobre características de tasas de transmisión

3.4.8. Conclusiones del análisis exploratorio

Las observaciones realizadas en el análisis exploratorio son el fundamento de las siguientes fases del proyecto, especialmente en lo relativo al preprocesamiento de datos y la selección de características. La distribución de clases en ambos conjuntos de datos de trabajo no presenta un desequilibrio extremo para una clasificación de aprendizaje supervisado. También hemos observado como el comportamiento temporal del tráfico presenta patrones realistas de un ataque de denegación de servicio distribuido (DDoS). Sin embargo, dado que el tráfico malicioso proviene de una única IP, el tipo de ataque a detectar será un ataque de denegación de servicio de un único origen. El tráfico benigno también presenta un comportamiento realista: menor intensidad mayor variabilidad. En conjunto, el análisis estadístico ha permitido identificar características con información redundante, escasa capacidad discriminativa o valores negativos que violan la naturaleza

del tráfico de red. Por último, el análisis de valores atípicos muestra distribuciones fuertemente asimétricas así como valores extremos por lo que se planteará el uso de transformaciones que permitan reducir la influencia de estos valores sin eliminarlos.

3.5. Preparación y transformación de los datos

Una vez finalizado el análisis exploratorio, aplicaremos diferentes técnicas de preparación y transformación sobre los conjuntos de datos para obtener las representaciones adecuadas para el entrenamiento de los modelos. Las decisiones adoptadas en esta sección se basan en las problemáticas identificadas durante el análisis exploratorio. Destacamos que las técnicas aplicadas no son idénticas en ambas fases de experimentación, ya que cada una intenta responder un objetivo metodológico diferente. En la primera fase, se aplica únicamente una codificación de variables categóricas y se realiza un análisis de correlación automatizado. En la segunda fase, se añaden técnicas de preparación más elaboradas sobre el conjunto completo.

3.5.1. Codificación y escalado de variables categóricas

En primer lugar, hemos optado por codificar los atributos *Source IP* y *Destination IP* a un formato entero sin signo de 32 bits entre 0 y 4.294.967.295. Esta es la forma en que los sistemas operativos almacenan las direcciones IPv4 en memoria y es cómo se transmiten en muchos protocolos. A continuación, escalamos ambos atributos usando *MinMax Scaler* a un rango fijo [0-1].

$$192.168.1.1 \longrightarrow 3234201857 \longrightarrow 0.75$$

Para el atributo *Protocol* hemos optado por codificarlo usando *One-Hot Encoder* creando una columna nueva para cada protocolo. El atributo sólo toma tres valores numéricos, 0(HOPOPT), 6(TCP), 17(UDP), por lo que no cargaremos al modelo con muchas características de entrada. Con *Label Encoding* el modelo podría asumir que $HOPOPT > TCP > UDP$. Los protocolos son categorías sin relación ordinal, y *One-Hot Encoder* resuelve el problema.

Protocol		is_0	is_6	is_17
0		1	0	0
6	→	0	1	0
17		0	0	1

Por último, las variables *Source Port* y *Destination Port* al tener una alta cardinalidad primero serán categorizadas en tres grupos distintos: *'Well-Known (0-1023)'*, *'Registered (1024-49151)'* y *'Dynamic/Private (49152-65535)'*. De esta manera preservamos el contexto del tráfico y podremos

aplicar *One-Hot Encoder*.

Source Port		Source Port	
80		Well-known (0-1023)	
1056		Registered (1024-49151)	
46879		Dynamic/Private (49152-65535)	

→

source_port_well_known	source_port_registered	source_port_dynamic_private
1	0	0
0	1	0
0	0	1

→

3.5.2. Filtro estructural

El filtro estructural es una técnica que sólo hemos aplicado al conjunto de datos completo de la segunda fase. A partir de las observaciones del análisis granular de varios grupos de características en 3.4.6, eliminaremos las siguientes de características: *Fwd PSH Flags*, *FIN Flag Count*, *RST Flag Count*, *ECE Flag Count*, *Active Std*, *Idle Std*, *Init_Win_bytes_forward*, *Init_Win_bytes_backward*. Adicionalmente, en esta segunda fase eliminaremos todas las características categóricas: *Flow ID*, *Source IP*, *Destination IP*, *Timestamp*, *Source Port*, *Destination Port*, *Protocol*. La justificación experimental de esta decisión se analizará posteriormente en el capítulo de experimentación. Como resultado de esta selección de características obtenemos el conjunto de datos llamado *full-225k-filtered*, que constituye una representación de partida para las transformaciones posteriores de la segunda fase.

3.5.3. Dependencias entre características y análisis de correlación

El análisis exploratorio nos ha dado ciertas pistas sobre posibles redundancias en los conjuntos de datos. Para ello utilizaremos el coeficiente de correlación de Pearson, propuesto originalmente por Karl Pearson, que permite cuantificar la intensidad y dirección de la relación lineal entre dos variables con un valor comprendido entre 1 y -1. Los valores cercanos a 1 o -1 indican una fuerte correlación, mientras que los valores cercanos a 0 sugieren una relación lineal mínima o nula. [33] En la primera fase se ha empleado un método automatizado, mientras que en la segunda fase se realiza un análisis más granular basado en los grupos semánticos de características y en un criterio experto para seleccionar las características conservadas.

Análisis de correlación automatizado - Fase I

Para la reducción automatizada hemos utilizado una implementación en Python de la función `findCorrelation` del paquete `caret` de R. Obtuvimos

mos dicha función a través de una respuesta publicada en el foro de *Stack Overflow*, donde se reproduce la lógica de `caret::findCorrelation` [30]. Esta función realiza lo siguiente: si una pareja de características tiene una correlación alta, se analiza la correlación absoluta media de cada variable y se elimina la que tenga la mayor correlación absoluta media [35].

Calcularemos la matriz de correlación de Pearson usando exclusivamente las características del conjunto de entrenamiento, evitando así una posible fuga de datos. El análisis lo realizamos sobre el conjunto de datos con mayor número de características de la primera fase, que contiene las variables categóricas previamente codificadas. A partir de la matriz, utilizamos dos umbrales diferentes, $R > 0.8$ y $R > 0.9$. Generamos así dos conjuntos de características independientes que permitirán estudiar experimentalmente el efecto de distintos niveles de reducción de dimensión. En la tabla 3.13 se observan los dos conjuntos de datos reducidos.

Conjunto	Umbral	Características iniciales	Eliminadas	Restantes
correlation_08	$R > 0.8$	78	43	35
correlation_09	$R > 0.9$	78	37	41

Cuadro 3.13: Reducción de características mediante análisis de correlación

Análisis de correlación por grupos semánticos - Fase II

Dado el elevado número de atributos, realizamos el análisis sobre el conjunto de datos resultante del filtro estructural. Para evitar *data leakage* calcularemos las correlaciones exclusivamente sobre el conjunto de entrenamiento. A partir de este conjunto, realizamos el análisis de forma independiente para cada uno de los grupos semánticos definidos anteriormente, con el objetivo de facilitar su visualización e interpretación. En cada matriz de correlación se mostrarán aquellas parejas de atributos cuyo coeficiente de correlación absoluto sea **0.98**. Eligiendo este valor tan elevado nos aseguramos que los atributos de cada pareja describan prácticamente el mismo comportamiento. En cada caso, se utilizará la selección de variables bajo criterio experto. Tras la eliminación de la variable/s redundante/s se vuelve a calcular la matriz de correlación utilizando únicamente las características restantes para verificar el proceso.

En la parte izquierda de la figura 3.11 se identifican cuatro parejas con una correlación perfecta:

- *Total Fwd Packets* $\longleftrightarrow$ *Subflow Fwd Packets*: $r = 1.000$
- *Total Length of Fwd Packets* $\longleftrightarrow$ *Subflow Fwd Bytes*: $r = 1.000$
- *Total Backward Packets* $\longleftrightarrow$ *Subflow Bwd Packets*: $r = 1.000$
- *Total Length of Bwd Packets* $\longleftrightarrow$ *Subflow Bwd Bytes*: $r = 1.000$

Conservamos las variables *Total Fwd Packets*, *Total Length of Fwd Packets*, *Total Backward Packets*, *Total Length of Bwd Packets*.

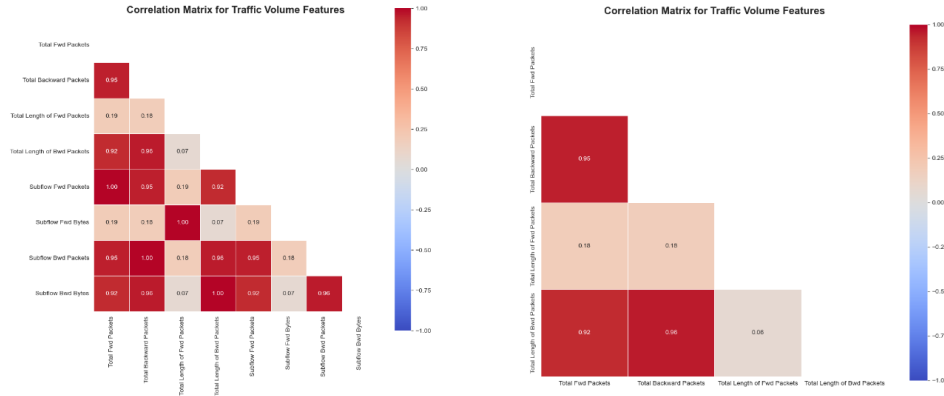


Figura 3.11: Matrices de correlación de volumen de tráfico pre/post eliminación

La parte izquierda de la figura 3.12 presenta el siguiente grupo de correlaciones muy elevadas:

- *Fwd Packet Length Max* $\longleftrightarrow$ *Fwd Packet Length Std*: $r = 0.992$
- *Bwd Packet Length Max* $\longleftrightarrow$ *Bwd Packet Length Std*: $r = 0.993$
- *Max Packet Length* $\longleftrightarrow$ *Packet Length Std*: $r = 0.984$
- *Packet Length Mean* $\longleftrightarrow$ *Average Packet Size*: $r = 0.999$
- *Fwd Packet Length Mean* $\longleftrightarrow$ *Avg Fwd Segment Size*: $r = 1.000$
- *Bwd Packet Length Mean* $\longleftrightarrow$ *Avg Bwd Segment Size*: $r = 1.000$

Conservamos las variables con una nomenclatura más descriptiva (*Fwd Packet Length Std*, *Bwd Packet Length Std*, *Max Packet Length*, *Packet Length Mean*, *Fwd Packet Length Mean* y *Bwd Packet Length Mean*), eliminando sus equivalentes.

En la matriz de la izquierda de 3.13 identificamos las siguientes variables temporales correlacionadas:

- *Flow Duration* $\longleftrightarrow$ *Fwd IAT Total*: $r = 0.997$
- *Flow IAT Max* $\longleftrightarrow$ *Fwd IAT Max*: $r = 0.995$
- *Fwd IAT Std* $\longleftrightarrow$ *Fwd IAT Max*: $r = 0.981$
- *Active Mean* $\longleftrightarrow$ *Active Min*: $r = 0.983$
- *Flow IAT Max* $\longleftrightarrow$ *Idle Max*: $r = 0.997$

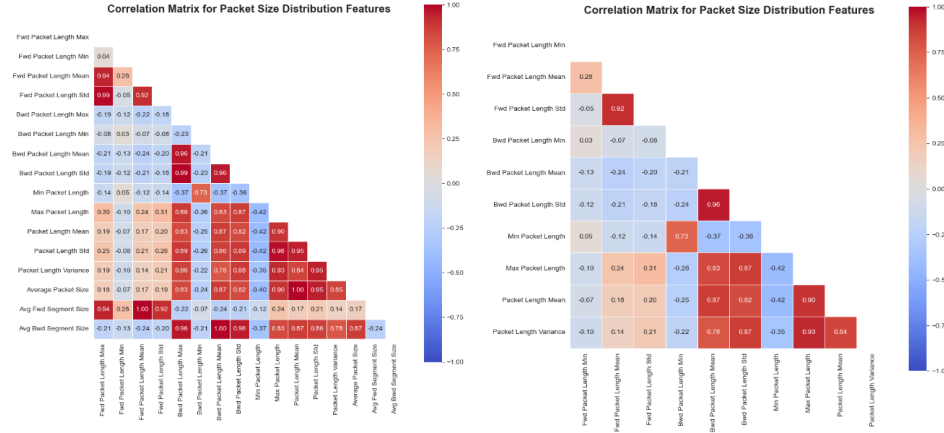


Figura 3.12: Matrices de correlación de tamaño de los paquetes pre/post eliminación

- $Fwd\ IAT\ Max \longleftrightarrow Idle\ Max: r = 0.993$

Conservamos las variables que describen de forma más general el comportamiento temporal del flujo: *Flow Duration*, *Flow IAT Max*, *Fwd IAT Std* y *Active Mean*.

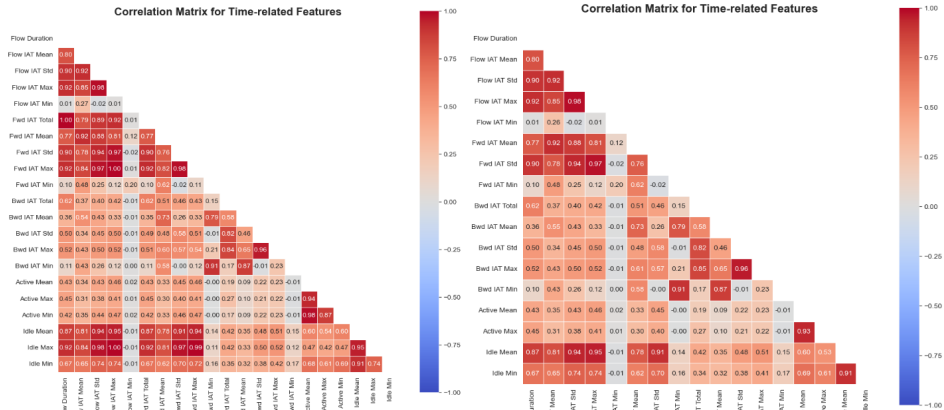


Figura 3.13: Matrices de correlación de temporales pre/post eliminación

Dentro de este grupo de características sólo encontramos la siguiente parejas. Véase figura 3.14:

- $Flow\ Packets/s \longleftrightarrow Fwd\ Packets/s: r = 0.986$

Dado que *Flow Packets/s* representa la tasa global de paquetes del flujo, mientras que *Fwd Packets/s* únicamente considera el sentido de envío, se conserva la primera.

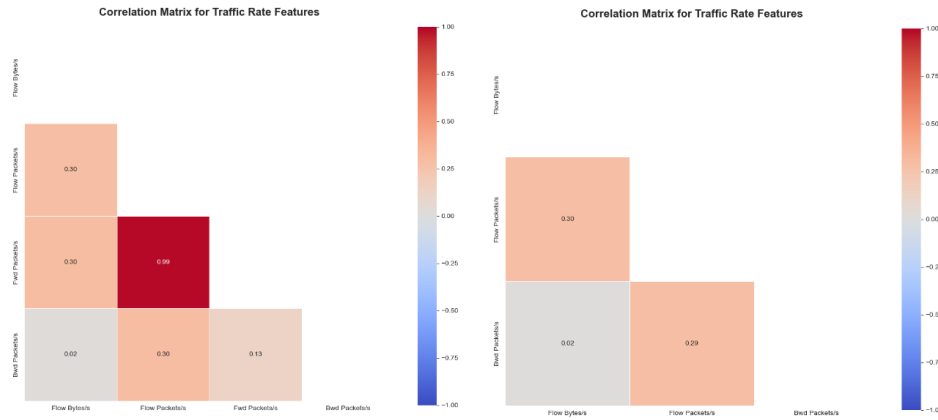


Figura 3.14: Matrices de correlación de tasas de transmisión pre/post eliminación

Por último, en este grupo confirmamos que ambas columnas están duplicadas al contener exactamente la misma información. Por tanto conservamos *Fwd Header Length*.

- *Fwd Header Length* $\longleftrightarrow$ *Fwd Header Length.1*: $r = 1.000$

Los tres grupos restantes de características (banderas de control TCP, ventanas TCP, características generales de conexión) no presentan parejas con una correlación superior al umbral establecido por lo que conservamos las variables. El resultado del análisis de correlación ha permitido eliminar variables duplicadas o casi duplicadas cuya información ya está representada por otras. Este procedimiento ha permitido reducir significativamente la dimensión del conjunto de datos y mejorar la interpretación de las variables sin provocar una pérdida de apreciable de información.

3.5.4. Transformación logarítmica

Como parte de la preparación del conjunto de datos completo de la segunda fase, hemos aplicado una transformación logarítmica sobre las características que presentaron una distribución fuertemente asimétrica. Además reduciremos la influencia de valores atípicos. Empleamos la función $\log(x+1)$ puesto que la mayor parte de las variables presentan valores iguales a cero. En caso de utilizar $\log(x)$ el resultado de $\log(0) = -\infty$. Cabe destacar que no hemos aplicado esta transformación a las variables correspondientes a las banderas TCP puesto que son variables binarias. Tampoco se ha aplicado la transformación a *Down/Up Ratio* ni *min_seg_size_forward* ya que sus distribuciones no son del todo asimétricas. Como resultado, se han transformado un total de **39 características**. En la figura 3.15 observamos el efecto que ha tenido la transformación para un grupo concreto de características.

En muchos casos encontramos un rango intercuartílico y una distribución mucho más equilibrada. Véase la diferencia con respecto a 3.10. Por último, cabe destacar que los valores atípicos no desaparecen pero su influencia disminuye.

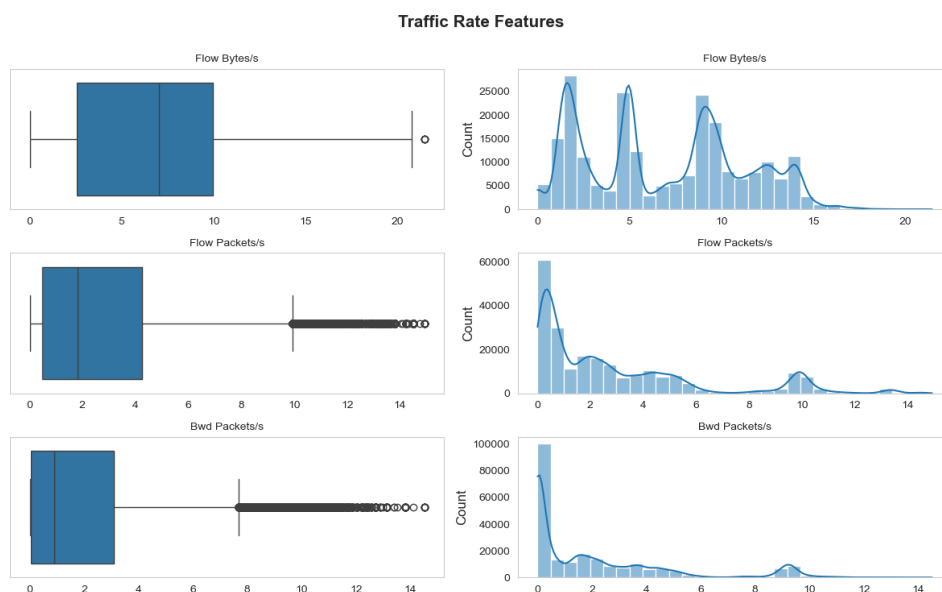


Figura 3.15: Resultado transformación logarítmica sobre características de tasas de transmisión

3.5.5. Escalado de características

Los conjuntos de datos de tráfico de red contienen características con mucho tipos de unidades y escalas, desde millones de bytes transmitidos hasta duraciones de flujos expresadas en microsegundos. Esta heterogeneidad implica aplicar un proceso de estandarización ya que estas diferencias de magnitud pueden introducir sesgos durante el proceso de aprendizaje para aquellos modelos sensibles a la escala como las redes neuronales multicapa, la regresión logística o las máquinas de vector soporte. Para evitar este problema se aplica una transformación de estandarización mediante la función *sklearn.preprocessing.StandardScaler* donde cada característica es transformada para que tenga media 0 y desviación típica 1:

$$z = (x - \mu) / \sigma$$

donde μ y σ son la media y desviación típica respectivamente. En la segunda fase el proceso de estandarización se realizará como un experimento específico de escalado sobre los conjuntos de características empleados por la

regresión logística y la red neuronal multicapa para analizar de forma aislada su efecto manteniendo el resto de condiciones experimentales constantes. Además, se aplicará exclusivamente al conjunto de entrenamiento para evitar una fuga de información. La media y desviación estándar obtenida en este conjunto se reutilizará para el conjunto test.

3.6. Particionado del conjunto de datos

Llegados a esta fase ya podemos concluir que tenemos varios conjuntos de datos realmente representativos y casi listos para el entrenamiento de nuestros modelos. A continuación, los dividiremos en diferentes subconjuntos independientes: un conjunto de entrenamiento para ajustar los parámetros internos de los modelos y un conjunto de test para estimar el rendimiento del modelo sobre datos nuevos. **La estrategia de división de cada subconjunto dependerá del tamaño del conjunto de datos disponible.** En nuestro caso, tenemos un conjunto de datos con un total de 8,000 muestras frente a otro con 225,745 muestras. Esto nos permite reducir el porcentaje destinado a test sin perder representatividad estadística en el conjunto mayor. Por lo tanto, emplearemos las siguientes proporciones: 90 % para entrenamiento y 10 % test para el conjunto de 225,745 muestras y 80 % entrenamiento y 20 % para el conjunto de 8,000 muestras. Además, el particionado se realizará de manera **aleatoria y estratificada** lo que significa que las muestras se distribuirán aleatoriamente entre los subconjuntos preservando la misma proporción de tráfico benigno y tráfico DDoS en cada uno de ellos.

Cabe destacar que no se ha utilizado una validación cruzada (*k-fold cross-validation*) como estrategia de evaluación, ya que las observaciones se distribuirían de manera aleatoria entre los diferentes *folds* pudiendo combinarse muestras pertenecientes a distintos momentos de la captura.

3.7. Construcción de conjuntos de entrenamiento

Una vez realizadas las diferentes estrategias de preparación de los datos vamos a construir los conjuntos de datos. De esta manera podemos analizar de forma consistente el impacto que tienen las diferentes técnicas de preprocesamiento sobre la capacidad predictiva, complejidad y coste computacional de los modelos.

En la primera fase, se han construido cuatro conjuntos de datos a partir del conjunto de referencia `numeric`. Sobre este conjunto se han aplicado únicamente las operaciones de limpieza de calidad de los datos. Además, solo contiene características numéricas. Sobre esta representación se construye `encoded` solo que se añade la representación codificada de las variables categóricas descrita anteriormente. Por último, `correlation_08` y

`correlation_09` parten del conjunto `encoded` y, mediante la eliminación automatizada de características, se obtienen ambos conjuntos de datos. Utilizamos umbrales de correlación de $R > 0.8$ y $R > 0.9$ respectivamente.

En la segunda fase, los conjuntos se construyen a partir del conjunto completo de 225,745 flujos de red. En primer lugar, `filtered` es el resultado de aplicar la limpieza universal y el filtro estructural derivado del análisis exploratorio. A partir de ese conjunto generamos `log`, en el que se ha aplicado la transformación logarítmica a las características seleccionadas. Por último, hemos construido un conjunto `correlation_098` que parte del conjunto `filtered`, sobre el que se ha realizado el análisis de correlación sobre grupos semánticos utilizando un umbral de $R > 0.98$ y seleccionando características mediante criterio experto. Además, se genera una variante escalada de este último conjunto para el experimento específico realizado con los modelos de regresión logística y MLP. Las tablas 3.14 y 3.15 resumen los conjuntos utilizados en la primera y segunda fase respectivamente.

Conjunto	Preparación	Nº atributos	Tamaño (MB)
<code>encoded</code>	Limpieza + codificación categórica	78	1.07
<code>numeric</code>	Limpieza universal	67	1.04
<code>correlation_09</code>	<code>encoded</code> + correlación ($R > 0.9$)	41	0.47
<code>correlation_08</code>	<code>encoded</code> + correlación ($R > 0.8$)	35	0.37

Cuadro 3.14: Conjuntos de características utilizados en la Fase I.

Conjunto	Preparación	Nº atributos	Tamaño (MB)
<code>filtered</code>	Limpieza universal + filtro estructural	59	23.91
<code>log</code>	<code>filtered</code> + transformación logarítmica	59	23.73
<code>correlation_098</code>	<code>filtered</code> + correlación ($R > 0.98$)	43	18.39
<code>correlation_098_scaled</code>	<code>correlation_098</code> + StandardScaler	43	18.39

Cuadro 3.15: Conjuntos de características utilizados en la Fase II.

Tras construir y almacenar los conjuntos de datos se ha implementado un procedimiento común para cargar y preparar cada uno antes de iniciar el entrenamiento: se recorre el conjunto de rutas definido para cada configuración experimental y, en cada iteración la función `load_dataset()` recibe la ruta del archivo `.parquet` y lo carga usando `pandas`. Seguidamente, `split_features_target()` divide la matriz de características X , con las variables predictoras, y el vector y , con la etiqueta *Label* a predecir. La variable *Label* se codifica numéricamente mediante un `sklearn.preprocessing.LabelEncoder` para trabajar con el formato adecuado en los modelos. Los resultados se almacenan en dos estructuras de datos: *X_SETS* e *Y_SETS*. El particionado y escalado se realizarán dentro del flujo de entrenamiento para evitar inconsistencias o fugas de información.

3.8. Entrenamiento

Una vez contruidos los diferentes conjuntos de características procedemos a explicar el proceso de entrenamiento de los modelos seleccionados. El objetivo de esta etapa es evaluar de forma sistemática y bajo unas condiciones experimentales homogéneas, la capacidad de los diferentes algoritmos para aprender patrones de tráfico benigno y malicioso.

3.8.1. Modelos seleccionados

Los modelos seleccionados tienen como objetivo representar diferentes paradigmas de aprendizaje automático. Para ello, se han organizado en tres categorías: modelos de referencia, modelos basados en árboles de decisión y redes neuronales. Esto permite analizar el rendimiento y efecto de los diferentes mecanismos de aprendizaje sobre el problema de detección de tráfico DDoS.

Regresión Logística: se trata de un modelo de referencia debido a su sencillez y bajo coste computacional, que servirá para establecer un punto de partida al problema de clasificación. La frontera de decisión permitirá saber si la separación entre el tráfico benigno y DDoS puede explicarse mediante relaciones lineales entre características.

Support Vector Classifier (SVC): establece una frontera de decisión entre las diferentes clases y, mediante el truco del *kernel*, representa relaciones no lineales del espacio de características. Con este modelo pretendemos saber si una frontera de separación basada en vectores soporte es adecuada para el problema. Por último, su sensibilidad a la escala nos permitirá concluir cómo ha variado el efecto de la estandarización en la preparación de los datos.

Árbol de decisión: este algoritmo ha sido seleccionado debido a su capacidad para moldear relaciones no lineales, su fácil interpretación, y su reducida sensibilidad a la escala de las características. Clasifican el tráfico mediante reglas. Además, son capaces de proporcionar qué características contribuyen en mayor o menor medida a la detección de ataques DDoS.

Perceptrón Multicapa (MLP): incorporamos este modelo como representante de las redes neuronales artificiales. El MLP es capaz de aprender representaciones no lineales permitiendo estudiar si existen relaciones complejas entre las características del tráfico de red. Se ha optado por utilizar el modelo más simple ya que el rendimiento de las redes neuronales en problemas de análisis de tráfico de red sobre datos tabulares es, en general, limitado [34]. Este modelo nos permitirá analizar el efecto que tiene el escalado de los datos y el proceso de optimización realizado sobre cada conjunto de datos.

3.8.2. Metodología de entrenamiento

Un aspecto fundamental es establecer una metodología experimental homogénea que facilite la evaluación de forma sistemática y automatizada de los distintos modelos de aprendizaje automático elegidos. Además, esta metodología también tiene como objetivo atribuir las diferencias observadas de los resultados al modelo o conjunto de datos empleado, en lugar de atribuirlos al proceso de entrenamiento. El algoritmo implementado en 1 realiza el mismo flujo de trabajo para cada combinación de modelo y conjunto de características. El proceso genera y almacena diferentes artefactos y resultados asociados a cada experimento para su posterior reutilización. Algunos aspectos interesantes de la función son:

- **Particionado de los datos:** cada conjunto de características se divide en entrenamiento y test. Utilizamos un valor fijo de *random_state* para garantizar la reproducibilidad de los experimentos.
- **Independencia entre experimentos:** en cada iteración de entrenamiento generamos una copia independiente del modelo mediante *sklearn.base.clone()*. Con esta función, los parámetros aprendidos durante un experimento no se reutilizan accidentalmente en los siguientes.
- **Fuga de información:** esta consideración constante por fin es tratada. Cuando se aplica escalado, el *sklearn.preprocessing.StandardScaler* se ajusta sobre X_{train} únicamente mediante *fit_transform()*. Posteriormente, la media y desviación estándar obtenidas se aplican al conjunto test mediante *transform()*.
- **Medición del tiempo de entrenamiento:** utilizamos la función *time.perf_counter()* para medir el tiempo transcurrido en segundos desde el inicio al fin del entrenamiento del modelo.
- **Evaluación durante el entrenamiento:** en cada iteración se calculan las métricas *Accuracy* y *F1* para cada conjunto de entrenamiento y validación.
- **Almacenamiento de resultados:** los modelos entrenados y los objetos *StandardScaler* son almacenados en formato Pickle mediante archivos *.pkl* para conservar los parámetros aprendidos sin necesidad de volver a entrenar. Los conjuntos X e y de *test* se almacenan en un formato proporcionado por Numpy (*.npz*) para facilitar la carga de los datos durante la fase de evaluación.

Algorithm 1 Model Training Pipeline

Require: X_{sets}, y_{sets} : datos y etiquetas; $models$: modelos; dir : directorio;

$scaler$: escalador; $test_size$: proporción test

Ensure: $saved$: modelos, artefactos y métricas de entrenamiento

```
1:  $models\_dir \leftarrow \text{createFolder}(dir)$ 
2:  $saved \leftarrow \emptyset$ 
3: for each  $(model\_name, model)$  in  $models.items()$  do
4:    $model\_dir \leftarrow \text{createFolder}(models\_dir / model\_name)$ 
5:   for each  $(feature\_set\_name, X)$  in  $X_{sets}.items()$  do
6:     if  $feature\_set\_name \notin y_{sets}$  then
7:       raise KeyError
8:     end if
9:      $y \leftarrow y_{sets}[feature\_set\_name]$ 
10:     $feature\_names \leftarrow \text{columns}(X)$ 
11:     $(X_{train}, X_{test}, y_{train}, y_{test}) \leftarrow$ 
12:       $\text{SplitData}(X, y, test\_size)$ 
13:    if  $scaler = \text{None}$  then
14:       $X_{train}^* \leftarrow X_{train}$ 
15:       $X_{test}^* \leftarrow X_{test}$ 
16:       $fitted\_scaler \leftarrow \text{None}$ 
17:    else
18:       $(X_{train}^*, X_{test}^*, fitted\_scaler) \leftarrow$ 
19:         $\text{ScaleData}(X_{train}, X_{test}, \text{Standard})$ 
20:    end if
21:     $(fitted\_model, train\_time) \leftarrow$ 
22:       $\text{TrainModel}(model, X_{train}^*, y_{train})$ 
23:     $train\_predictions \leftarrow$ 
24:       $fitted\_model.predict(X_{train}^*)$ 
25:     $train\_accuracy \leftarrow$ 
26:       $\text{Accuracy}(y_{train}, train\_predictions)$ 
27:     $train\_F1 \leftarrow$ 
28:       $F1(y_{train}, train\_predictions, \text{weighted})$ 
29:     $(model\_path, scaler\_path, test\_path) \leftarrow$ 
30:       $\text{SaveArtifacts}(fitted\_model, fitted\_scaler, X_{test}^*, y_{test})$ 
31:     $saved[(model\_name, feature\_set\_name)] \leftarrow \{$ 
32:       $\text{"model": } model\_path,$ 
33:       $\text{"scaler": } scaler\_path,$ 
34:       $\text{"test": } test\_path,$ 
35:       $\text{"n\_features": } |feature\_names|,$ 
36:       $\text{"feature\_names": } feature\_names,$ 
37:       $\text{"train\_time": } \text{round}(train\_time, 3),$ 
38:       $\text{"train\_accuracy": } train\_accuracy,$ 
39:       $\text{"train\_F1": } train\_F1$ 
40:     $\}$ 
41:  end for
42: end for
43: return  $saved$ 
```

3.9. Implementación

En esta sección se describen los detalles técnicos acerca de las tecnologías utilizadas, el entorno de desarrollo y la organización del código. Todos los *notebooks* desarrollados se encuentran disponibles públicamente en el repositorio del proyecto.

3.9.1. Tecnologías y entorno de desarrollo

El trabajo se ha desarrollado únicamente con el lenguaje de programación Python en su versión 3.13.9, pues es ampliamente utilizado en el aprendizaje automático. Las principales librerías y tecnologías utilizadas son:

- *Pandas*: manipulación y análisis de datos mediante estructuras como *DataFrames* y *Series*.
- *Numpy*: librería fundamental para operaciones numéricas y tratamiento de estructuras de datos.
- *Scikit-Learn*: librería de aprendizaje automático que incluye diferentes herramientas para tareas de clasificación, preprocesamiento, selección de modelos, etc.
- *Matplotlib*: usada para la creación de visualizaciones estáticas y animadas.
- *Seaborn*: construida a partir de *Matplotlib* proporcionando una interfaz de alto nivel que genera visualizaciones estadísticas.
- *Jupyter Notebook*: entorno interactivo ideal para el desarrollo de trabajos de aprendizaje automático ya que permite combinar código, texto y visualizaciones en un mismo *notebook*.
- *Joblib*: herramientas de almacenamiento en disco optimizada para trabajar con arrays de *NumPy*.
- *ipaddress*: módulo de la biblioteca estándar de Python para la manipulación de direcciones IPv4 e IPv6.

El desarrollo y entrenamiento se realizó en un portátil Victus Gaming Laptop 15-fa2047nr con sistema operativo Windows 11, con modelo de CPU 12th Gen Intel(R) Core(TM) i7-12700H (2.30 GHz), una RAM de 16GB de almacenamiento y un modelo de GPU NVIDIA GeForce RTX 3050.

3.9.2. Organización del proyecto

El código desarrollado se encuentra en un repositorio público en [GitHub](#) con el objetivo de facilitar la revisión de los tutores y de servir como copia de seguridad. Principalmente se ha hecho un uso local durante el desarrollo del proyecto debido a la constante naturaleza exploratoria del trabajo, en la que el código no estaba consolidado. Se ha organizado en dos niveles. Por un lado, los *notebooks* contienen los diferentes experimentos realizados junto con el flujo de trabajo seguido en cada etapa. Por otro lado, se han desarrollado funciones auxiliares que se han agrupado en módulos dentro de la carpeta `src` para evitar repetir implementaciones. La estructura general del proyecto es la siguiente:

- `data/`: carpeta con los diferentes conjuntos de datos según su nivel: `raw` para los datos originales extraídos de Kaggle, `intermediate` para los conjuntos de datos de referencia y completo, `processed` para los conjuntos construidos como resultados de los experimentos.
- `models/`: carpeta donde se almacenan los modelos, *scalers* y otros artefactos organizados en dos sub-carpetas según la fase experimental: `stage1` o `stage2`.
- `notebooks/`: carpeta que contiene los diferentes experimentos organizados de la siguiente manera:
 - `00-data-preparation/`: carga y preparación inicial de los datos. `01-dataset-construction.ipynb` construye los conjuntos de datos de trabajo de 8,000 y 225,000 muestras, mientras que `02-data-cleaning.ipynb` lleva a cabo la limpieza universal de los datos.
 - `01-EDA/`: análisis exploratorio datos: `03-exploratory-data-analysis.ipynb`.
 - `02-stage1/`: carpeta donde se encuentran los *notebooks* de la primera etapa experimental: `04-feature-engineering.ipynb`, `05-feature-selection.ipynb`, `06-comparative-modeling.ipynb`, `07-analysis.ipynb`
 - `03-stage2/`: carpeta donde se encuentran los *notebooks* de la segunda etapa experimental: `04-feature-engineering.ipynb`, `05-feature-selection.ipynb`, `06-experimental-feature-selection.ipynb`, `07-comparative-modeling.ipynb`, `08-analysis.ipynb`
- `results/`: resultados en formato *.json* obtenidos durante la experimentación.
- `src/`: código fuente con las diferentes funciones que se utilizarán en cada fase. Se dividen en módulos:

- `preprocessing.py`: carga, limpieza, particionado y codificación de los datos.
- `feature_engineering.py`: funciones relacionadas con la ingeniería de características como la codificación variables categóricas o la transformación logarítmica.
- `models.py`: configuración de los modelos utilizados.
- `train.py`: funciones relacionadas con el algoritmo de entrenamiento sistemático de los modelos.
- `evaluation.py`: funciones para evaluación modelos y cálculo de métricas.
- `utils.py`: funciones auxiliares.

Capítulo 4

Experimentación y Análisis de resultados

Una vez establecidas la metodología y las decisiones adoptadas durante el desarrollo del sistema, abordamos en este capítulo la fase de experimentación y análisis de resultados. En primer lugar, se presentan las métricas utilizadas para la evaluación del rendimiento de los modelos. A continuación, se detallarán los diferentes experimentos realizados en cada fase del proyecto, orientados primero, a establecer una línea base y después, a estudiar nuevas técnicas de optimización de los datos y nuevos modelos al aumentar el volumen de datos. A continuación, se analizarán los resultados estudiando cómo se comportan los distintos modelos sobre los distintos conjuntos de características, así como los errores de clasificación y experimentos adicionales. Finalmente, se realizará una comparación global con el objetivo de concluir qué configuración proporciona el mejor equilibrio para el problema de la detección de intrusiones en red.

4.1. Métricas

Para medir el rendimiento de un sistema de detección de intrusiones no solo se requiere el uso de métricas capaces de medir la exactitud global del clasificador, sino también se requiere medir la capacidad para minimizar el número de falsos positivos y falsos negativos. En ciberseguridad, cada error de tipo falso positivo o falso negativo tiene consecuencias totalmente diferentes. Un falso positivo implica clasificar erróneamente una actividad legítima como maliciosa lo que puede provocar una cantidad excesiva de alertas. Un falso negativo implica no detectar un ataque o vulnerabilidad lo que puede permitir que una actividad maliciosa pase inadvertida. Idealmente, buscamos reducir ambos errores pero en la práctica, la reducción de los falsos positivos puede implicar un aumento de los falsos negativos y viceversa. Es por ello que es necesario no limitarse sólo a la proporción global de

predicciones correctas, sino a considerar ambos tipos de error. [26]

Todas las métricas se calculan a partir de la matriz de confusión formada por: *True Positives* (TP) o instancias positivas correctamente clasificadas como positivas, *True Negatives* (TN) o instancias negativas correctamente clasificadas como negativas, *False Positives* (FP) o instancias negativas erróneamente clasificadas como positivas, *False Negatives* (FN) o instancias positivas erróneamente clasificadas como negativas.

4.1.1. F1 Score

El F1-Score es la media armónica entre *Precision* y *Recall*. Mientras que la media aritmética trata todos los valores como iguales, la armónica da importancia a los valores más bajos. Como consecuencia, el clasificador obtendrá un alto *F1-Score* si la precisión y sensibilidad son altos:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Donde:

- *Precision*: tasa de aciertos entre los predichos como positivos. Se calcula:

$$Precision = \frac{TP}{TP + FP}$$

- *Recall*: tasa de aciertos entre los reales positivos. Se calcula:

$$Recall = \frac{TP}{TP + FN}$$

4.1.2. Tasa de falsos positivos

La tasa de falsos positivos (en inglés, *False Positive Rate*, FPR) es la proporción de instancias de tráfico benigno que son clasificadas erróneamente como maliciosas. En los sistemas de detección de intrusiones de red, esta métrica muy útil ya que cuantifica el número de falsas alarmas generadas por el sistema:

$$FPR = \frac{FP}{FP + TN}$$

En la práctica, cualquier sistema de detección de intrusiones siempre va a tener problemas de falsos positivos. Cada falso positivo genera una alerta que posteriormente es enviada a un analista para que investigue la intrusión. Este aumento de falsos positivos puede favorecer la fatiga de alertas, haciendo cada vez más difícil distinguir las amenazas reales entre todas las alertas generadas [2]. En este trabajo, esta métrica debe ser utilizada conjuntamente con el F1-Score para evaluar tanto la capacidad de detección como el impacto de las falsas alarmas.

4.2. Diseño experimental

El diseño experimental se divide en dos fases o etapas diferenciadas, cada una con una finalidad metodológica concreta. Para garantizar una consistencia en la reproducibilidad de los resultados, se estableció una configuración inicial para cada uno de los modelos empleados: en la regresión logística se limita el número máximo de iteraciones a `max_iter=1000`. En SVC, se utiliza el kernel RBF y una penalización de `C=1.0`. En MLP, se utiliza una única capa oculta con 100 neuronas. La función de activación es ReLU y el algoritmo utilizado es Adam. Además, se limitan las iteraciones de entrenamiento a `max_iter=200`. Por último, para el árbol de decisión, se utiliza el criterio Gini para seleccionar las divisiones de los nodos (`criterion=gini`). A continuación, se ejecutan de manera sistemática las combinaciones de modelos y conjuntos de características definidos en cada fase. En la primera fase, el objetivo es establecer una línea base sobre un conjunto de datos de menor tamaño. Empleamos tres modelos de referencia (regresión logística, SVC y MLP) sobre cuatro representaciones diferentes de conjuntos de características (`numeric`, `encoded`, `correlation_08`, `correlation_09`). La combinación da lugar a un total de:

$$3 \text{ modelos} \times 4 \text{ conjuntos} = 12 \text{ experimentos}$$

En la segunda etapa se amplió el estudio utilizando el **conjunto de datos completo**. Se incorporaron algunas conclusiones obtenidas de la primera fase y aplicaremos diferentes técnicas de preparación y selección de características más elaboradas. Estas técnicas dan lugar a tres conjuntos de características derivados del conjunto completo: `filtered`, `log`, `correlation_098`. Además, se amplió el conjunto de modelos utilizado, incorporando *Decision Tree* a los tres modelos anteriores. De este modo la combinación da lugar a un total de:

$$4 \text{ modelos} \times 3 \text{ conjuntos} = 12 \text{ experimentos}$$

Adicionalmente se realizó un experimento específico de escalado sobre el conjunto `correlation_098` sobre los modelos regresión logística, SVC y MLP sensibles a la escala. Por tanto, el número de experimentos total realizado es de **27 experimentos** entre ambas fases más experimentos específicos.

4.3. Análisis e interpretación de resultados

Una vez definido el diseño experimental y completada la ejecución de los experimentos, pasamos al análisis de los resultados.

4.3.1. Fase I: establecimiento de la línea base

En la tabla 4.1 se muestran todos los resultados para los 12 experimentos realizados en la primera etapa. De manera general, los modelos seleccionados han sido capaces de obtener un rendimiento elevado sobre el conjunto de referencia de 8000 muestras.

Model	Features Count	Train Time	FP	FN	TP	TN	FP Rate	F1
Logistic Regression	67	2.056	93	6	794	707	0.116	0.9379
Logistic Regression	78	2.069	91	11	789	709	0.113	0.9360
Logistic Regression	35	2.155	7	14	786	793	0.008	0.9868
Logistic Regression	41	2.056	33	16	784	767	0.041	0.9693
MLP	67	0.850	18	5	795	782	0.022	0.9856
MLP	78	0.651	26	6	794	774	0.032	0.9799
MLP	35	1.030	38	14	786	762	0.047	0.9674
MLP	41	0.646	3	280	520	797	0.003	0.8176
SVC	67	0.570	84	87	713	716	0.105	0.8931
SVC	78	0.599	84	87	713	716	0.105	0.8931
SVC	35	0.507	73	126	674	727	0.091	0.8754
SVC	41	0.501	74	126	674	726	0.092	0.8748

Cuadro 4.1: Resultados de los modelos Fase I

Resultados generales

En la figura 4.1 podemos apreciar que el F1-score oscila entre 0.81 para MLP con el conjunto de características `correlation_09` y 0.98 para Regresión Logística con `correlation_08`. La regresión logística es el algoritmo

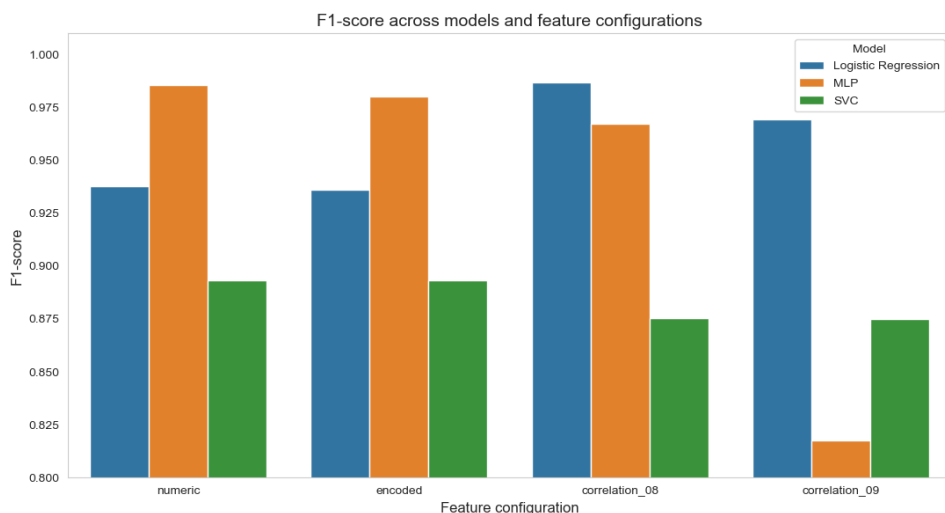


Figura 4.1: Comparación métrica F1

más destacado. Su comportamiento es estable y consigue, de media, mayor rendimiento. En la figura 4.2 observamos que la reducción de características

permite mejorar considerablemente la capacidad de clasificación. Conseguimos aumentar la métrica F1 de un 93 % para el conjunto `encoded` a un 98,6 % para el conjunto `correlation_08`. La tasa de falsos positivos también se ve beneficiada, reduciéndose de un 11 % en las configuraciones iniciales a un 0,88 % en el conjunto `correlation_08`. Sin embargo, los modelos MLP y

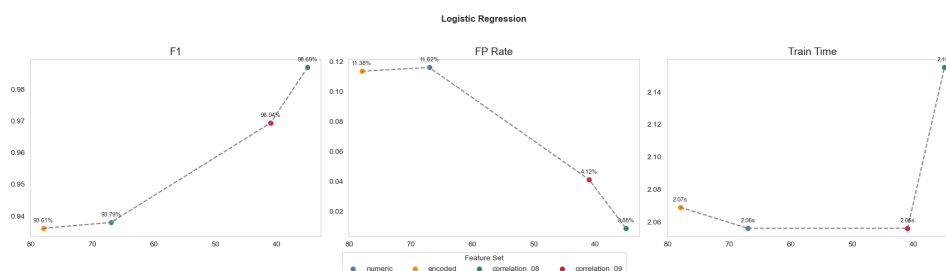


Figura 4.2: Efecto reducción características en regresión logística

SVC muestran una tendencia diferente. Por una parte, a pesar de que el modelo MLP también obtiene resultados competitivos, `numeric` con $F1=98,5\%$ y `encoded` con $F1=97,9\%$, la aplicación del filtro de correlación provoca una reducción del rendimiento acompañado de un aumento de la tasa de falsos negativos. Por otra parte, el modelo SVC presenta el comportamiento más estable. Tanto la configuración `numeric` como `encoded` obtienen mismos resultados. La reducción de dimensión no tiene un efecto del todo positivo ya que, a pesar de que se reducen el número de falsos positivos, el número de falsos negativos aumenta de 87 a 126. En consecuencia, la comparación general y el estudio de la **reducción de características tiene un efecto dependiente del modelo**. Para la regresión logística esta reducción es positiva mientras que para los modelos MLP y SVC supone una pérdida del rendimiento en términos generales.

Efecto de las variables categóricas

A continuación analizamos el efecto que tiene la inclusión de las variables categóricas codificadas. Para este análisis, comparamos las configuraciones `numeric` y `encoded` usando las métricas F1 y FPR. En la figura 4.3 observamos que la incorporación de estas variables tiene un efecto muy limitado. En la regresión logística la inclusión de estas variables supone una disminución de 0.25 puntos porcentuales en la tasa de falsos positivos. En el modelo MLP se observa que tiene un efecto negativo. En SVC los resultados permanecen idénticos. En consecuencia, el **incremento de 11 atributos no proporciona una mejora consistente del rendimiento**.

En conjunto, este primer análisis de resultados de la primera fase nos ha permitido marcar una **línea base experimental**, al igual que ha permitido identificar problemas que guiarán el diseño de la segunda fase. En primer

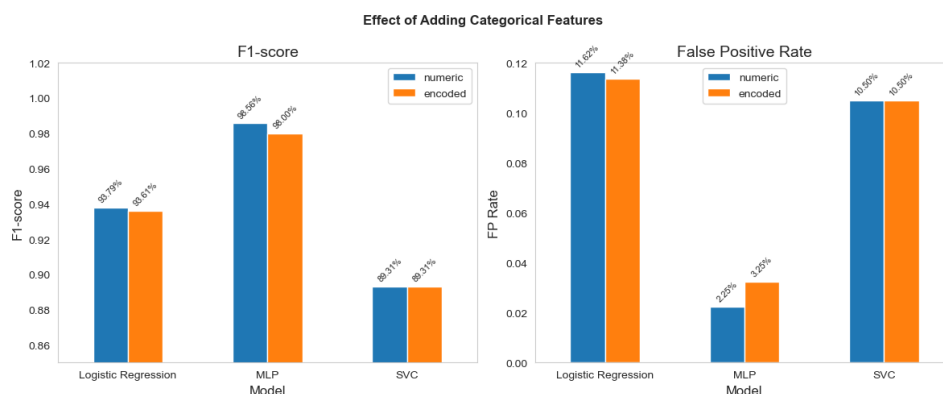


Figura 4.3: Resultado de las configuraciones **numeric** y **encoded**

lugar, la reducción de características tiene un efecto que depende de las características concretas seleccionadas y del modelo empleado. Por lo tanto, en la segunda fase, analizaremos de forma más granular esta selección para obtener un conjunto que permita mejorar el rendimiento de todos los modelos. En segundo lugar, descartaremos las variables categóricas ya que no han proporcionado beneficios en cuanto a rendimiento. Por último, incorporaremos un árbol de decisión que se suma a los modelos base con el objetivo de representar diferentes niveles de complejidad y obtener información sobre la importancia de las características.

4.3.2. Fase II: optimización sobre el conjunto completo

En la tabla 4.2 se muestran todos los resultados para los 12 experimentos realizados en la segunda fase. En términos generales, observamos un rendimiento mucho más heterogéneo que en la primera fase. El F1-score sobre el conjunto test oscila entre un 63%, correspondiente a SVC con el conjunto log, y un 99%, correspondiente a un árbol de decisión también sobre el conjunto log.

Model	Features Count	Train Time	FP	FN	FP Rate	F1
Decision Tree	59	2.748	9	9	0.0009	0.9992
Decision Tree	59	2.714	8	9	0.0008	0.9992
Decision Tree	43	2.147	10	9	0.0010	0.9991
Logistic Regression	59	21.358	811	116	0.0830	0.9586
Logistic Regression	59	20.611	586	12	0.0600	0.9733
Logistic Regression	43	20.229	769	215	0.0787	0.9562
MLP	59	38.152	576	17	0.0590	0.9736
MLP	59	76.496	300	84	0.0307	0.9829
MLP	43	40.355	437	18	0.0447	0.9797
SVC	59	4785.073	849	563	0.0869	0.9372
SVC	59	2243.156	6934	73	0.7104	0.6380
SVC	43	2898.864	787	543	0.0806	0.9409

Cuadro 4.2: Resultados de los modelos Fase II

Resultados generales

En la figura 4.4 observamos que los árboles de decisión son los modelos más consistentes, pues presentan un mejor comportamiento global. Alcanzan valores de F1 superiores al 99,9 % en cualquier tipo de representación con una tasa de falsos positivos por debajo de 0.1 %. Esto implica que la reducción de características mediante correlación resulta poco relevante para este modelo. Los modelos MLP y regresión logística se sitúan por detrás de los árboles de decisión aunque con un rendimiento también muy competitivo. Ambos obtienen sus mejores resultados con el conjunto `log`, un F1-score de un 98 % y de un 97 % y una tasa de falsos positivos de un 3.07 % y de un 6 % respectivamente. Finalmente, el modelo SVC obtiene resultados que no llegan a ser competitivos con el resto. La configuración `log` no tiene éxito en este modelo que se traduce en un F1 del 63 % y una tasa de falsos positivos del 71 %.

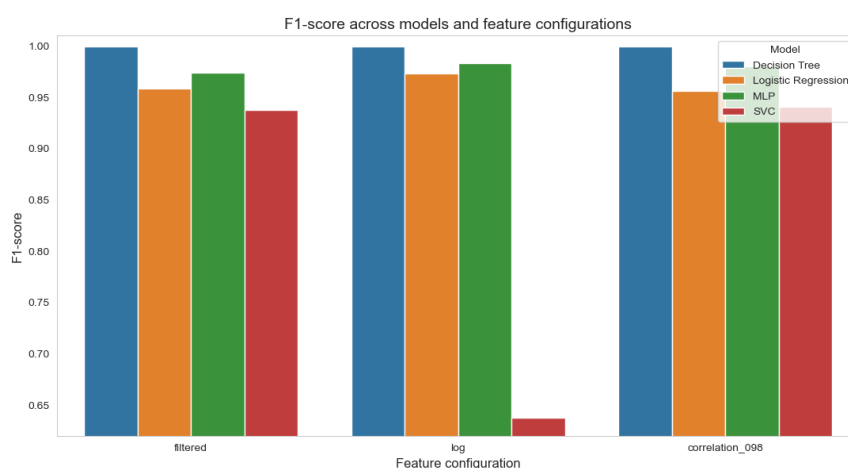


Figura 4.4: Comparación métrica F1

El gran coste computacional de las máquinas de vector soporte

En la tabla 4.3 se observa como el modelo SVC presenta un coste computacional de entrenamiento muy elevado para cualquier tipo de conjunto de datos. De media, el tiempo de entrenamiento requerido es 3,571.66 segundos, es decir, **casi 1 hora**. Esto implica que para utilizar este modelo sobre el conjunto de 225745 muestras sería necesario aplicar técnicas específicas de optimización.

Conjunto	Número de características	Tiempo de entrenamiento (s)
log	59	4316
filtered	59	3611
correlation_098	43	2788

Cuadro 4.3: Coste computacional SVC

Efecto de la transformación logarítmica

La transformación logarítmica realizada sobre el conjunto inicial **filtered** tiene un efecto positivo para todos los modelos. En la figura 4.5 se puede observar que dicha transformación permite reducir en 0.01 puntos porcentuales la tasa de falsos positivos en el árbol de decisión. Los modelos sensibles a la escala y representación de los datos son los que se ven más beneficiados con esta transformación. En los resultados de la regresión logística el número de falsos positivos pasa de 811 a 586, es decir se producen 225 falsas alarmas menos. Además, el número de falsos negativos también disminuye de 116 a 12, es decir se han eliminado 104 detecciones de ataques fallidas. En el modelo MLP, se produce una disminución en el número de falsos positivos de 576 a 300 pero aumentan a 67 los falsos negativos. Por ello, únicamente se produce una mejora de 0.94 puntos porcentuales en F1.

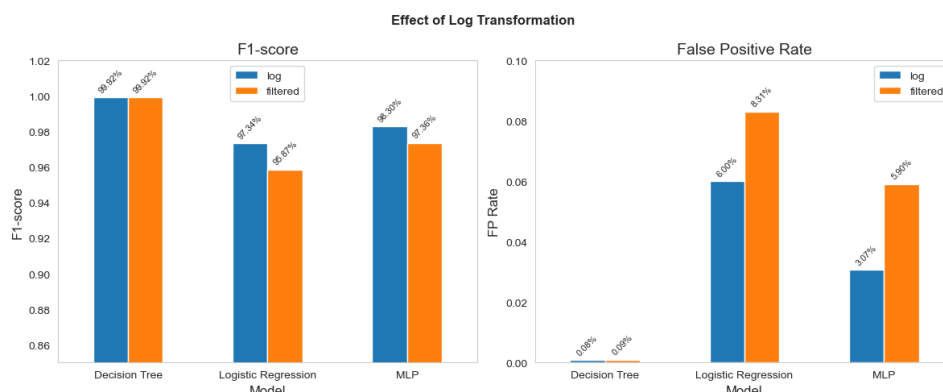


Figura 4.5: Resultados de la transformación logarítmica

Árbol de decisión e interpretabilidad

El estudio de interpretabilidad de los árboles de decisión permite identificar exactamente qué reglas se han usado para cada división de las ramas y la importancia de cada una de las características basadas en el índice de entropía de Gini. Estudiamos el árbol de decisión utilizando el conjunto de características `correlation_098`, al ser la configuración que mejor rendimiento ha obtenido. En la figura 4.6 se muestran 5 de 23 niveles del árbol de decisión. Observamos que la primera regla de clasificación divide el 30 % de las muestras en tráfico benigno mediante la variable *Fwd Packet Length Mean*. En el siguiente nivel le sigue la variable *Total Length of Fwd Packets* con el 70 % restante de las muestras. A continuación, el árbol continúa realizando divisiones utilizando otras características como *Bwd Header Length*, *Bwd Packet Length Min*, *Total Length of Fwd Packets* o *Flow IAT Min*.

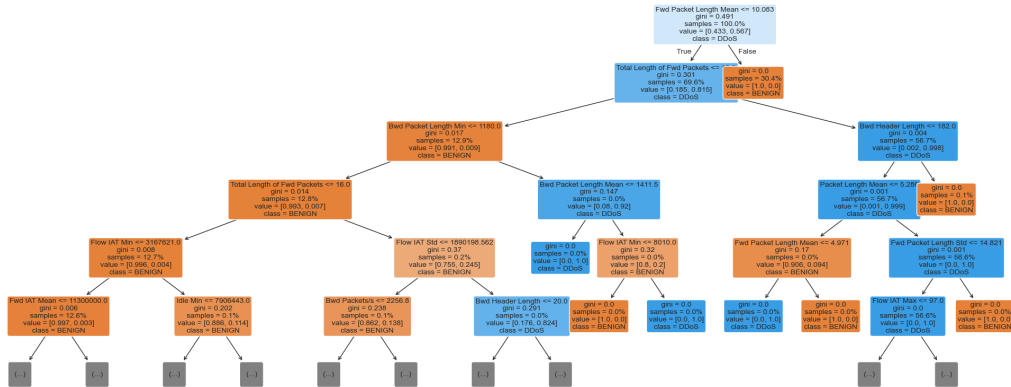


Figura 4.6: Árbol de decisión sobre conjunto `correlation_098`

En la figura 4.7 se aprecia que las 3 características con mayor importancia son *Fwd Packet Length Mean*, *Total Length of Fwd Packets* y *Bwd Header Length*. A pesar de que las dos primeras concentren prácticamente toda la importancia no debe interpretarse que ambas son suficientes por sí solas para obtener el rendimiento observado. Como se ha demostrado anteriormente en 2.3.2, el objetivo de un IDS/NIDS no es sólo identificar correctamente los comportamientos maliciosos sino también es mantener una tasa de falsas alarmas extremadamente baja. Por ello, **eliminar el resto de características de menor importancia perjudicaría la capacidad del modelo para distinguir casos ambiguos que determinan la aparición de falsas alarmas.**

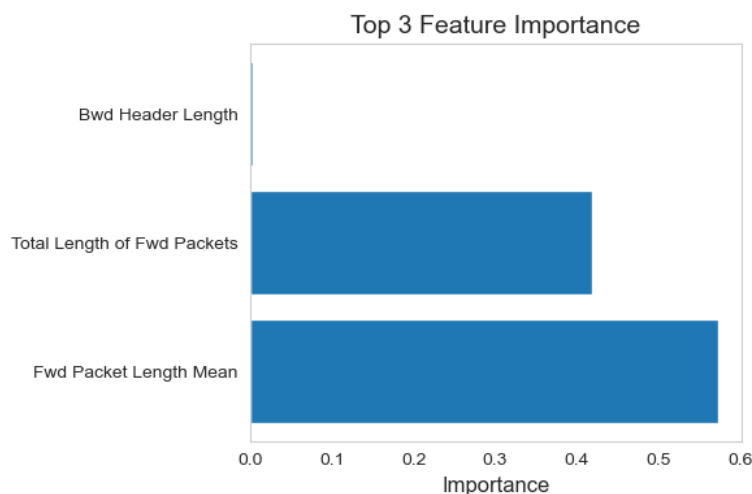


Figura 4.7: Top 3 características árbol decisión

4.3.3. Experimentos adicionales: efecto de la estandarización

Durante la experimentación anterior, la regresión logística, MLP y SVC no son capaces de obtener el mismo rendimiento fiable que los árboles de decisión. La mejor tasa de falsos positivos obtenida por estos 3 modelos es superior al 3 %, es decir, tenemos 300 instancias categorizadas como falsas alarmas. Para reducir este número se ha planteado la hipótesis de que las diferencias de magnitud entre las características de tráfico de red siguen afectando al proceso de aprendizaje. Para comprobarlo se ha realizado un nuevo entrenamiento usando el conjunto `correlation_098` de 43 características pero esta vez aplicando una estandarización mediante `sklearn.preprocessing.StandardScaler`. En la tabla 4.4 se puede observar que nuestra **hipótesis planteada es correcta**. Los tres modelos superan un F1-score del 99 % con unas tasas de falsos positivos por debajo del 0.2 %. El modelo MLP obtiene el mejor resultado con únicamente 7 falsos positivos y 13 falsos negativos. Además, el efecto es especialmente notable en SVC desde el punto de vista de coste computacional. Hemos reducido el tiempo de entrenamiento en aproximadamente un 98 % pasando de 2788 segundos a 41 segundos. Estos resultados serán considerados en la comparación final.

Model	Train Time	FP	FN	TP	TN	FP Rate	F1
Logistic Regression	3.763	24	16	12785	9736	0.002459	0.998227
SVC	41.194	17	15	12786	9743	0.001742	0.998582
MLP	23.523	7	13	12788	9753	0.000717	0.999114

Cuadro 4.4: Resultados de los modelos con el conjunto de características `correlation_098_scaled`

4.4. Comparación final de configuraciones

En esta sección realizaremos una comparación global de todas las configuraciones con el objetivo de determinar cuál de ellas presenta el mejor equilibrio entre cinco criterios: rendimiento, control de errores, coste computacional, dimensión e interpretabilidad. A cada criterio se normalizará a un intervalo entre $[0,1]$ y se le asociará un peso dependiendo de su importancia. En primer lugar hemos asociado bajo criterio experto a la métrica F1 el mayor peso con un 50 %. Le sigue la tasa de falsos positivos con un 20 %, el número de características con un 10 %, el tiempo de entrenamiento con otro 10 % y la interpretabilidad del modelo con un 10 %.

En la tabla 4.5 se observa un claro éxito de la reducción de características realizada ya que, la parte superior de la tabla está liderada por el conjunto de características `correlation_098` junto con sus variantes estandarizadas. Observamos que el **árbol de decisión con correlation_098** es el claro ganador gracias a su elevado rendimiento, su bajo coste computacional y su elevada interpretabilidad.

	Model	Feature Set	F1	FP Rate	Train Time
1	Decision Tree	<code>correlation_098</code>	0.9992	0.0010	2.139
2	Logistic Regression	<code>correlation_098_scaled</code>	0.9982	0.0025	3.763
3	MLP	<code>correlation_098_scaled</code>	0.9991	0.0007	23.523

Cuadro 4.5: Ranking final

Capítulo 5

Conclusiones y trabajo a futuro

En la actualidad, los actores maliciosos realizan ataques cada vez más sofisticados utilizando nuevos vectores y técnicas de ataque. Esta evolución dificulta la utilización de mecanismos de detección basados en reglas estáticas, y hace necesario el uso de otras técnicas que identifiquen los comportamientos que no hayan sido contemplados nunca mediante mecanismos de detección robustos con una gran capacidad de generalización. En este Trabajo Fin de Grado, se ha propuesto y evaluado un sistema de detección de tráfico DDoS riguroso basado en técnicas de aprendizaje automático. Además, se han analizado los diferentes comportamientos atendiendo a diferentes métricas, como la capacidad de detección o el coste computacional, con el objetivo de determinar su viabilidad en un entorno NIDS real.

5.1. Conclusiones

La evaluación de los modelos desarrollados en este trabajo se ha realizado mediante métricas cuantitativas orientadas a diferentes aspectos del comportamiento del sistema. Los modelos de Regresión Logística, Árbol de decisión, Máquinas de vector soporte y Perceptrón Multicapa, se han evaluado usando métricas como Accuracy, Tasa de falsos positivos, F1-Score, Precisión y Recall, entre otras. Tras entrenar cada modelo sobre diferentes conjuntos de datos y evaluarlos sobre una partición de prueba, se ha observado que el modelo que ofrece mejor rendimiento y equilibrio con respecto a otras métricas es el árbol de decisión sobre el conjunto `correlation_098`, con un F1-Score de 99,92% y una tasa de falsos positivos del 0,1%. Este resultado junto con el resto, indica que se ha mantenido un rendimiento de detección muy elevado, en el que conseguimos diferenciar entre tráfico benigno y DDoS mediante un modelo con una estructura sencilla e interpretable. No obstante, no se puede pasar por alto la dificultad inherente del

problema de detección de ataques de red. El tráfico de una red real contiene diversos comportamientos de usuarios, pero sólo un tipo de ataque DDoS. Otros ataques DDoS se llevan a cabo en otras capas del modelo OSI, con una naturaleza, intensidad y objetivo diferente. De esta manera, la obtención de métricas elevadas constituye un resultado favorable, pero no garantiza un rendimiento universal ante tráfico completamente desconocido.

Este resultado refuerza las principales conclusiones del trabajo:

- Se han diseñado cuatro modelos de aprendizaje supervisado automático: Regresión Logística, Árbol de Decisión, Máquina de Vector Soporte y Perceptrón Multicapa. Cada uno de ellos, ha sido entrenado sobre el conjunto de datos público *CIC-DDoS2019 Evaluation Dataset*.
- Se han aplicado diferentes técnicas de preprocesamiento de los datos e ingeniería de características debido a la naturaleza del tráfico de red, que contiene una alta variabilidad y heterogeneidad. Se han estudiado, entre otras, la transformación logarítmica o la selección de características a partir de un análisis de correlación. El análisis realizado muestra que el efecto de estas técnicas depende del algoritmo empleado.
- Se han ajustado y comparado los modelos utilizando métricas orientadas a la capacidad de detección (F1-score) y métricas orientadas a la generación de falsos positivos (FPR). La comparación de estas métricas ha permitido determinar las configuraciones con mejor equilibrio entre ambas.
- Se ha evaluado el rendimiento de cada modelo mediante un análisis de los resultados donde se muestran tablas e imágenes que estudian los efectos de las transformaciones de datos y las diferencias entre los cuatro modelos desarrollados. Los resultados han permitido el estudio de su aplicabilidad en entornos reales, considerando, entre otros aspectos, el coste computacional.

Las ventajas del trabajo realizado son notables según aspectos. Una clara ventaja es el elevado rendimiento obtenido por los modelos tras la experimentación y la aplicación de técnicas de optimización para el tráfico de red. La simplicidad estructural del modelo seleccionado es otra ventaja relevante ya que, en el ámbito de la ciberseguridad, no sólo basta identificar la actividad maliciosa sino también es necesario analizar por qué se ha producido. Desde el punto de vista del desarrollo, la metodología empleada también supone una clara ventaja puesto que, la experimentación escalonada ha permitido evaluar diferentes alternativas sobre un conjunto reducido para posteriormente, trasladarlas a un conjunto de datos mayor. Sin embargo, este traslado de conclusiones corre el riesgo de perder una parte de la variabilidad de los datos originales. Asimismo, la reducción del conjunto de características presenta otra ventaja desde el punto de vista computacional,

ya que se ha evitado disminuir el rendimiento de los modelos a la vez que se ha facilitado su aplicación en sistemas donde los recursos disponibles o requisitos estén limitados. Sin embargo, el sistema también presenta diversas desventajas y limitaciones. Entre ellas, destaca la falta de entrenamiento y evaluación del modelo ante nuevas variantes de ataques DDoS. Otra desventaja, es la posible pérdida de información relevante debido al proceso de ingeniería de características. La aplicación de este sistema a un entorno diferente podría requerir un nuevo análisis del conjunto de datos de tráfico de red. Por último, no se ha determinado al completo la viabilidad de los modelos en un entorno de tiempo real de un NIDS.

Este trabajo no se ha limitado a implementar un modelo de aprendizaje automático, sino que ha requerido de la integración de diferentes áreas de conocimiento relacionadas con la ingeniería informática. Entre ellas, destacan los aspectos de seguridad y redes, técnicas de rendimiento y eficiencia computacional, técnicas de ciencia de datos, técnicas de aprendizaje automático, etcétera. De manera resumida, todo el desarrollo del TFG ha permitido consolidar tanto competencias técnicas adquiridas a lo largo de la titulación, como otras capacidades fundamentales para el desarrollo profesional como son: el desarrollo de una capacidad de diagnóstico, el desarrollo de una mentalidad de análisis crítico, una adaptación ante imprevistos técnicos y una resolución autónoma de problemas.

5.2. Trabajo a futuro

En próximas etapas podría desarrollarse e investigarse técnicas de aprendizaje no supervisado aplicadas a la detección de intrusiones. Este enfoque permitiría estudiar la capacidad de identificar patrones anómalos reduciendo la dependencia de conjuntos de datos etiquetados. Además, permitiría abordar escenarios en los que no se dispone de información suficiente sobre los ataques que se pretenden detectar. Otra vertiente sería incorporar técnicas que se acerquen más a las condiciones de un entorno real como por ejemplo la incorporación de una partición temporal en la que el modelo aprenda patrones de tráfico del pasado para detectar patrones del futuro y así, aumentar su capacidad de generalización. Finalmente, sería necesario evaluar la capacidad de generalización frente a otro tipo de ataques DDoS que tienen como objetivo otros servicios y protocolos, como WebDDoS, TCP SYN o UDP Flood. De esta manera podríamos determinar si las características seleccionadas representan patrones generales de tráfico malicioso o patrones específicos del conjunto CIC-DDoS2019.

Más allá de los datos y modelos, otra vertiente interesante sería el traslado del modelo desarrollado a un entorno de producción mediante técnicas de MLOps. Sería necesario controlar las versiones de los datos y modelos para su despliegue o automatizar los procesos de entrenamiento y validación.

Además se deberían de incorporar mecanismos de monitorización para detectar aspectos como la aparición de nuevas amenazas o la degradación del rendimiento del modelo. La incorporación de estas técnicas permitiría que un modelo evaluado experimentalmente sobre un conjunto de datos estático se traslade hacia un sistema de detección capaz de operar de forma continua.

5.3. Viabilidad comercial y consideraciones de sostenibilidad

Este trabajo ha estudiado la importancia y necesidad del desarrollo de soluciones frente a los ataques DDoS debido a su capacidad para comprometer la disponibilidad de servicios y recursos de red. A día de hoy, las soluciones mayormente adoptadas que detectan ataques de denegación de servicio se basan en estrategias de prueba y error: *establece un límite máximo de 500 peticiones por IP*. Sin embargo, estas medidas no son efectivas durante un ataque DDoS puesto que el tráfico puede que no se desvíe del tráfico legítimo. Si establecemos este límite muy bajo, los usuarios normales pueden verse afectados, pero si lo elevamos mucho, el ataque puede situarse por debajo del mismo. Como resultado, las compañías que quieran ofrecer soluciones ante la protección de ataques DDoS deberían poner el foco de atención sobre el desarrollo de nuevos métodos basados en los comportamientos más genuinos de los ataques DDoS ¹ En este contexto, la solución desarrollada presenta un posible potencial para ser transformada en un producto comercial puesto que, las técnicas de aprendizaje automático, permiten detectar relaciones y características intrínsecas del tráfico de red que no dependen exclusivamente de reglas o umbrales previamente establecidos. La solución podría aplicarse en un sistema de detección especializado dentro de un SOC (*Security Operations Center*) que gestione alertas en entornos reales.

Genéricamente añadir que dada la importancia de los Objetivos de Desarrollo Sostenible (ODS), que inciden en poner fin a la pobreza, proteger el planeta y garantizar que para el 2030 todas las personas disfruten de paz y prosperidad, interesa que se aporte creatividad, conocimiento, tecnología y recursos financieros. En nuestro caso entendemos que este trabajo se alinea principalmente con el Objetivo 9 (Industria, innovación e infraestructura), al desarrollar una tecnología que mejora y moderniza la disponibilidad de una infraestructura o servicio digital y se presenta una solución innovadora basada en inteligencia artificial sobre un problema real.

¹Fuente: [Blog Mnemonic](#).

Bibliografía

- [1] Emad E. Abdallah, Wafa' Eleisah y Ahmed Fawzi Otoom. "Intrusion Detection Systems using Supervised Machine Learning Techniques: A survey". En: *Procedia Computer Science* 201 (2022). The 13th International Conference on Ambient Systems, Networks and Technologies (ANT) / The 5th International Conference on Emerging Data and Industry 4.0 (EDI40), págs. 205-212. ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2022.03.029>. URL: <https://www.sciencedirect.com/science/article/pii/S1877050922004422>.
- [2] Omar Abouabdalla et al. "False positive reduction in intrusion detection system: A survey". En: *2009 2nd IEEE International Conference on Broadband Network & Multimedia Technology*. 2009, págs. 463-466. DOI: [10.1109/ICBNMT.2009.5348536](https://doi.org/10.1109/ICBNMT.2009.5348536).
- [3] Ethem Alpaydin. *Introduction to Machine Learning*. 4.^a ed. MIT Press, 2020. URL: <https://mitpress.mit.edu/9780262043793/introduction-to-machine-learning/>.
- [4] Stefan Axelsson. "The base-rate fallacy and the difficulty of intrusion detection". En: *ACM Trans. Inf. Syst. Secur.* 3.3 (ago. de 2000), págs. 186-205. ISSN: 1094-9224. DOI: [10.1145/357830.357849](https://doi.org/10.1145/357830.357849). URL: <https://doi.org/10.1145/357830.357849>.
- [5] Zahedi Azam, Md Motaharul Islam y Mohammad Nurul Huda. "Comparative analysis of intrusion detection systems and machine learning-based model analysis through decision tree". En: *Ieee Access* 11 (2023), págs. 80348-80391.
- [6] Manjula C. Belavagi y Balachandra Muniyal. "Performance Evaluation of Supervised Machine Learning Algorithms for Intrusion Detection". En: *Procedia Computer Science* 89 (2016). Twelfth International Conference on Communication Networks, ICCN 2016, August 19– 21, 2016, Bangalore, India Twelfth International Conference on Data Mining and Warehousing, ICDMW 2016, August 19-21, 2016, Bangalore, India Twelfth International Conference on Image and Signal Processing, ICISP 2016, August 19-21, 2016, Bangalore, India, págs. 117-123. ISSN: 1877-0509. DOI: <https://doi.org/10.1016/j.procs.2016>.

- 06.016. URL: <https://www.sciencedirect.com/science/article/pii/S187705091631081X>.
- [7] Chih-Chung Chang y Chih-Jen Lin. “LIBSVM: A library for support vector machines”. En: *ACM Trans. Intell. Syst. Technol.* 2.3 (mayo de 2011). ISSN: 2157-6904. DOI: [10.1145/1961189.1961199](https://doi.org/10.1145/1961189.1961199). URL: <https://doi.org/10.1145/1961189.1961199>.
 - [8] Bahzad Charbuty y Adnan Abdulazeez. “Classification based on decision tree algorithm for machine learning”. En: *Journal of applied science and technology trends* 2.01 (2021), págs. 20-28.
 - [9] Jerome Cornfield, Tavia Gordon y Willie W Smith. “Quantal response curves for experimentally uncontrolled variables”. En: *Bull Int Stat Inst* 38.3 (1961), págs. 97-115.
 - [10] Lucas Costa et al. “Multilayer Perceptron”. En: *Introduction to Computational Intelligence*. IEEE Computational Intelligence Society, 2023. Cap. 9, págs. 105-116. DOI: [10.5281/zenodo.7537827](https://doi.org/10.5281/zenodo.7537827).
 - [11] Christos Douligeris y Aikaterini Mitrokotsa. “DDoS attacks and defense mechanisms: classification and state-of-the-art”. En: *Computer Networks* 44.5 (2004), págs. 643-666. ISSN: 1389-1286. DOI: <https://doi.org/10.1016/j.comnet.2003.10.003>. URL: <https://www.sciencedirect.com/science/article/pii/S1389128603004250>.
 - [12] Wesley Eddy. *TCP SYN flooding attacks and common mitigations*. Inf. téc. 2007.
 - [13] European Union Agency for Cybersecurity (ENISA). *ENISA Threat Landscape 2025*. 2025. URL: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025>.
 - [14] Behrouz A. Forouzan. *TCP/IP Protocol Suite*. 4.^a ed. McGraw-Hill, 2010. ISBN: 9780073376042.
 - [15] Aurelien Geron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, Inc., 2022.
 - [16] Waheed A. H. M. Ghanem y Aman Jantan. “A new approach for intrusion detection system based on training multilayer perceptron by using enhanced Bat algorithm”. En: *Neural Computing and Applications* 32.15 (ago. de 2020). Published online 2020-08-01, págs. 11665-11698. ISSN: 1433-3058. DOI: [10.1007/s00521-019-04655-2](https://doi.org/10.1007/s00521-019-04655-2). URL: <https://doi.org/10.1007/s00521-019-04655-2>.
 - [17] Neha Gupta et al. “DDoS attack algorithm using ICMP flood”. En: *2016 3rd International Conference on Computing for Sustainable Global Development (INDIACom)*. 2016, págs. 4082-4084.

- [18] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. Springer, 2009. DOI: [10.1007/978-0-387-84858-7](https://doi.org/10.1007/978-0-387-84858-7).
- [19] Nazrul Hoque, Dhruba K. Bhattacharyya y Jugal K. Kalita. “Botnet in DDoS Attacks: Trends and Challenges”. En: *IEEE Communications Surveys and Tutorials* 17.4 (2015), págs. 2242-2270. DOI: [10.1109/COMST.2015.2457491](https://doi.org/10.1109/COMST.2015.2457491).
- [20] Shuyuan Jin y D.S. Yeung. “A covariance analysis model for DDoS attack detection”. En: *2004 IEEE International Conference on Communications (IEEE Cat. No.04CH37577)*. Vol. 4. 2004, 1882-1886 Vol.4. DOI: [10.1109/ICC.2004.1312847](https://doi.org/10.1109/ICC.2004.1312847).
- [21] Joseph Migga Kizza. *Guide to Computer Network Security*. 6.^a ed. Texts in Computer Science. Hardcover ISBN: 978-3-031-47548-1; Softcover ISBN: 978-3-031-47743-0. Cham, Switzerland: Springer Cham, 2024, págs. XXVI, 646. ISBN: 978-3-031-47549-8. DOI: [10.1007/978-3-031-47549-8](https://doi.org/10.1007/978-3-031-47549-8). URL: <https://doi.org/10.1007/978-3-031-47549-8>.
- [22] Maxime Lanvin et al. “Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes”. En: *Risks and Security of Internet and Systems*. Ed. por Slim Kallel et al. Cham: Springer Nature Switzerland, 2023, págs. 18-33. ISBN: 978-3-031-31108-6.
- [23] Hieu Le Ngoc. “Application of representation learning in detecting botnet attacks”. En: *Scientific Reports* 16.1, 11977 (4 de mar. de 2026). DOI: [10.1038/s41598-026-40172-8](https://doi.org/10.1038/s41598-026-40172-8).
- [24] Lisa Liu et al. “Error Prevalence in NIDS datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018”. En: *2022 IEEE Conference on Communications and Network Security (CNS)*. 2022, págs. 254-262. DOI: [10.1109/CNS56114.2022.9947235](https://doi.org/10.1109/CNS56114.2022.9947235).
- [25] Fernando Martínez-Plumed et al. “CRISP-DM Twenty Years Later: From Data Mining Processes to Data Science Trajectories”. En: *IEEE Transactions on Knowledge and Data Engineering* 33.8 (2021), págs. 3048-3061. DOI: [10.1109/TKDE.2019.2962680](https://doi.org/10.1109/TKDE.2019.2962680).
- [26] Manish Marwah et al. *Is F_1 Score Suboptimal for Cybersecurity Models? Introducing C_{score} , a Cost-Aware Alternative for Model Assessment*. Jul. de 2024. DOI: [10.48550/arXiv.2407.14664](https://arxiv.org/abs/10.48550/arXiv.2407.14664).
- [27] Mohammad Masdari y Marzie Jalali. “A survey and taxonomy of DoS attacks in cloud computing”. En: *Security and Communication Networks* 9.16 (2016), págs. 3724-3751. DOI: <https://doi.org/10.1002/sec.1539>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/https://doi.org/10.1002/sec.1539>.

- 10.1002/sec.1539. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/sec.1539>.
- [28] Marvin Minsky y Seymour Papert. "An introduction to computational geometry". En: *Cambridge tiass., HIT* 479.480 (1969), pág. 104.
- [29] Jelena Mirkovic y Peter Reiher. "A taxonomy of DDoS attack and DDoS defense mechanisms". En: *SIGCOMM Comput. Commun. Rev.* 34.2 (abr. de 2004), págs. 39-53. ISSN: 0146-4833. DOI: [10.1145/997150.997156](https://doi.org/10.1145/997150.997156). URL: <https://doi.org/10.1145/997150.997156>.
- [30] Stack Overflow. *Remove highly correlated columns from a pandas dataframe*. <https://stackoverflow.com/questions/44889508/remove-highly-correlated-columns-from-a-pandas-dataframe>. Consulta: 28 de agosto de 2026.
- [31] Alexis Rosen Owner. "Understanding and Defending Against SYN Attacks". En: ().
- [32] Burak Öztornaci, Baris Ata y Serkan Kartal. "Analysing Household Food Consumption in Turkey Using Machine Learning Techniques". En: *Agris on-line Papers in Economics and Informatics* 16 (jun. de 2024), págs. 97-105. DOI: [10.7160/aol.2024.160207](https://doi.org/10.7160/aol.2024.160207).
- [33] Karl Pearson. "Notes on regression and inheritance in the case of two parents". En: *Proceedings of the Royal Society of London* 58 (1895), págs. 240-242. DOI: [10.1098/rspl.1895.0041](https://doi.org/10.1098/rspl.1895.0041).
- [34] Adrian Pekar, Richard Plny y Karel Hynek. *Tutorial on Flow-Based Network Traffic Classification Using Machine Learning*. 2026. arXiv: [2601.04089](https://arxiv.org/abs/2601.04089) [cs.NI]. URL: <https://arxiv.org/abs/2601.04089>.
- [35] R Documentation. *findCorrelation: Find Correlations*. <https://rdrr.io/cran/caret/src/R/findCorrelation.R>. Código fuente del paquete caret. Consulta: 28 de agosto de 2026.
- [36] Iman Sharafaldin et al. "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy". En: *2019 International Carnahan Conference on Security Technology (ICCST)*. 2019, págs. 1-8. DOI: [10.1109/CCST.2019.8888419](https://doi.org/10.1109/CCST.2019.8888419).
- [37] Rajat Tandon. *A Survey of Distributed Denial of Service Attacks and Defenses*. 2020. arXiv: [2008.01345](https://arxiv.org/abs/2008.01345) [cs.CR]. URL: <https://arxiv.org/abs/2008.01345>.
- [38] Nikhil Tripathi y Neminath Hubballi. "Slow rate denial of service attacks against HTTP/2 and detection". En: *Computers & Security* 72 (2018), págs. 255-272. ISSN: 0167-4048. DOI: <https://doi.org/10.1016/j.cose.2017.09.009>. URL: <https://www.sciencedirect.com/science/article/pii/S0167404817301980>.

- [39] Chih-Fong Tsai et al. "Intrusion detection by machine learning: A review". En: *Expert Systems with Applications* 36.10 (2009), págs. 11994-12000. ISSN: 0957-4174. DOI: <https://doi.org/10.1016/j.eswa.2009.05.029>. URL: <https://www.sciencedirect.com/science/article/pii/S0957417409004801>.
- [40] Dirk Valkenborg et al. "Support vector machines". En: *American Journal of Orthodontics and Dentofacial Orthopedics* 164.5 (2023), págs. 754-757. ISSN: 0889-5406. DOI: [10.1016/j.ajodo.2023.08.003](https://doi.org/10.1016/j.ajodo.2023.08.003). URL: <https://doi.org/10.1016/j.ajodo.2023.08.003>.
- [41] Rüdiger Wirth y Jochen Hipp. "CRISP-DM: Towards a standard process model for data mining". En: *Proceedings of the 4th international conference on the practical applications of knowledge discovery and data mining*. Vol. 1. Manchester. 2000, págs. 29-39.
- [42] Barry J. Wythoff. "Backpropagation neural networks: A tutorial". En: *Chemometrics and Intelligent Laboratory Systems* 18.2 (1993), págs. 115-155. ISSN: 0169-7439. DOI: [10.1016/0169-7439\(93\)80052-J](https://doi.org/10.1016/0169-7439(93)80052-J). URL: <https://www.sciencedirect.com/science/article/pii/016974399380052J>.